Rerandomizable Garbling, Revisited

Raphael Heitjohann, Jonas von der Heyden, and Tibor Jager

University of Wuppertal, Germany
{heitjohann,jvdh,jager}@uni-wuppertal.de

Abstract. In *key-and-message homomorphic encryption* (KMHE), the key space is a subset of the message space, allowing encryption of secret keys such that the *same* homomorphism can be applied to both the key and the message of a given ciphertext. KMHE with suitable security properties is the main building block for constructing *rerandomizable garbling schemes* (RGS, Gentry et al., CRYPTO 2010), which enable advanced cryptographic applications like multi-hop homomorphic encryption, the YOSO-like MPC protocol SCALES (Acharya et al., TCC 2022 and CRYPTO 2024), and more.

The BHHO scheme (Boneh et al., CRYPTO 2008) is currently the *only* known KMHE scheme suitable for constructing RGS. An encryption of a secret key consists of $\mathcal{O}(\lambda^2)$ group elements, where λ is the security parameter, which incurs a significant bandwidth and computational overhead that makes the scheme itself, and the RGS protocols building upon it, impractical.

We present a new, more efficient KMHE scheme with *linear*-size ciphertexts. Despite using heavier cryptographic tools (pairings instead of plain DDH-hard groups), the concrete ciphertext size and computational costs are very significantly reduced. We are able to shrink the ciphertext by 97.83 % (from 16.68 MB to 360 kB) and reduce the estimated computations for encryption by 99.996 % (from 4 minutes to 0.01 seconds) in comparison to BHHO.

Additionally, we introduce *gate KMHE* as a new tool to build more efficient RGS. Our RGS construction shrinks the size of a garbled gate by 98.99 % (from 133.43 MB to 1.35 MB) and decreases the estimated cost of garbling by 99.998 % (from 17 minutes to 0.02 seconds per gate) in comparison to Acharya et al. In summary, our work shows for the first time that RGS and the SCALES protocol (and hence YOSO-like MPC) are practically feasible for simple circuits.

1 Introduction

The BHHO encryption scheme [BHHO08] provides two very remarkable properties:

1. It achieves *circular security* in the standard model. Hence, it provides security even when a "key cycle" is encrypted, *i.e.*, secret key sk_i is encrypted under public key $\mathsf{pk}_{i \bmod n+1}$ for $i = 1, \ldots, n$.

2. Another distinguishing feature is that it is a *key-and-message homomorphic encryption* (KMHE) scheme. Thus, it enables efficiently computable homomorphisms in the key and the message space, such that in both spaces it is the *same* homomorphism. Furthermore, it provides *KMH rerandomizability*, which essentially guarantees that a rerandomized ciphertext is indistinguishable from a fresh one, even for the party that created the original ciphertext, and *key privacy under leakage*, which guarantees that this even holds when information about the rerandomization function is leaked.

The original work [BHHO08] mainly focused on circular security, but the unique combination of a key-and-message homomorphism with the aforementioned additional properties of the BHHO scheme has enabled many further, advanced applications, where circular security is not required. This includes *rerandomizable garbling schemes* (RGS) [GHV10], a rerandomizable version of Yao's garbled circuits [Yao86]. RGS enable applications such as i-hop homomorphic encryption [GHV10], secure computation without simultaneous interaction [HLP11], combiners for indistinguishability obfuscation (IO) schemes [AJN+16], reusable garbled circuits [ZLXL23], and (outsourced) multi-party computation (MPC) [AHKP22, AHKP24] in the YOSO ("you only speak once") paradigm [GHK+21].

Protocols in the YOSO paradigm are particularly interesting, as they allow for secure outsourced computation in the presence of a mobile adversary. Concretely, this means that lightweight input providers can outsource expensive MPC computations to a small committee of servers while maintaining security against a mobile adversary with corruption budget much larger than the size of the committee. This is due to the fact that servers are unknown until they speak once and then disappear again. Unfortunately, most known approaches to YOSO-MPC [GHK+21, KRY22, CDGK22] are still theoretical due to the use of expensive primitives, with the YOSO-like protocol SCALES [AHKP22] being the closest to practical feasability. Remarkably, SCALES also allows for dishonest majority MPC with round complexity independent of function depth, a feature previously only achieved by BMR- and FHE-based constructions [BMR90, LPSY15, HSS20, Sma23]. We observe that the main bottleneck preventing an efficient instantiation of SCALES is the BHHO encryption scheme. Hence, it is worthwhile to consider improvements and alternatives to BHHO.

Alternatives to and limitations of BHHO. Even though BHHO was published already about 17 years ago, it is currently still the *only* known encryption scheme that provides the required functional and security properties for RGS. The main limitation of BHHO is that encrypting κ bits in a way that allows for the necessary homomorphisms yields a ciphertext of length $\mathcal{O}(\kappa^2)$ group elements. Both encryption and decryption require $\mathcal{O}(\kappa^2)$ exponentiations. Since garbling a circuit C using RGS requires $\mathcal{O}(|C|)$ ciphertexts, it seems currently practically infeasible to instantiate and use RGS based on BHHO in applications, even for simple circuits. In this work, we describe a KHME scheme that improves space

and time complexity in comparison to BHHO by almost 4 orders of magnitude, while preserving properties essential for RGS (homomorphism, rerandomizability) and losing non-essential ones (circular security). We therefore claim to enable the first practical YOSO-like MPC construction for small circuits, for example computing the maximum. See Table 1 for detailed performance numbers.

Our contributions. We summarize our contributions as follows:

Basic KMHE. In Section 3, we describe our new "basic" KMHE scheme, which can be seen as a pairing-based analogue of [BHHO08]. It reduces the ciphertext size and the computational costs both asymptotically and concretely very significantly. A ciphertext of our scheme consists of a *linear* (in the security parameter) number of group elements, as opposed to a *quadratic* number in BHHO. Concretely, when instantiated with the BLS12-381 pairing, a ciphertext of our scheme has size 360 kB, while (expanded) BHHO instantiated with the high-performance Curve25519 [Ber06] requires 16.68 MB. This is an improvement by 97.83 %. Using the RELIC toolkit [AGM+] to estimate the number of clock cycles, an encryption takes about 6.10×10^8 clock cycles (about 0.01 seconds with a parallelized implementation on a processor with 16 cores running at 4 Ghz), while BHHO takes 1.6×10^{13} clock cycles (4 minutes), which is an improvement by 99.96 %.
This scheme does not only serve to explain the main idea and new techniques underlying our subsequent, more complex constructions, but we consider it also of independent interest. It might prove useful for applications that require a practical encryption scheme that provides the same homomorphism in the key- and the message space, but not a full-fledged RGS scheme.

Gate KMHE. In Section 4 we introduce the new notion of a *gate KMHE* scheme. Essentially, a gate KMHE scheme adopts the above "basic" KMHE scheme to the use in a (rerandomizable) garbled circuit, following Yao's seminal construction [Yao86]. In addition to the trivial syntactic changes (e.g., the encryption now takes four keys and messages as input, as required to garble a gate of a given circuit), we also define and prove several new, non-trivial functional and security properties that are required to build *rerandomizable* garbled circuits from gate KMHE. The main advantage of using gate KMHE instead of the *shareable KMHE* from [AHKP22] is improved performance. While RGS from shareable KMHE requires eight ciphertexts per garbled gate, gate KMHE uses double-key encryption to reduce this to four ciphertexts, and enables randomness reuse to decrease the number of group elements used in each ciphertext even further.
Comparing the concrete space and time improvements of Gate KMHE to [AHKP22], for the same setting as considered above we achieve a garbled gate size of 1.35 MB, where [AHKP22] requires 133.43 MB (98.99 % improvement), and garbling a gate is estimated to take 0.04 seconds (33 minutes for [AHKP22], 99.996 % improvement).

RGS from Gate KMHE. In Section 5 we show how to build RGS from gate KMHE. The construction follows the approaches from [GHV10, AHKP22], which in turn are based on [Yao86].

We have identified some small theoretical issues with the RGS construction in [AHKP22]: In particular, their Rerand privacy was not sufficient to achieve *incremental Decomposable Randomized Encoding* (iDRE, a construction leading directly towards *SCALES*) privacy (for details see Appendix F). Acharya et al. acknowledged the existence of this gap and accordingly updated their paper with strengthened KMH privacy and Rerand privacy properties. In addition, they switched their KMHE from the plain version of BHHO to the expanded version to exhibit perfect rerandomizability. This allows their KMHE and RGS schemes to fulfill the stronger versions of KMH privacy and Rerand privacy, but comes at the cost of decreased efficiency: In comparison to their old scheme, the new KMHE scheme's ciphertext is κ group elements larger and encryption is slower by a multiplicative factor of κ. As our pairing-based KMHE scheme is only computationally rerandomizable, we take a different approach to fixing the original gap by defining an extended variant of Acharya et al.'s original computational Rerand privacy notion. We additionally introduce new notions of *KMH rerandomizability* and *key privacy under leakage*. Overall, these changes allow for the use of our pairing-based KMHE, facilitating significant reductions in space and runtime complexity. We remark that our variant of RGS can be also used for the maliciously-secure version of SCALES [AHKP24].

Improvements in space and runtime complexity As shown in Table 1, our work significantly reduces space and runtime complexity of RGS by four orders of magnitude. Garbling a 966 gate circuit computing the maximum of eight 8-bit numbers yields a RGS circuit size of 1.27 GB (125.87 GB in [AHKP22]) and is estimated to take 37 seconds (22 days in [AHKP22]). Garbling an AES-128 circuit with 34 576 gates requires an RGS circuit size of 45.60 GB (4.4 TB in [AHKP22]) and is estimated to take 20 minutes (2 years in [AHKP22]). Thus, the space requirements are improved by 98.99 % and the computational requirements even by 99.998 %. Two key aspects further highlight the significance of these optimizations:

1. Our improvements are particularly impactful in protocols requiring multiple rerandomizations of a garbling, especially when these garblings must be exchanged among multiple parties, as in the SCALES protocol from [AHKP22].
2. While our performance analysis assumes computation on 16 4-GHz CPUs, both garbling and rerandomization are parallelizable on up to $\kappa = 526$ cores. Hence, on high-performance servers with $n \leq \kappa$ cores, runtimes are even lower those shown in Table 1.

Hence, while RGS and the SCALES protocol remain relatively expensive primitives, we demonstrate for the first time that this approach is within reach of practical feasibility, particularly for simple circuits.

Related work. Gentry et al. [GHV10] were the first to show that [BHHO08] can be used to build RGS. Acharya et al. [AHKP22] built the SCALES protocol from RGS, and also pointed out a gap in [GHV10] by demonstrating that

Table 1. Comparison of the RGS from [AHKP22] and our work in bytes and clock cycles (cc). For our benchmarking methodology, technical details and a more granular comparison, see Appendix A.

	[AHKP22]	Our work	Improvement
Size of one garbled gate	133.43 MB	1.35 MB	98.99 %
Size of one garbled Max circuit	125.87 GB	1.27 GB	98.99 %
Size of one garbled Mult circuit	1.74 TB	18.04 GB	98.99 %
Size of one garbled AES circuit	4.4 TB	45.60 GB	98.99 %
Garbling one gate (cc)	1.28×10^{14} (33 min)	2.44×10^{9} (0.04 s)	99.998 %
Garbling a Max circuit (cc)	1.24×10^{17} (22 d)	2.36×10^{12} (37 s)	99.998 %
Garbling a Mult circuit (cc)	1.75×10^{18} (317 d)	3.33×10^{13} (9 min)	99.998 %
Garbling an AES circuit (cc)	4.43×10^{18} (2 yr)	8.43×10^{13} (20 min)	99.998 %
Rerand. a garbled gate (cc)	5.13×10^{14} (2.23 h)	3.35×10^{9} (0.05 s)	99.9993 %
Rerand. a garbl. Max circuit (cc)	4.95×10^{17} (90 d)	3.24×10^{12} (51 s)	99.9993 %
Rerand. a garbl. Mult circuit (cc)	7.01×10^{18} (3.5 yr)	4.58×10^{13} (11 min)	99.9993 %
Rerand. a garbl. AES circuit (cc)	1.77×10^{19} (9 yr)	1.16×10^{14} (30 min)	99.9993 %

rerandomizability cannot be reduced to semantic security and key leakage resilience alone, introduced KMH privacy as a stronger property, and proved it for BHHO. In a recent follow-up work, Acharya et al. [AHKP24] then extend SCALES to a dishonest majority of input providers and servers in the malicious model, respectively.

Technical Overview. In the following, we give a technical overview about how our novel KMHE and RGS constructions allow for the aforementioned efficiency improvements.

There are three aspects to consider when designing a KMHE scheme suitable for constructing an RGS: *Firstly*, for the scheme to be able to encrypt its own secret keys, the key space must be a subset of the message space. *Secondly*, we must be able to apply the *same* homomorphism to both the secret key and the message. *Finally*, the homomorphism must be able to transform the key in such a way that a transformed key is indistinguishable from a fresh one, even given some information about the homomorphism. We call this property *key privacy under leakage*.

Key-and-message homomorphism. We will now take the example of the ElGamal scheme and explore how it could be adapted to these requirements. A secret key is $x \leftarrow\!\!\$\ \mathbb{Z}_q$, the corresponding public key is g^x where $g \in \mathbb{G}$ is a generator in a finite, cyclic group $\mathbb{G}$ of prime order q. To encrypt a message $m \in \mathbb{G}$, encryption samples $a \leftarrow\!\!\$\ \mathbb{Z}_q$, and outputs the ciphertext $(g^a, g^{xa} \cdot m)$. However, the message m is now an element of the group $\mathbb{G}$ while the key x is an element

in the exponent field $\mathbb{Z}_q$, violating our first KMHE requirement. A common solution is to instead encode the message as an exponent in $\mathbb{Z}_q$. Encryption of such a $m \in \mathbb{Z}_q$ is then done by sampling $a \leftarrow\!\!\$\ \mathbb{Z}_q$, and outputting the ciphertext $(c_1, c_2) = (g^a, g^{xa} \cdot g^m)$. Now the key space and message space are both $\mathbb{Z}_q$. This also addresses the homomorphism and rerandomization requirements: We can add an offset $\sigma \in \mathbb{Z}_q$ to the secret key by computing $c_2 \cdot c_1^{\sigma} = g^{(x+\sigma)a} \cdot g^m$. Then $x + \sigma$ looks like a fresh uniform secret key. The message homomorphism works via $c_2 \cdot g^{\tau} = g^{x\tau} \cdot g^{m+\tau}$ for any $\tau \in \mathbb{Z}_q$.

However, note that now decryption requires computing the discrete log of $g^{m+\tau}$; this only works if the message is fairly small. Since we want to encrypt and rerandomize secret keys, which are uniform over $\mathbb{Z}_q$, the above approach is not feasible.

Key privacy under leakage. There is a second reason why the construction with the $(\mathbb{Z}_q, +)$ homomorphism will not work: For RGS, our KMHE also needs to provide key privacy under leakage (previously part of the *KMH privacy* property from [AHKP22]), which informally means the following:

We sample two random secret keys sk^1 and sk^2 and a random key transformation function σ (in the previous ElGamal example this was the offset $\sigma \in \mathbb{Z}_q$, implying a function $m \mapsto m + \sigma$). Given sk^1, sk^2, and $\sigma(\mathsf{sk}^2)$, the value $\sigma(\mathsf{sk}^1)$ must still have high average-min entropy, i.e. look random to a distinguisher. This can only work if the values sk^2 and $\sigma(\mathsf{sk}^2)$ leak very little information about the function σ. In our ElGamal example, sk^2 would correspond to $x_2 \leftarrow\!\!\$\ \mathbb{Z}_q$ and $\sigma(\mathsf{sk}_2) = x_2 + \sigma$. Clearly, given both these values, an adversary can recover σ and then the value $\sigma(\mathsf{sk}^1) = x_1 + \sigma$ has no entropy at all.

The solution to this, introduced by [GHV10], is to use a permutation homomorphism over the key space $HW_{\kappa,\kappa/2}$, consisting of all bit-strings of length κ with Hamming weight $\kappa/2$ (meaning exactly half the bits are zeroes and half are ones). As shown in [GHV10, Lemma 9], obtaining the input $\mathsf{sk} \in HW_{\kappa,\kappa/2}$ and output $\sigma(\mathsf{sk})$ for some permutation σ leaks very little information about the permutation itself. Intuitively, this is because for each input sk and output sk', there are many permutations that could map sk to sk'.

A permutation homomorphism, however, decreases efficiency of the scheme significantly: A message $m \in HW_{\kappa,\kappa/2} \subset \{0,1\}^{\kappa}$ must be encrypted bit-wise, leading to a $O(\kappa) = O(\lambda)$ multiplicative blowup in ciphertext size compared to regular ElGamal.

We also need the ability to support the same permutation homomorphism for the secret key. Here, the BHHO scheme [BHHO08] comes into play. A BHHO public key consists of bases $g_1, \ldots, g_\kappa \in \mathbb{G}^\kappa$ and the "hash" $\prod_{j\in\kappa} g_j^{\mathsf{sk}_j}$. Thanks to the left-over hash lemma [HILL99], as long as sk has sufficient min-entropy, $\prod_{j\in[\kappa]} g_j^{\mathsf{sk}_j}$ looks like a random group element g^x, so security of the scheme follows from DDH. To encrypt a message $m \in \{0,1\}^\kappa$, each message bit m_i is encrypted as $(g_1^{a_i}, \ldots, g_\kappa^{a_i}, \prod_{j\in\kappa} g_j^{a_i \mathsf{sk}_j} \cdot g^{m_i})$, where $a_i \leftarrow\!\!\$\ \mathbb{Z}_q$. The key permutation itself can then be applied by first permuting $g_1, \ldots, g_\kappa$, and then rerandomizing the public key and the ciphertext. BHHO public keys can be easily rerandomized

by exponentiating all elements in the public key with a random $r \leftarrow\!\!\$\ \mathbb{Z}_q$. The ciphertext elements can be rerandomized analoguosly.

Improving efficiency using pairings. We will now examine the efficiency of the BHHO scheme as presented in [GHV10, AHKP22] and then outline our idea to improve space and computation complexity quadratically. Since each message bit adds $\kappa + 1$ group elements to the ciphertext, for κ message bits, we require $\kappa^2 + \kappa$ many group elements. The quadratic runtime and space requirement is due to the fact that the bases $g_1, \ldots, g_\kappa$ have to be combined with the randomness a_i of each message bit m_i, resulting in elements $g_1^{a_i}, \ldots, g_\kappa^{a_i}$ for $i \in [\kappa]$. To avoid this blowup one could separate the $g_1, \ldots, g_\kappa$ and $a_1, \ldots, a_\kappa$ and give them out separately so that the combinations can later be computed dynamically during decryption. While revealing $a_1, \ldots, a_\kappa$ in plain is clearly not secure, a bilinear pairing $\mathrm{e} : \mathbb{G} \times \mathbb{H} \to \mathbb{G}_T$ allows us to compute g_T^{ab} given just g^a and h^b, where $g \in \mathbb{G}$, $h \in \mathbb{H}$, and $a, b \in \mathbb{Z}_q$. We can therefore encode the randomness $a_1, \ldots, a_\kappa$ as group elements $h^{a_1}, \ldots, h^{a_\kappa}$ and use the pairing to compute $\mathrm{e}(g_j, h^{a_i}) = \mathrm{e}(g_j^{a_i}, h)$ for any combination $i, j \in [\kappa]$. This allows us to compute a ciphertext as $c_i = (h^{a_i}, g_T^{m_i} \cdot y^{a_i}) \quad \forall i \in [\kappa]$ for $y = \mathrm{e}\left(\prod_{j\in[\kappa]} g_j^{\mathsf{sk}_j}, h\right)$, resulting in the aforementioned quadratic reductions in space and time complexity.

Further efficiency improvements through gate KMHE. In order to utilize a KMHE scheme as part of a garbling scheme, one needs a variant of KMHE that requires *two* keys to decrypt any ciphertext. To this end, Acharya et al. [AHKP22] introduced *sharable KMHE* which solves this problem by 2-out-of-2-secret-sharing the message, and encrypting each share using a single-key KMHE scheme. This approach results in two full ciphertexts per message; for a Boolean gate with two input wires, we thus need eight ciphertexts per garbled gate. We introduce *gate KMHE* as a way to improve upon the performance and space requirements of shareable KMHE, making use of the specific algebraic properties of our KMHE scheme. We will now take a closer look at the techniques used to achieve these improvements. Recall that we previously encrypted a message $m \in \{0,1\}^\kappa$ as $c_i = (h^{a_i}, g_T^{m_i} \cdot y^{a_i}) \quad \forall i \in [\kappa]$, where $y = \mathrm{e}\left(\prod_{j\in[\kappa]} g_j^{\mathsf{sk}_j}, h\right)$. To halve the number of ciphertexts per gate, we do not secret-share the message, but instead use "double encryption" and encrypt a single ciphertext using both secret keys sk^0 and sk^1. The functionality remains the same: Both keys are necessary to obtain the full message m. The resulting ciphertext is given by $c_i = (h^{a_i}, g_T^{m_i} \cdot y^{a_i}) \quad \forall i \in [\kappa]$, where $y = \mathrm{e}\left(\prod_{j\in[\kappa]} (g_{0,j}^{\mathsf{sk}_j^0} \cdot g_{1,j}^{\mathsf{sk}_j^1}), h\right)$.

We further halve the number of elements in $\mathbb{G}$ required per garbled gate by using only two sets of bases $g_1, \ldots, g_\kappa$ for four ciphertexts. This works as follows: Recall that a garbled gate uses four secret keys $\mathsf{sk}^{\alpha\beta}$, for $\alpha, \beta \in \{0,1\}$, two per

input wire. Using double encryption, a garbled gate can then be visualized as

$$\begin{pmatrix} \mathsf{Enc}(\mathsf{sk}^{00}, \mathsf{sk}^{10}, m^1) \\ \mathsf{Enc}(\mathsf{sk}^{00}, \mathsf{sk}^{11}, m^2) \\ \mathsf{Enc}(\mathsf{sk}^{01}, \mathsf{sk}^{10}, m^3) \\ \mathsf{Enc}(\mathsf{sk}^{01}, \mathsf{sk}^{11}, m^4) \end{pmatrix}$$

To construct an RGS, we only need to be able to apply the same key homomorphism σ_0 to the "left" keys sk^{00} and sk^{01}, and similarly for σ_1 and the "right" keys sk^{10} and sk^{11}. Hence, we only need two sets of bases, a set $g_{0,1}, \ldots, g_{0,\kappa}$ to apply σ_0, and a set $g_{1,1}, \ldots, g_{1,\kappa}$ to apply σ_1. To make it possible to actually share these random bases across the different ciphertexts, we introduce the gate KMHE primitive, which internally creates these four double encryptions while enabling a reuse of these left and right bases.

2 Preliminaries

Notation. Let $\mathbb{N}$ be the set of natural numbers, not including 0. For some $Q \in \mathbb{N}$, we write $[Q] := \{1, 2, \ldots, Q\}$. We write $x \leftarrow\!\!\$\ X$ to denote that $x \in X$ is sampled uniformly from X.

Let $s \in \{0,1\}^\ell$ be a bit-string of length ℓ. The Hamming weight of s then denotes the number of 1's in s. By $HW_{\kappa,\kappa/2}$ denote the set of all bit-strings of length κ with Hamming weight $\kappa/2$. S_κ denotes the set of all permutations mapping $[\kappa] \to [\kappa]$. For $\sigma \in S_\kappa$ and a vector $x = (x_1, \ldots, x_\kappa)$ of κ elements, we overload notation and may occasionally also write $\sigma(x)$ to denote the vector $(x_{\sigma^{-1}(1)}, \ldots, x_{\sigma^{-1}(\kappa)})$. By $\sigma_2 \circ \sigma_1$ we denote standard function composition, so $\sigma_2 \circ \sigma_1 : x \mapsto \sigma_2(\sigma_1(x))$. To convert numbers from binary to decimal we define the function $\mathsf{Bin2Dec} : \{0,1\}^* \mapsto \mathbb{N} \cup \{0\}$. It uses the standard approach for converting binary numbers to non-negative decimal.

For a group $\mathbb{G}$ we then denote the identity element in $\mathbb{G}$ by $1_\mathbb{G}$. For a vector of group elements $\mathbf{g} = (g_1, \ldots, g_\nu) \in \mathbb{G}^\kappa$ and a bit vector $\mathsf{sk} = (s_1, \ldots, s_\kappa) \in \{0,1\}^\kappa$ we write

$$\mathbf{g}^{\mathsf{sk}} = \prod_{j=1}^{\kappa} g_j^{s_j}$$

If we say that an algorithm is *efficient*, if it runs in probabilistic polynomial-time (PPT). Let D_1 and D_2 be two distributions over some finite domain $\mathcal{W}$. Then define the statistical distance of D_1 and D_2 to be

$$\Delta(D_1, D_2) = \frac{1}{2} \sum_{x \in \mathcal{W}} |\Pr[D_1 = x] - \Pr[D_2 = x]|.$$

Let D_1 and D_2 be parameterized by $\lambda \in \mathbb{N}$. We write $D_1 \overset{\text{s}}{\approx} D_2$, read "$D_1$ and D_2 are statistically indistinguishable", if $\Delta(D_1, D_2) \leq \mathsf{negl}(\lambda)$, and $D_1 \overset{\text{c}}{\approx} D_2$ if D_1 and D_2 are computationally indistinguishable. We write $D_1 \equiv D_2$ if D_1 and

D_2 are the same distributions, i.e. $\Delta(D_1, D_2) = 0$. For random variables X, Y, we denote the expectation of X, and the conditional expectation of X given Y as $\mathbb{E}[X]$, and $\mathbb{E}_{y \leftarrow Y}[X|y]$, respectively.

2.1 Bilinear groups

We define the algorithm $\mathcal{G}(1^\lambda)$ that takes in the security parameter λ and outputs a description $(q, g, h, g_T, \mathbb{G}, \mathbb{H}, \mathbb{G}_T, \mathrm{e})$ of a bilinear group. The output of $\mathcal{G}$ must fulfill the following properties:

- $\mathbb{G}$, $\mathbb{H}$, and $\mathbb{G}_T$ are groups of prime order q and g, h, and g_T are respective generators.
- $\mathrm{e} : \mathbb{G} \times \mathbb{H} \to \mathbb{G}_T$ is an efficiently computable function satisfying the properties of bilinearity, i.e. $\forall X \in \mathbb{G}, Y \in \mathbb{H}$ and $a, b \in \mathbb{Z}_q$ it holds that $\mathrm{e}(X^a, Y^b) = \mathrm{e}(X, Y)^{ab}$, and non-degeneracy, i.e. $\mathrm{e}(g, h) \neq 1_{\mathbb{G}_T}$. Note that we use multiplicative notation for all groups.

We also define a number of assumptions that we assume to hold in those groups, as well as needed lemmas that build on them.

Definition 1 (Decisional Diffie-Hellman). *Let $\mathbb{G}$ be a group of prime order q. We say that the decisional Diffie-Hellman assumption holds in $\mathbb{G}$ if it holds that*

$$\{(g_1, g_2, g_1^r, g_2^r) : g_1, g_2 \leftarrow\!\$\, \mathbb{G}, r \leftarrow\!\$\, \mathbb{Z}_q^*\} \overset{c}{\approx} \{(g_1, g_2, v_1, v_2) : g_1, g_2, v_1, v_2 \leftarrow\!\$\, \mathbb{G}\}.$$

The following is a generalization of the standard DDH assumption to more than two generators, as introduced by Naor and Reingold [NR97]. It is implied by DDH without any loss in tightness, see Lemma 4.2 in [NR97] for the proof.

Lemma 2 (Generalized DDH). *Let $\mathbb{G}$ be a group with prime order q. Assuming the decisional Diffie-Hellman assumption holds for $\mathbb{G}$, we have that*

$$\begin{aligned}\left\{(g_1, \ldots, g_\rho, g_1^r, \ldots, g_\rho^r) : g_1, \ldots, g_\rho \leftarrow\!\$\, \mathbb{G}, r \leftarrow\!\$\, \mathbb{Z}_q^*\right\} \\ \overset{c}{\approx} \left\{(g_1, \ldots, g_\rho, v_1, \ldots, v_\rho,) : g_1, \ldots, g_\rho, v_1, \ldots, v_\rho \leftarrow\!\$\, \mathbb{G}\right\},\end{aligned}$$

where $\rho = \rho(\lambda)$ is a polynomial of constant degree.

The following lemma, a special case of Lemma 1 in [BHHO08], will also be needed. It is implied by DDH with logarithmic reduction loss [Vil12].

Lemma 3 (Matrix DDH). *Let $\eta, \nu \in \mathbb{N}$ and let $\mathbb{G}$ be a group with prime order q. Then, assuming decisional Diffie-Hellman assumption holds for $\mathbb{G}$, it holds that*

$$\mathbf{G} \overset{c}{\approx} \mathbf{G}',$$

where $\mathbf{G} \leftarrow\!\$\, \mathbb{G}^{\eta \times \nu}$ is uniform and $\mathbf{G}' \leftarrow\!\$\, Rk_1(\mathbb{G}^{\eta \times \nu})$ is a uniform rank-1 matrix.

The *symmetric external Diffie-Hellman assumption* is an extension of the DDH assumption to the source groups $\mathbb{G}$ and $\mathbb{H}$ of a bilinear group setting.

Definition 4 (Symmetric External Diffie-Hellman (SXDH)). *The symmetric external Diffie-Hellman assumption holds for $\mathcal{G}$ if for every $\lambda \in \mathbb{N}$, $(p, g, h, g_T, \mathbb{G}, \mathbb{H}, \mathbb{G}_T, \mathrm{e}) \leftarrow \mathcal{G}(1^\lambda)$, the decisional Diffie-Hellman assumption holds for $\mathbb{G}$ and $\mathbb{H}$.*

We also formulate a SXDH-analogue of Lemma 2 for convenience, it is straightforwardly implied by the hardness of DDH in the source groups of the bilinear group setting.

Corollary 5 (Generalized SXDH). *Assuming the symmetric external Diffie-Hellman assumption holds for $\mathcal{G}$, then generalized DDH (Lemma 2) holds for groups $\mathbb{G}$ and $\mathbb{H}$, respectively.*

3 Basic KMHE Scheme

In the following section we describe our basic KMHE scheme. This scheme alone will not yet be sufficient to construct an RGS, and we will describe an extension to a *gate KMHE* variant in Section 4, which is tailored to the use in garbled circuits. However, the basic scheme illustrates the main idea of the construction, and helps to explain the proof techniques used for the gate KMHE in a simpler setting. We also rely on it for our performance comparison (Appendix A) with the BHHO-based KMHE scheme from [AHKP22].

A noteworthy aspect of our basic KMHE scheme is that it can be viewed as a batched version of BHHO, allowing for a more compact encryption of multiple messages. If we encrypt ℓ messages using BHHO with a key of length κ, then we get a ciphertext containing $\mathcal{O}(\ell\kappa)$ group elements, while our scheme only requires $\mathcal{O}(\ell + \kappa)$ group elements, with significantly better concrete efficiency.

Furthermore, our scheme inherits the leakage resilience of BHHO, and thus could potentially also be used in constructions where BHHO is used as a leakage-resilient scheme, e.g. leakage-resilient CCA-secure PKE [FKMV12] or tamper-resilient PKE [DFMV17].

However, in our opinion the unique feature of BHHO and our scheme is that it provides the same homomorphism in the key and message space. RGS are a prime application that use this feature, and we are currently not aware of others. But we believe that our work might also pave the way towards more practical applications that exploit this unique feature.

We proceed by defining the syntax of KMHE, adopted from [AHKP22]. We then continue with our construction.

Definition 6 (Key-and-Message-Homomorphic Encryption). *A* key-and-message-homomorphic encryption *(KMHE) scheme is a tuple of PPT algorithms* $\mathsf{KMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *with the following syntax.*

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$**:** *This algorithm takes as input the security parameter λ and returns public parameters* pp*. The public parameters define a key space $\mathcal{K}$, a message space $\mathcal{M}$, and a ciphertext space $\mathcal{C}$.*

$\mathsf{sk} \leftarrow \mathsf{KGen}(\mathsf{pp})$**:** *This algorithm takes* pp *and returns a secret key* $\mathsf{sk} \in \mathcal{K}$*.*

$\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}, m)$: *Given a message $m \in \mathcal{M}$ and a secret key sk, this algorithm returns a ciphertext $\mathsf{ct} \in \mathcal{C}$.*

$\mathsf{ct}' \leftarrow \mathsf{Eval}(\mathsf{pp}, \sigma, \tau, \mathsf{ct})$: *Given a ciphertext $\mathsf{ct} \in \mathcal{C}$ and two functions $\sigma, \tau \in \mathcal{F}$, where the set $\mathcal{F}$ is defined by λ, this algorithm returns a ciphertext ct'.*

$m' \leftarrow \mathsf{Dec}(\mathsf{pp}, \mathsf{sk}, \mathsf{ct})$: *This algorithm takes as input pp and sk, and returns a message $m' \in \mathcal{M}$.*

By R_{Enc} and R_{Eval} we denote the randomness spaces for Enc and Eval, respectively. Unless specified otherwise, randomness is always assumed to be sampled uniformly at random.

Construction 7 (Basic KMHE scheme). *Our basic KMHE scheme consists of the following algorithms:*

$\mathsf{Setup}(1^\lambda)$: *Generate and output the description of a bilinear group*

$$\mathsf{pp} = (q, g, h, g_T, \mathbb{G}, \mathbb{H}, \mathbb{G}_T, e) \leftarrow \mathcal{G}(1^\lambda).$$

We define κ as the smallest positive and even integer such that $\kappa - \frac{1}{2}\log\kappa - 1/2 \geq \log q + 2\lambda$. These parameters define the following sets:

- *the message space as $\mathcal{M} = \{0,1\}^\kappa$.*
- *the ciphertext space as $\mathcal{C} = (\mathbb{H} \times \mathbb{G})^\kappa \times \mathbb{G}^\kappa \times \mathbb{G}_T$.*
- *the key space as $\mathcal{K} = HW_{\kappa,\kappa/2}$, the set of bit strings of length κ with Hamming weight $\kappa/2$. Note that we have $\mathcal{K} \subset \mathcal{M}$, i.e. all keys are in the message space and thus the scheme is able to encrypt its own keys.*
- *the set of key- and message-homomorphisms as $\mathcal{F} = S_\kappa$, the set of permutations over κ elements,*
- *and randomness spaces $R_{\mathsf{Enc}} = \mathbb{G}^\kappa \times \mathbb{Z}_q^\kappa$ and $R_{\mathsf{Eval}} = \mathbb{Z}_q^\kappa \times \mathbb{Z}_q^*$.*

$\mathsf{KGen}(\mathsf{pp})$: *Sample and output $\mathsf{sk} \leftarrow\!\!\$\, \mathcal{K}$.*

$\mathsf{Enc}(\mathsf{pp}, \mathsf{sk}, m; r)$: *Parse $m = (m_1, \ldots, m_\kappa) \in \{0,1\}^\kappa$, $\mathsf{sk} = (s_1, \ldots, s_\kappa) \in \{0,1\}^\kappa$, and $r = (g_1, \ldots, g_\kappa, a_1, \ldots, a_\kappa) \in R_{\mathsf{Enc}}$. Then compute*

$$y = \mathrm{e}\left(\prod_{j\in[\kappa]} g_j^{s_j}, h\right),$$

and

$$c_i = (h^{a_i}, g_T^{m_i} \cdot y^{a_i}) \quad \forall i \in [\kappa].$$

Finally output ciphertext $\mathsf{ct} = (c_1, \ldots, c_\kappa, g_1, \ldots, g_\kappa, y)$.

$\mathsf{Eval}(\mathsf{pp}, \sigma, \tau, \mathsf{ct}; r)$: *Parse $\mathsf{ct} = (c_1, \ldots, c_\kappa, g_1, \ldots, g_\kappa, y)$ and $r = (a'_1, \ldots, a'_\kappa, d) \in R_{\mathsf{Eval}}$. Apply the key permutation σ to obtain $\hat{g}_1, \ldots, \hat{g}_\kappa$ and the message permutation τ to obtain $\hat{c}_1, \ldots, \hat{c}_\kappa$ such that*

$$\hat{g}_{\sigma(i)} = g_i \qquad \text{and} \qquad \hat{c}_{\tau(i)} = c_i$$

for all $i \in [\kappa]$. To computationally hide the permutations and make the ciphertext indistinguishable from a ciphertext produced by $\mathsf{Enc}(\mathsf{pp}, \sigma(\mathsf{sk}), \tau(m))$,

rerandomize the ciphertext by computing

$$c'_i = \left(\left(\hat{c}_{i,1} \cdot h^{a'_i}\right)^{1/d}, \hat{c}_{i,2} \cdot y^{a'_i}\right) \quad \forall i \in [\kappa],$$
$$g'_i = \hat{g}_i^d \quad \forall i \in [\kappa], \text{ and}$$
$$y' = y^d.$$

Finally, output $\mathsf{ct}' = (c'_1, \ldots, c'_\kappa, g'_1, \ldots, g'_\kappa, y')$.

$\mathsf{Dec}(\mathsf{pp}, \mathsf{sk}, \mathsf{ct})$**:** *Parse* $\mathsf{ct} = (c_1, \ldots, c_\kappa, g_1, \ldots, g_\kappa, \cdot)$ *and* $\mathsf{sk} = (s_1, \ldots, s_\kappa) \in \{0,1\}^\kappa$.

For all $i \in [\kappa]$*, compute*

$$g_T^{m'_i} = c_{i,2} \cdot \mathrm{e}\left(\prod_{i \in [\kappa]} g_i^{s_j}, c_{i,1}\right)^{-1}$$

and output $m' = (m'_1, \ldots, m'_\kappa) \in \{0,1\}^\kappa$.

Verifying correctness of the scheme is a straightforward calculation, see Appendix B

3.1 KMH Correctness

KMH correctness ensures that a ciphertext created by Eval is well-formed and that the homomorphisms are applied correctly. More precisely, given a ciphertext produced by $\mathsf{Enc}(\mathsf{pp}, \mathsf{sk}, m)$, which encrypts message m under secret key sk, we can run $\mathsf{Eval}(\mathsf{pp}, \sigma, \tau, \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}, m))$ to obtain an encryption of message $\tau(m)$ under secret key $\sigma(\mathsf{sk})$. The KMH correctness proof is given in Appendix C.

Definition 8 (KMH Correctness). $\mathsf{KMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *is KMH-correct, if for all* $\lambda \in \mathbb{N}$, $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $\mathsf{sk} \leftarrow \mathsf{KGen}(\mathsf{pp})$, $m \in \mathcal{M}$, $\sigma, \tau \in \mathcal{F}$, $r_1 \in R_{\mathsf{Enc}}$, $r_2 \in R_{\mathsf{Eval}}$ *there exists randomness* $r' \in R_{\mathsf{Enc}}$ *such that*

$$\mathsf{Eval}(\mathsf{pp}, \sigma, \tau, \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}, m; r_1); r_2) = \mathsf{Enc}(\mathsf{pp}, \sigma(\mathsf{sk}), \tau(m); r').$$

3.2 CPA Security

Definition 9 (IND-CPA Security). $\mathsf{KMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *is IND-CPA-secure, if for all PPT adversaries* $\mathcal{A}$ *there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\left|2 \cdot \Pr\left[\mathsf{KMHE\text{-}IND\text{-}CPA}_{\mathcal{A},\mathsf{KMHE}}(1^\lambda) = 1\right] - 1\right| \leq \mathsf{negl}(\lambda),$$

where the KMHE-IND-CPA *experiment is defined in Figure 1.*

We now formally prove our basic scheme to be IND-CPA-secure. Our gate KMHE scheme will follow the same proof structure later, albeit with increased complexity. Therefore this proof serves as a simpler illustration of the used techniques.

Theorem 10. *Assuming hardness of the SXDH-problem (Definition 4) in the bilinear groups defined by* pp*, Construction 7 is* IND-CPA-secure.

$\mathsf{KMHE\text{-}IND\text{-}CPA}(1^\lambda)$

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$

$(m_0, m_1) \leftarrow \mathcal{A}(\mathsf{pp}, 1^\lambda)$

$\mathsf{sk} \leftarrow \mathsf{KGen}(\mathsf{pp})$

$b \leftarrow\!\!\$\ \{0,1\}$

$\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}, m_b)$

$b' \leftarrow \mathcal{A}^{\mathsf{Enc}(\mathsf{pp},\mathsf{sk},\cdot)}(\mathsf{ct})$

return $(b' = b)$

Fig. 1. Definition of Game KMHE-IND-CPA.

Building blocks of the security proof. The security analysis will use that secret keys have sufficient entropy, and apply the left-over hash lemma [HILL99, Lemma 4.8]. The *min-entropy* of a random variable X is defined as

$$H_\infty(X) = -\log\left(\max_{x \in \mathrm{supp}(X)} \Pr[X = x]\right)$$

Lemma 11 (Left-over Hash Lemma (LHL)). *Let q be a prime, $\kappa \in \mathbb{N}$, $\epsilon > 0$, and let $X = (X_1, \ldots, X_\kappa \in \{0,1\}^\kappa$ be a random variable such that $H_\infty(X) \geq \log q + 2\log(1/\epsilon)$. Then for $a \leftarrow\!\!\$\ \mathbb{Z}_q^\kappa$ it holds that*

$$\Delta((a, \sum_{i \in [\kappa]} a_i X_i), (a, U(\mathbb{Z}_q))) \leq \epsilon.$$

where $U(\mathbb{Z}_q)$ is the uniform distribution over $\mathbb{Z}_q$.

To use the LHL we will need a bound on the min-entropy of secret keys, given by the following lemma.

Lemma 12. *Let $\kappa \geq 2$ be an even integer and let $\mathsf{sk} \leftarrow\!\!\$\ HW_{\kappa,\kappa/2}$. Then*

$$H_\infty(\mathsf{sk}) \geq \kappa - \frac{1}{2}\log\kappa - 1/2.$$

The proof of Lemma 12 uses bounds for the binomial coefficient from [Sta01]. For details see Appendix D. Lastly, the following lemma will be useful to simplify the proof of CPA security.

Lemma 13. *Let $\eta, \nu \in \mathbb{N}$, $\mathbf{g}_1, \ldots, \mathbf{g}_\eta \leftarrow\!\!\$\ \mathbb{G}^\nu$, where $\mathbb{G}$ is a group of prime order q where the DDH assumption (Definition 1) holds, and $\mathsf{sk} = (s_1, \ldots, s_\nu) \in \{0,1\}^\nu$ such that $H_\infty(\mathsf{sk}) \geq \log q + 2\lambda$. Then it holds that*

$$\left\{\mathbf{g}_1, \ldots, \mathbf{g}_\eta, \mathbf{g}_1^{\mathsf{sk}}, \ldots, \mathbf{g}_\eta^{\mathsf{sk}}\right\} \overset{c}{\approx} \left\{\mathbf{g}_1, \ldots, \mathbf{g}_\eta, g_1', \ldots, g_\eta'\right\},$$

where $g_1', \ldots, g_\eta' \leftarrow\!\!\$\ \mathbb{G}$.

The proof of Lemma 13 uses the Matrix DDH lemma (Lemma 3) to enable usage of the LHL (Lemma 11) with minimal entropy requirements on sk. For details see Appendix E.

Proof of Theorem 10. We describe a sequence of games that moves from the regular IND-CPA security game to a game where the message is perfectly hidden. In the latter game the adversary has no chance of winning, indistinguishability of the sequence of games then ensures that the adversary also cannot win the original IND-CPA security game except with negligible probability.

Game 1. This is the KMHE-IND-CPA game.

Game 2. Let $\mathsf{ct}_1, \ldots, \mathsf{ct}_Q$, where

$$\mathsf{ct}_i = (c_{i,1}, \ldots, c_{i,\kappa}, g_{i,1}, \ldots, g_{i,\kappa}, y_i) \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}, m'_i)$$

for $i \in [Q], Q \in \mathsf{poly}(\lambda)$, be the ciphertexts given to the adversary via the encryption queries and let $\mathsf{ct}_{Q+1} = (c_{Q+1,1}, \ldots, c_{Q+1,\kappa}, g_{Q+1,1}, \ldots, g_{Q+1,\kappa}, y_{Q+1})$ be the challenge ciphertext encrypting the challenge message m_b. We alter the behaviour of Enc in this hybrid, specifically how it computes the y_i in each ct_i for all $i \in [Q+1]$. Enc samples an additional $x_i \leftarrow\!\!\$\ \mathbb{Z}_q$ and then sets

$$y_i = \mathrm{e}\left(g^{x_i}, h\right),$$

where $g \in \mathbb{G}$ is from pp. Enc then uses these y_i for the remaining computation of ct_i, specifically the $c_{i,1}, \ldots, c_{i,\kappa}$.

Game 3. Consider now the challenge ciphertext ct_{Q+1}. In Game 2, the message-dependent part of each $c_{Q+1,j}$, $j \in [\kappa]$, is given by

$$c_{Q+1,j,2} = g_T^{m_j} \cdot y_{Q+1}^{a_j} = g_T^{m_j} \cdot (\mathrm{e}(g^{x_{Q+1}}, h))^{a_j} = g_T^{m_j} \cdot \mathrm{e}\left(g, h^{x_{Q+1} a_j}\right).$$

For Game 3 we change this to

$$c_{Q+1,j,2} = g_T^{m_j} \cdot \mathrm{e}\left(g, h_j\right),$$

where $h_1, \ldots, h_\kappa \leftarrow\!\!\$\ \mathbb{G}$. Now m_j is perfectly hidden by the uniform value $\mathrm{e}\left(g, h_j\right)$.

We proceed to prove indistinguishability of these hybrids:

Game 1 $\overset{\mathbf{c}}{\approx}$ Game 2: Define $\mathbf{g}_i = (g_{i,1}, \ldots, g_{i,\kappa})$ for all $i \in [Q+1]$. By definition of our KMHE scheme it then holds that in Game 1, each y_i in ct_i is computed as

$$y_i = \mathrm{e}\left(\prod_{j \in [\kappa]} g_{i,j}^{s_j}, h\right) = \mathrm{e}\left(\mathbf{g}_i^{\mathsf{sk}}, h\right) \quad \forall i \in [Q+1].$$

Our choice of κ and Lemma 12 ensures that

$$H_\infty(\mathsf{sk}) \geq \kappa - \frac{1}{2}\log\kappa - 1/2 \geq \log q + 2\lambda.$$

Since an adversary distinguishing Game 1 and Game 2 obtains no other information on sk except for the y_i, we can apply Lemma 13, resulting in

$$\left\{\mathbf{g}_1, \ldots, \mathbf{g}_{Q+1}, \mathbf{g}_1^{\mathsf{sk}}, \ldots, \mathbf{g}_{Q+1}^{\mathsf{sk}}\right\} \overset{\mathrm{c}}{\approx} \left\{\mathbf{g}_1, \ldots, \mathbf{g}_{Q+1}, g'_1, \ldots, g'_{Q+1}\right\}, \qquad (1)$$

where $g'_1, \ldots, g'_{Q+1} \leftarrow\!\!\$\ \mathbb{G}$. Since $\mathbb{G}$ is cyclic, for all $i \in [Q+1]$ there exists a uniformly distributed $x_i \in \mathbb{Z}_q$ such that $g'_i = g^{x_i}$. The right side of Equation (1) thus corresponds to Game 2.

Game 2 $\overset{c}{\approx}$ Game 3: From Game 2 to Game 3 we are only changing ct_{Q+1}. In Game 2 it is given by

$$\begin{aligned} y_{Q+1} &= \mathrm{e}\left(g^{x_{Q+1}}, h\right) = \mathrm{e}\left(g, h^{x_{Q+1}}\right), \\ g_{Q+1,j} &\leftarrow\!\!\$\ \mathbb{G} \quad \forall j \in [\kappa], \\ c_{Q+1,j} &= \left(h^{a_{Q+1,j}}, g_T^{m_j} \cdot y_{Q+1}^{a_{Q+1,j}} = g_T^{m_j} \cdot \mathrm{e}\left(g, h^{x_{Q+1} a_{Q+1,j}}\right)\right) \quad \forall j \in [\kappa]. \end{aligned}$$

Since the $a_{Q+1,1}, \ldots, a_{Q+1,\kappa}$ are uniformly distributed and not given to the adversary, the elements $h^{a_{Q+1,1}}, \ldots, h^{a_{Q+1,\kappa}}$ look uniform over $\mathbb{G}$. Also, h was uniformly sampled as well. Therefore, we can apply Generalized SXDH (Corollary 5), resulting in

$$\begin{aligned} h, h^{a_{Q+1,1}}, \ldots, h^{a_{Q+1,\kappa}}, h^{x_{Q+1}}, h^{x_{Q+1} a_{Q+1,1}}, \ldots, h^{x_{Q+1} a_{Q+1,\kappa}} \\ \overset{c}{\approx} h, h^{a_{Q+1,1}}, \ldots, h^{a_{Q+1,\kappa}}, h', h_1, \ldots, h_\kappa, \end{aligned}$$

where $h', h_1, \ldots, h_\kappa \leftarrow\!\!\$\ \mathbb{G}$. For ct_{Q+1} this means that

$$c_{Q+1,j} \overset{c}{\approx} \left(h^{a_{Q+1,j}}, g_T^{m_j} \cdot \mathrm{e}\left(g, h_j\right)\right) \quad \forall j \in [\kappa].$$

The right side corresponds to Game 3, concluding the proof.

3.3 KMH Rerandomizability

As indicated in the technical overview, RGS requires a KMHE scheme with rerandomizability. Intuitively, it should hold that a ciphertext produced by $\mathsf{Eval}(\mathsf{pp}, \sigma, \tau, \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}, m))$ is computationally indistinguishable from a *fresh* ciphertext produced by $\mathsf{Enc}(\mathsf{pp}, \sigma(\mathsf{sk}), \tau(m))$. We will later formally prove this property for the gate KMHE extension of the basic KHME scheme in Section 4. Note that our basic KMHE scheme is also rerandomizable. However, we do not provide a formal definition of rerandomizability since we will not need it. The basic idea of the proof is to use Lemma 2 to show that $g'_i = \hat{g}_i^d \; \forall i \in [\kappa]$ look like fresh group elements, and that further rerandomizing the remaining randomness a_i by adding $a'_i \; \forall i \in [\kappa]$ results in a fresh-looking ciphertext.

4 Gate KMHE

We will now define and construct a new building block for RGS that we call *gate KMHE*. Gate KMHE can be seen as an extension of standard KMHE, tailored to the encryption of gates in a garbled circuit, and will be our main building block for RGS. This tailoring makes it possible to implement some optimizations, such as the re-use of randomness, which are not possible when the basic KMHE scheme is used as a black-box, and thus to realize rerandomizable garbling schemes more efficiently.

4.1 Construction

Intuitively, a gate KMHE scheme provides the functionality required to garble a gate in Yao's garbled circuit construction [Yao86]. Recall that in a garbled circuit, each garbled gate consists of four ciphertexts, each encrypting a message. Some messages may be identical, depending on the type of gate being garbled. These messages are encrypted under four different secret keys, using double encryption in a way such that knowing any two out of four secret keys allows to decrypt one message.

Our notion of gate KMHE reflects this, but extends it with additional functional requirements (most importantly, the ability to apply the same homomorphism to keys and messages), and with additional security properties (such as the computational indistinguishability of homomorphically-evaluated ciphertexts from fresh encryptions).

Definition 14 (Gate KMHE). *A* gate *KMHE scheme is a tuple of PPT algorithms* $\mathsf{GKMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *with the following syntax.*

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$**:** *This algorithm takes as input the security parameter* λ *and returns public parameters* pp*. The public parameters define a key space* $\mathcal{K}$*, a message space* $\mathcal{M}$*, and a ciphertext space* $\mathcal{C}$*.*

$\mathsf{sk} \leftarrow \mathsf{KGen}(\mathsf{pp})$**:** *This algorithm takes* pp *and returns a secret key* $\mathsf{sk} \in \mathcal{K}$*.*

$\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}^{00}, \mathsf{sk}^{01}, \mathsf{sk}^{10}, \mathsf{sk}^{11}, m^1, m^2, m^3, m^4)$**:** *On input four messages* m^1*,* m^2*,* m^3*,* $m^4 \in \mathcal{M}$ *and four secret keys* $\mathsf{sk}^{\beta_0\beta_1}$*,* $\beta_0, \beta_1 \in \{0,1\}$*, this algorithm returns a ciphertext* $\mathsf{ct} \in \mathcal{C}$*.*

$\mathsf{ct}' \leftarrow \mathsf{Eval}(\mathsf{pp}, \sigma_0, \sigma_1, \tau, \mathsf{ct})$**:** *On input a ciphertext* $\mathsf{ct} \in \mathcal{C}$ *and three functions* $\sigma_0, \sigma_1, \tau \in \mathcal{F}$*, where the set* $\mathcal{F}$ *is defined by* λ*, this algorithm returns a ciphertext* ct'*.*

$m' \leftarrow \mathsf{Dec}(\mathsf{pp}, \mathsf{sk}^0, \mathsf{sk}^1, \mathsf{ct})$**:** *This algorithm takes as input* pp *and two secret keys* $\mathsf{sk}^0, \mathsf{sk}^1$*, and returns a message* $m' \in \mathcal{M}$*.*

By R_{KGen}, R_{Enc}, and R_{Eval} we denote the randomness spaces for KGen, Enc, and Eval, respectively. Unless specified otherwise, randomness is always assumed to be sampled uniformly at random.

4.2 Construction

The following construction of gate KMHE is an extension of the basic KMHE scheme from Section 3.

Construction 15 (Gate KMHE). *Our gate KMHE scheme consists of the following algorithms.*

$\mathsf{Setup}(1^\lambda)$**:** *Generate and output the description of a bilinear group*

$$\mathsf{pp} = (q, g, h, g_T, \mathbb{G}, \mathbb{H}, \mathbb{G}_T, \mathrm{e}) \leftarrow \mathcal{G}(1^\lambda).$$

We define κ as the smallest positive and even integer such that $\kappa - \frac{3}{2}\log\kappa \geq \log q + 2\lambda$.[1] *These parameters define the following sets:*

- *the message space as $\mathcal{M} = \{0,1\}^\kappa$.*
- *the key space as $\mathcal{K} = HW_{\kappa,\kappa/2}$, the set of bit strings of length κ with Hamming weight $\kappa/2$. Note that we have $\mathcal{K} \subset \mathcal{M}$.*
- *the ciphertext space as $\mathcal{C} = \left((\mathbb{H}\times\mathbb{G})^\kappa \times \mathbb{G}_T\right)^4 \times \mathbb{G}^{2\kappa}$,*
- *the set of key- and message-homomorphisms as $\mathcal{F} = S_\kappa$, the set of permutations over κ elements,*
- *and randomness spaces $R_{\mathsf{KGen}} = \{0,1\}^\lambda$, $R_{\mathsf{Enc}} = \mathbb{G}^{2\kappa} \times \mathbb{Z}_q^{4\kappa} \times S_4$, and $R_{\mathsf{Eval}} = \mathbb{Z}_q^{4\kappa} \times \mathbb{Z}_q^* \times S_4$.*

$\mathsf{KGen}(\mathsf{pp}; r)$**:** *Use r to sample and output $\mathsf{sk} \leftarrow\!\!\$\ HW_{\kappa,\kappa/2}$.*

$\mathsf{Enc}(\mathsf{pp}, \mathsf{sk}^{00}, \mathsf{sk}^{01}, \mathsf{sk}^{10}, \mathsf{sk}^{11}, m^1, m^2, m^3, m^4; r)$**:** *Parse*

$$\begin{aligned}
&(g_{0,1}, \ldots, g_{0,\kappa}, g_{1,1}, \ldots, g_{1,\kappa},\\
&\ a_1^{(1)}, \ldots, a_\kappa^{(1)}, a_1^{(2)}, \ldots, a_\kappa^{(2)},\\
&\ a_1^{(3)}, \ldots, a_\kappa^{(3)}, a_1^{(4)}, \ldots, a_\kappa^{(4)}, \pi) = r \in R_{\mathsf{Enc}},\\
&(s_1^{0\alpha}, \ldots, s_\kappa^{0\alpha}) = \mathsf{sk}^{0\alpha} \in \{0,1\}^\kappa \quad \forall \alpha \in \{0,1\},\\
&(s_1^{1\beta}, \ldots, s_\kappa^{1\beta}) = \mathsf{sk}^{1\beta} \in \{0,1\}^\kappa \quad \forall \beta \in \{0,1\},\\
&(m_1^i, \ldots, m_\kappa^i) = m^i \in \{0,1\}^\kappa \quad \forall i \in [4].
\end{aligned}$$

Then compute

$$y_i = \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{0\alpha}} \cdot g_{1,j}^{s_j^{1\beta}}\right), h\right) \quad \forall \alpha, \beta \in \{0,1\},$$

where $i = \mathsf{Bin2Dec}(\alpha\beta) + 1$, and encrypt each message m^i by computing

$$c_{i,j} = \left(h^{a_j^{(i)}}, g_T^{m_j^i} \cdot y_i^{a_j^{(i)}}\right) \quad \forall j \in [\kappa].$$

Finally, set

$$\mathsf{ct}^i = (c_{i,1}, \ldots, c_{i,\kappa}, y_i) \quad \forall i \in [4],$$

and output ciphertext

$$\mathsf{ct} = \left(\pi\left(\mathsf{ct}^1, \mathsf{ct}^2, \mathsf{ct}^3, \mathsf{ct}^4\right), g_{0,1}, \ldots, g_{0,\kappa}, g_{1,1}, \ldots, g_{1,\kappa}\right).$$

$\mathsf{Eval}(\mathsf{pp}, \sigma_0, \sigma_1, \tau, \mathsf{ct}; r)$**:** *Parse*

$$\begin{aligned}
&(\hat{a}_1^{(1)}, \ldots, \hat{a}_\kappa^{(1)}, \hat{a}_1^{(2)}, \ldots, \hat{a}_\kappa^{(2)},\\
&\ \hat{a}_1^{(3)}, \ldots, \hat{a}_\kappa^{(3)}, \hat{a}_1^{(4)}, \ldots, \hat{a}_\kappa^{(4)}, d, \pi') = r \in R_{\mathsf{Eval}},\\
&(\mathsf{ct}^1, \mathsf{ct}^2, \mathsf{ct}^3, \mathsf{ct}^4, g_{0,1}, \ldots, g_{0,\kappa}, g_{1,1}, \ldots, g_{1,\kappa}) = \mathsf{ct},\\
&(c_{i,1}, \ldots, c_{i,\kappa}, y_i) = \mathsf{ct}^i \quad \forall i \in [4].
\end{aligned}$$

[1] Note that the gate KMHE construction requires a slightly larger lower bound on κ than the basic scheme. This is because the gate KMHE construction needs to tolerate additional leakage of information about the secret key. However, for reasonable concrete choices of q and λ, the size of κ is almost the same.

Apply key permutations σ_0 and σ_1 by computing

$$\left(\hat{g}_{\beta,\sigma_\beta(1)},\ldots,\hat{g}_{\beta,\sigma_\beta(\kappa)}\right)=(g_{\beta,1},\ldots,g_{\beta,\kappa})\quad\forall\beta\in\{0,1\},$$

and then apply message permutation τ by computing

$$\left(\hat{c}_{i,\tau(1)},\ldots,\hat{c}_{i,\tau(\kappa)}\right)=(c_{i,1},\ldots,c_{i,\kappa})\quad\forall i\in[4].$$

Next, we rerandomize the result by computing

$$\begin{aligned}
y'_i &= y_i^d \quad \forall i\in[4],\\
g'_{\beta,j} &= \hat{g}_{\beta,j}^d \quad \forall j\in[\kappa],\beta\in\{0,1\},\\
c'_{i,j} &= \left(\left(\hat{c}_{i,j,1}\cdot h^{\hat{a}_j^{(i)}}\right)^{1/d}, \hat{c}_{i,j,2}\cdot y_i^{\hat{a}_j^{(i)}}\right)\forall j\in[\kappa], i\in[4].
\end{aligned}$$

Finally, set

$$\mathsf{ct}'^i=(c'_{i,1},\ldots,c'_{i,\kappa},y'_i)\quad\forall i\in[4],$$

and define the output ciphertext as

$$\mathsf{ct}'=\left(\pi'\left(\mathsf{ct}'^1,\mathsf{ct}'^2,\mathsf{ct}'^3,\mathsf{ct}'^4\right),g'_{0,1},\ldots,g'_{0,\kappa},g'_{1,1},\ldots,g'_{1,\kappa}\right).$$

$\mathsf{Dec}(\mathsf{pp},\mathsf{sk}^0,\mathsf{sk}^1,\mathsf{ct})$**:** *Parse*

$$\begin{aligned}
&(s_1^\beta,\ldots,s_\kappa^\beta)=\mathsf{sk}^\beta\in\{0,1\}^\kappa\quad\forall\beta\in\{0,1\},\\
&(\mathsf{ct}^1,\mathsf{ct}^2,\mathsf{ct}^3,\mathsf{ct}^4,g_{0,1},\ldots,g_{0,\kappa},g_{1,1},\ldots,g_{1,\kappa})=\mathsf{ct},\\
&(c_{i,1},\ldots,c_{i,\kappa},y_i)=\mathsf{ct}^i\quad\forall i\in[4],
\end{aligned}$$

First we need to identify which of the four sub-ciphertexts ct^i can be correctly decrypted with the given keys. Compute

$$y=\mathrm{e}\left(\prod_{j\in[\kappa]}\left(g_{0,j}^{s_j^0}\cdot g_{1,j}^{s_j^1}\right),h\right)$$

and identify the y_i, $i\in[4]$, such that $y=y_i$. If there is more than one, then abort by outputting $\perp$. Otherwise, proceed by computing and outputting $m'=(m'_1,\ldots,m'_\kappa)$, where

$$g_T^{m'_j}=c_{i,j,2}\cdot\mathrm{e}\left(\prod_{n\in[\kappa]}\left(g_{0,n}^{s_n^0}\cdot g_{1,n}^{s_n^1}\right),c_{i,j,1}\right)^{-1}\quad\forall j\in[\kappa].$$

4.3 Correctness

Since there are four possible messages to decrypt, Dec must be able to identify, given two keys, which of the messages to decrypt. Correctness ensures that this is done correctly, up to some negligible error probability.

Definition 16 (Correctness). $\mathsf{GKMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *is* correct*, if for all* $\lambda \in \mathbb{N}$*,* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$*,* $\mathsf{sk}^{00}, \mathsf{sk}^{01}, \mathsf{sk}^{10}, \mathsf{sk}^{11} \leftarrow \mathsf{KGen}(\mathsf{pp})$*, and*

$$\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}^{00}, \mathsf{sk}^{01}, \mathsf{sk}^{10}, \mathsf{sk}^{11}, m^{00}, m^{01}, m^{10}, m^{11})$$

with $m^{00}, m^{01}, m^{10}, m^{11} \in \mathcal{M}$ *there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\Pr\Big[\mathsf{Dec}(\mathsf{pp}, \mathsf{sk}^{0\alpha}, \mathsf{sk}^{1\beta}, \mathsf{ct}) = m^{\alpha\beta}\Big] \geq 1 - \mathsf{negl}(\lambda) \quad \forall \alpha, \beta \in \{0,1\}.$$

Theorem 17. *Construction 15 is correct (Definition 16).*

Proof sketch. Verifying correctness is a straightforward calculation, almost exactly as for the basic scheme (cf. Appendix B). The only difference is that the decryption algorithm additionally computes y and identifies the y_i, $i \in [4]$, such that $y = y_i$. To show correctness, we additionally have to argue that $y_i \neq y_j$ for $i \neq j$, except for negligible probability. For any $y' \in \{y_1, y_2, y_3, y_4\}$ in a ciphertext it holds that $y' = \mathrm{e}\left(\mathbf{g_0}^{\mathsf{sk}^0}\mathbf{g_1}^{\mathsf{sk}^1}, h\right)$ for *uniform* $\mathbf{g_\nu}$ and secret keys sk^ν, $\nu \in \{0,1\}$. Our choice of κ and Lemma 12 then ensures that we can apply the LHL (Lemma 11) to show that every y' is indeed statistically close to uniform, and thus the probability that $y_i = y_j$ for $i \neq j$ is negligible. □

4.4 KMH Correctness

KMH correctness for gate KMHE is almost identical to the corresponding definition for the basic KMHE scheme (Definition 8), but extended to match the gate KMHE syntax.

Definition 18 (KMH Correctness). $\mathsf{GKMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *is* KMH-correct*, if for all* $\lambda \in \mathbb{N}$*,* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$*,* $\mathsf{sk}^{00}, \mathsf{sk}^{01}, \mathsf{sk}^{10}, \mathsf{sk}^{11} \leftarrow \mathsf{KGen}(\mathsf{pp})$*,* $m^1, m^2, m^3, m^4 \in \mathcal{M}$*,* $\sigma_0, \sigma_1, \tau \in \mathcal{F}$*,* $r_1 \in R_{\mathsf{Enc}}$*,* $r_2 \in R_{\mathsf{Eval}}$ *there exists randomness* $r' \in R_{\mathsf{Enc}}$ *such that*

$$\begin{aligned}&\mathsf{Eval}(\mathsf{pp}, \sigma_0, \sigma_1, \tau, \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}^{00}, \mathsf{sk}^{01}, \mathsf{sk}^{10}, \mathsf{sk}^{11}, m^1, m^2, m^3, m^4; r_1); r_2)\\ &= \mathsf{Enc}(\mathsf{pp}, \sigma_0(\mathsf{sk}^{00}), \sigma_0(\mathsf{sk}^{01}), \sigma_1(\mathsf{sk}^{10}), \sigma_1(\mathsf{sk}^{11}), \tau(m^1), \tau(m^2), \tau(m^3), \tau(m^4); r').\end{aligned}$$

Theorem 19. *Construction 15 is KMH-correct (Definition 18).*

The proof is a rather straightforward calculation, but due to the complexity of the scheme description somewhat tedious. It is deferred to Appendix H.

4.5 On Hiding The Key and Message Positions

When garbling a circuit, it is necessary that the resulting garbled gates hide the wire values that the input and output keys correspond to, as otherwise the keys obtained could leak wire assignments or even the secret circuit input. A common approach (see e.g. [LP09, AHKP22]) is to permute the ciphertexts of

the garbled gate as part of the garbling scheme construction, such that there is no longer a publicly known correspondence between the ciphertext position inside the garbled gate's ciphertext table, and the input and output wire values of the gate. Our construction follows this approach by randomly permuting the elements ct^1, ct^2, ct^3, and ct^4. Calls to Eval then refresh this permutation. This way, an adversarial decryptor will neither know which of the four encrypted messages he obtains through decryption, nor which of the four key combinations $(\mathsf{sk}^{00}, \mathsf{sk}^{10})$, $(\mathsf{sk}^{01}, \mathsf{sk}^{10})$, $(\mathsf{sk}^{00}, \mathsf{sk}^{11})$, or $(\mathsf{sk}^{01}, \mathsf{sk}^{11})$ he is in possession of.

We call this property of the gate KMHE scheme *position-hiding*. This intuitive understanding will suffice for the remainder of the paper, for the formal details we refer to Appendix G.

4.6 CPA-Security

We require IND-CPA security against an adversary which knows two out of the four secret keys used to encrypt a ciphertext, and therefore is able to decrypt one out of the four messages encrypted in the gate KMHE ciphertext. The other three messages should remain indistinguishable.

More precisely, we define the IND-CPA-security experiment for a gate KMHE in Figure 2. The experiment is parametrized by $(\beta_0, \beta_1) \in \{0,1\}^2$, which specify two of the four keys known to the adversary, and thereby also the message m^{β_0,β_1} that can be decrypted with these two keys.

1. The experiment generates parameters $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$ and samples a random bit $b \xleftarrow{\$} \{0,1\}$
2. The adversary receives pp and outputs seven messages:
   - One "known" message $m^{\beta_0\beta_1} \in \mathcal{M}$, for which the adversary will receive the corresponding secret keys
   - Three "challenge" message pairs $(m_0^{\beta_0,1-\beta_1}, m_1^{\beta_0,1-\beta_1})$, $(m_0^{1-\beta_0,\beta_1}, m_1^{1-\beta_0,\beta_1})$, and $(m_0^{1-\beta_0,1-\beta_1}, m_1^{1-\beta_0,1-\beta_1})$, each of which corresponds to at least one secret key that will remain unknown to the adversary.
3. The experiment generates four keys $\mathsf{sk}^{0,0}, \mathsf{sk}^{0,1}, \mathsf{sk}^{1,0}, \mathsf{sk}^{1,1} \leftarrow \mathsf{KGen}(\mathsf{pp})$ and uses them to encrypt the "known" message $m^{\beta_0\beta_1}$ along with the three "challenge" messages $m_b^{\beta_0,1-\beta_1}$, $m_b^{1-\beta_0,\beta_1}$, $m_b^{1-\beta_0,1-\beta_1}$ in the challenge ciphertext.
4. The adversary receives the challenge ciphertext and the two secret keys $\mathsf{sk}^{0,\beta_0}, \mathsf{sk}^{1,\beta_1}$, and outputs a bit b'. During this phase, the adversary also has access to three oracles which it can use to obtain encryptions under the unknown keys $\mathsf{sk}^{0,1-\beta_0}$, and $\mathsf{sk}^{0,1-\beta_1}$.
5. The adversary wins the game, if $b' = b$. We say that it breaks the IND-CPA-security of the gate KMHE scheme, if it wins with significantly better probability than guessing.

Definition 20 (IND-CPA Security). $\mathsf{GKMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *is IND-CPA-secure, if for all PPT adversaries* $\mathcal{A}$ *there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all* $\beta_0, \beta_1 \in \{0,1\}$ *it holds that*

$$\left| 2 \cdot \Pr\left[\mathsf{GKMHE\text{-}IND\text{-}CPA}^{\beta_0,\beta_1}_{\mathcal{A},\mathsf{GKMHE}}(1^\lambda) = 1 \right] - 1 \right| \leq \mathsf{negl}(\lambda),$$

where the GKMHE-IND-CPA *experiment is defined in Figure 2.*

$\mathsf{GKMHE\text{-}IND\text{-}CPA}^{\beta_0,\beta_1}_{\mathcal{A},\mathsf{GKMHE}}(1^\lambda)$

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda); b \leftarrow\!\!\$\ \{0,1\}$

$\left(m^{\beta_0,\beta_1}, m_0^{\beta_0,1-\beta_1}, m_1^{\beta_0,1-\beta_1}, m_0^{1-\beta_0,\beta_1}, m_1^{1-\beta_0,\beta_1}, m_0^{1-\beta_0,1-\beta_1}, m_1^{1-\beta_0,1-\beta_1}\right) \leftarrow \mathcal{A}(1^\lambda, \mathsf{pp})$

$m_0^{\beta_0,\beta_1}, m_1^{\beta_0,\beta_1} \leftarrow m^{\beta_0,\beta_1}$

$\mathsf{sk}^{0,0}, \mathsf{sk}^{0,1}, \mathsf{sk}^{1,0}, \mathsf{sk}^{1,1} \leftarrow \mathsf{KGen}(\mathsf{pp})$

$\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}^{0,0}, \mathsf{sk}^{0,1}, \mathsf{sk}^{1,0}, \mathsf{sk}^{1,1}, m_b^{0,0}, m_b^{0,1}, m_b^{1,0}, m_b^{1,1})$

$b' \leftarrow \mathcal{A}^{O(\cdot,\mathsf{sk}^{0,1-\beta_0},\cdot,\mathsf{sk}^{1,1-\beta_1},\cdot,\cdot,\cdot,\cdot),O(\cdot,\mathsf{sk}^{0,1-\beta_0},\cdot,\cdot,\cdot,\cdot,\cdot,\cdot),O(\cdot,\cdot,\cdot,\mathsf{sk}^{1,1-\beta_1},\cdot,\cdot,\cdot,\cdot)}(\mathsf{ct}, \mathsf{sk}^{0,\beta_0}, \mathsf{sk}^{1,\beta_1})$

return $(b' = b)$

Fig. 2. Definition of Game GKMHE-IND-CPA. The oracle $O(\mathsf{sk}^{0,\beta_0}, \mathsf{sk}^{0,1-\beta_0}, \mathsf{sk}^{1,\beta_1}, \mathsf{sk}^{1,1-\beta_1}, m^1, m^2, m^3, m^4)$, given secret keys $\mathsf{sk}^{0,\beta_0}, \mathsf{sk}^{0,1-\beta_0}, \mathsf{sk}^{1,\beta_1}, \mathsf{sk}^{1,1-\beta_1}$, and messages m^1, m^2, m^3, m^4, returns the output of $\mathsf{Enc}(\mathsf{pp}, \mathsf{sk}^{0,0}, \mathsf{sk}^{0,1}, \mathsf{sk}^{1,0}, \mathsf{sk}^{1,1}, m^1, m^2, m^3, m^4)$.

Theorem 21. *Construction 15 is IND-CPA-secure, assuming hardness of the SXDH-problem (Definition 4) in the bilinear groups defined by* pp*.*

The proof adopts the techniques from the security proof of the basic KMHE scheme (Theorem 10) to the gate KMHE scheme. See Appendix I for the full proof.

4.7 Key Privacy

Key privacy formalizes the notion that a transformed secret key should be statistically close to a fresh secret key.

Definition 22 (Key Privacy). $\mathsf{GKMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *has key privacy, if for all* $\lambda \in \mathbb{N}$, $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $\sigma \leftarrow\!\!\$\ \mathcal{F}$, $\mathsf{sk}, \mathsf{sk}' \leftarrow \mathsf{KGen}(\mathsf{pp})$ *it holds that*

$$\{\mathsf{pp}, \mathsf{sk}, \sigma(\mathsf{sk})\} \overset{s}{\approx} \{\mathsf{pp}, \mathsf{sk}, \mathsf{sk}'\},$$

Theorem 23. *Construction 15 has key privacy (Definition 22).*

Proof. This proof is the same as for Claim 7 in [AHKP22]. The key space is given by $HW_{\kappa,\kappa/2}$, consisting of all bit-strings with $\kappa/2$ 0's and $\kappa/2$ 1's. Permutations are bijective over $HW_{\kappa,\kappa/2}$, therefore $\sigma(\mathsf{sk})$ is distributed uniformly over $HW_{\kappa,\kappa/2}$, just like a fresh key sk'. □

4.8 KMH Rerandomizability

The next important security property is that of KMH rerandomizability. KMH rerandomizability essentially provides two guarantees:

1. Eval, when applied to a ciphertext, rerandomizes that ciphertext, resulting in a fresh-looking ciphertext. This holds as long as the randomness used to create the initial ciphertext was uniformly sampled, even if it is known to the adversary.
2. Applying a sequence of $t-1$ Eval operations with homomorphisms $(\sigma_{0,i}, \sigma_{1,i}, \tau_i)$, $i \in [t-1]$, to an honestly generated ciphertext yields a ciphertext that is computationally indistinguishable from a fresh ciphertext that uses the same secret keys (but with all the homomorphisms $\sigma_{0,i}, \sigma_{1,i}$ applied before encryption) to encrypt the same messages (but with all homomorphisms τ_i applied to the initial messages).

This must hold even against an adversary that "knows" the homomorphisms applied in Eval, the randomness used to generate the initial ciphertext, and all randomness used in Eval except for the last call of Eval. KMH rerandomizability is strongly related to KMH correctness. However, the latter focuses on the correctness of the ciphertext homomorphisms, and thus does not make any requirements about the distribution of ciphertexts, unlike KMH rerandomizability.

Definition 24 (KMH Rerandomizability). *A gate KMHE scheme* $\mathsf{GKMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *is* KMH-rerandomizable*, if for any PPT adversary* $\mathcal{A}$ *there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\left|2 \cdot \Pr\left[\mathsf{GKMHE\text{-}RERAND}_{\mathcal{A},\mathsf{GKMHE}}(1^\lambda) = 1\right] - 1\right| \leq \mathsf{negl}(\lambda)$$

where the $\mathsf{GKMHE\text{-}RERAND}$ *experiment is defined in Figure 3.*

Theorem 25. *Construction 15 is KMH-rerandomizable, assuming hardness of the DDH-problem (Definition 1) over the group* $\mathbb{G}$ *defined by* pp.

Proof sketch. The correct application of the homomorphic computations follows via a similar straightforward calculation as for the KMH correctness proof (Theorem 19). The argument that the final rerandomized ciphertext is distributed as a fresh encryption is based on generalized DDH (Lemma 2) over the group $\mathbb{G}$. Roughly, the uniformity of the initial randomness r_1 guarantees that all the group elements $g_{\beta,j}$ for all $\beta \in \{0,1\}, j \in [\kappa]$, in r_1 are uniformly distributed. Uniformity of r_t, the fact that r_t is hidden from the adversary $\mathcal{A}$, and generalized DDH then ensure that the rerandomized bases $g^d_{\beta,j}$, for $d \in \mathbb{Z}^*_q$ given in r_t, look like fresh group elements. We refer to Appendix J for the full proof.

4.9 Key Privacy Under Leakage

Our construction of RGS from gate KMHE will require another strong security property of the gate KMHE scheme that we call *key privacy under leakage*

$\mathsf{GKMHE\text{-}RERAND}_{\mathcal{A},\mathsf{GKMHE}}(1^\lambda)$

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$

$(t, (\sigma_{0,i}, \sigma_{1,i}, \tau_i)_{i\in[t-1]}, m^1, m^2, m^3, m^4) \leftarrow \mathcal{A}(\mathsf{pp}, 1^\lambda)$

$r^{00}, r^{01}, r^{10}, r^{11} \leftarrow\!\$\ R_{\mathsf{KGen}};\ \mathsf{sk}^{\beta_0\beta_1} \leftarrow \mathsf{KGen}(\mathsf{pp}; r^{\beta_0\beta_1}) \quad \forall \beta_0, \beta_1 \in \{0,1\}$

$r_1 \leftarrow\!\$\ R_{\mathsf{Enc}};\ r_2, \ldots, r_t, r' \leftarrow\!\$\ R_{\mathsf{Eval}}$

$b \leftarrow\!\$\ \{0,1\}$

if $b = 0$ **then**

$\quad \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}^{00}, \mathsf{sk}^{01}, \mathsf{sk}^{10}, \mathsf{sk}^{11}, m^1, m^2, m^3, m^4; r_1)$

$\quad \mathsf{ct}' \leftarrow \mathsf{Eval}(\mathsf{pp}, \sigma_{0,t-1}, \sigma_{1,t-1}, \tau_{t-1}, \ldots \mathsf{Eval}(\sigma_{0,1}, \sigma_{1,1}, \tau_1, \mathsf{ct}; r_2) \ldots; r_t)$

else

$\quad \mathsf{sk}'^{\beta_0\beta_1} = \sigma_{\beta_0,t-1}(\sigma_{\beta_0,t-2}(\ldots(\mathsf{sk}^{\beta_0\beta_1})\ldots)) \quad \forall \beta_0, \beta_1 \in \{0,1\}$

$\quad m'^i = \tau_{t-1}(\tau_{t-2}(\ldots(m^i)\ldots)) \quad \forall i \in [4]$

$\quad \mathsf{ct}' \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}'^{00}, \mathsf{sk}'^{01}, \mathsf{sk}'^{10}, \mathsf{sk}'^{11}, m'^1, m'^2, m'^3, m'^4; r')$

endif

$b' \leftarrow \mathcal{A}(\mathsf{ct}', r^{00}, r^{01}, r^{10}, r^{11}, r_1, \ldots, r_{t-1})$

return $(b' = b)$

Fig. 3. Definition of Game GKMHE-RERAND.

(KPUL). Intuitively, KPUL guarantees that a ciphertext that was evaluated with key homomorphisms σ_0 and σ_1 is indistinguishable from a fresh ciphertext, even given some information about the homomorphisms. Concretely, for our RGS construction we will have to consider a setting where the adversary first generates and outputs a ciphertext (and thus knows all four secret keys and the encrypted messages), and then random key homomorphisms σ_0 and σ_1 are applied, hidden from the adversary, to rerandomize the keys of the original ciphertext. Subsequently, the adversary receives back the resulting ciphertext. We want that the resulting ciphertext is computationally indistinguishable from a ciphertext encrypted with *independent* keys.

There are two complications:

1. We have to consider a setting where the same key homomorphism $\sigma \in \{\sigma_0, \sigma_1\}$ is applied to *multiple* gate KMHE ciphertexts. This is because in the RGS construction secret keys and homomorphisms are chosen for *wires* of the garbled circuit; the same wire might be an input to $Q \geq 1$ gates in a circuit, and thus the same keys and homomorphisms are used in Q gate KMHE ciphertexts. These Q ciphertexts might encrypt different messages and use different secret keys on their other input wires. Therefore we have to consider a setting where the adversary outputs the keys $(\mathsf{sk}^0, \mathsf{sk}^1)$ shared across all Q ciphertexts, along with specifications α_i, $i \in [Q]$, whether these

correspond to the "left" or "right" input wire of the i^{th} gate, as well as two additional secret keys and four messages for each of the $Q \geq 1$ ciphertexts.
2. Along with the resulting challenge ciphertexts, the adversary (which evaluates the garbled circuit) will also receive one of two rerandomized secret keys of the wire. This *leaks* some information about the homomorphism σ. Therefore we have to consider a setting where the adversary outputs secret keys sk^0, sk^1 and receives the rerandomized wire key $\sigma(\mathsf{sk}^\beta)$ as additional leakage about σ along with the challenge ciphertexts. In the challenge ciphertexts, either the rerandomization $\sigma(\mathsf{sk}^{1-\beta})$ of the other wire key $\mathsf{sk}^{1-\beta}$ is used, or an independent random key sk^*, and the adversary has to distinguish between these two cases.

Taking these requirements into account yields the KPUL security experiment in Figure 4. Note that the figure actually defines two security experiments, one for each $\beta \in \{0,1\}$.

Definition 26 (Key Privacy under Leakage). *We say that a gate KMHE scheme* $\mathsf{GKMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *has* key privacy under leakage*, if for any PPT adversary* $\mathcal{A}$ *there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all* $\beta \in \{0,1\}$ *it holds that*

$$\left| 2 \cdot \Pr\left[\mathsf{GKMHE\text{-}KPUL}^{\beta}_{\mathcal{A},\mathsf{GKMHE}}(1^\lambda) = 1 \right] - 1 \right| \leq \mathsf{negl}(\lambda)$$

where the GKMHE-KPUL *experiment is defined in Figure 4.*

Theorem 27. *Construction 15 has key privacy under leakage (Definition 26), assuming hardness of the DDH-problem (Definition 1) in the group* $\mathbb{G}$ *defined by* pp.

Building blocks of the proof. The following definitions are taken from [DRS04] and were used in a similar context by Naor and Segev [NS09] to prove key-leakage resilience of the circular-secure encryption scheme of [BHHO08].

The *conditional min-entropy* of a random variable X, conditioned on some information y, is defined as $H_\infty(X \mid y) = -\log\left(\max_{x \in \mathrm{supp}(X)} \Pr[X = x \mid y]\right)$. The *average min-entropy* of a random variable X conditioned on the value of a random variable Y is then defined as

$$\tilde{H}_\infty(X \mid Y) = -\log \mathbb{E}_{y \leftarrow Y}\left[2^{-H_\infty(X|y)}\right].$$

We use the following instantiation of the generalized left-over hash lemma from [DRS04], which uses that the inner-product over a field $\mathbb{Z}_q$ is a pairwise independent hash function (see also [NS09]).This variant of the left-over hash lemma also takes leakage of the secret into account.

Lemma 28 (LHL with leakage). *Let* q *be a prime,* $\kappa \in \mathbb{N}$, $\epsilon > 0$, *and let* $X = (X_1, \ldots, X_\kappa) \in \{0,1\}^\kappa$ *and* Y *be random variables such that* $\tilde{H}_\infty(X \mid Y) \geq \log q + 2\log(1/\epsilon)$. *Then for* $a \leftarrow\!\!\$\, \mathbb{Z}_q^\kappa$ *it holds that*

$$\Delta((a, \sum_{i \in [\kappa]} a_i X_i, Y), (a, U(\mathbb{Z}_q), Y)) \leq \epsilon.$$

$\mathsf{GKMHE\text{-}KPUL}^{\beta}_{\mathcal{A},\mathsf{GKMHE}}(1^\lambda)$:

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$

$(\mathsf{sk}^0, \mathsf{sk}^1, (m_i^{00}, m_i^{01}, m_i^{10}, m_i^{11}, \alpha_i, \mathsf{sk}_i^{1-\alpha_i,0}, \mathsf{sk}_i^{1-\alpha_i,1})_{i\in[Q]}) \leftarrow \mathcal{A}(\mathsf{pp}, 1^\lambda)$

$\sigma \leftarrow\!\!\$\ \mathcal{F};\ b \leftarrow\!\!\$\ \{0,1\};\ \mathsf{sk}^* \leftarrow \mathsf{KGen}(\mathsf{pp})$

if $b = 0$ **then**

$\quad \widehat{\mathsf{sk}}_i^{\alpha_i,1-\beta} \leftarrow \sigma(\mathsf{sk}^{1-\beta}) \quad \forall i \in [Q]$

else

$\quad \widehat{\mathsf{sk}}_i^{\alpha_i,1-\beta} \leftarrow \mathsf{sk}^* \quad \forall i \in [Q]$

endif

$\widehat{\mathsf{sk}}_i^{\alpha_i,\beta} \leftarrow \sigma(\mathsf{sk}^{\beta}) \quad \forall i \in [Q]$

$\widehat{\mathsf{sk}}_i^{1-\alpha_i,j} \leftarrow \mathsf{sk}_i^{1-\alpha_i,j}$ for $\forall i \in [Q]$ and $j \in \{0,1\}$

$\mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{pp}, \widehat{\mathsf{sk}}_i^{0,0}, \widehat{\mathsf{sk}}_i^{0,1}, \widehat{\mathsf{sk}}_i^{1,0}, \widehat{\mathsf{sk}}_i^{1,1}, m_i^{00}, m_i^{01}, m_i^{10}, m_i^{11}) \quad \forall i \in [Q]$

$b' \leftarrow \mathcal{A}\left(\sigma(\mathsf{sk}^{\beta}), (\mathsf{ct}_i)_{i\in[Q]}\right)$

return $(b' = b)$

Fig. 4. Definition of Game $\mathsf{GKMHE\text{-}KPUL}^{\beta}$.

where $U(\mathbb{Z}_q)$ is the uniform distribution over $\mathbb{Z}_q$.

The following lemma from [GHV10] gives a lower-bound on the average-min entropy of a permuted key, even if some leakage about the key transformation function is present.

Lemma 29 (Lemma 9 in [GHV10]). *Let $\kappa \geq 2$ be an even integer, let $\mathsf{sk}_1, \mathsf{sk}_2 \leftarrow\!\!\$\ HW_{\kappa,\kappa/2}$ and $\sigma \leftarrow\!\!\$\ S_\kappa$. Then*

$$H_\infty(\sigma(\mathsf{sk}_1) \mid \mathsf{sk}_1, \mathsf{sk}_2, \sigma(\mathsf{sk}_2)) \geq \kappa - \frac{3}{2}\log\kappa.$$

Finally, we will use a variant of Lemma 13, extended with leakage. The proof of the following lemma follows almost exactly the proof of the non-leakage variant Lemma 13, except that we replace the use of the LHL (Lemma 11) with the leakage variant (Lemma 28).

Lemma 30. *Let $\eta, \nu \in \mathbb{N}$, $\mathbf{g}_1, \ldots, \mathbf{g}_\eta \leftarrow\!\!\$\ \mathbb{G}^\nu$, where $\mathbb{G}$ is a group of prime order q where the DDH assumption (Definition 1) holds, Y be a random variable, and $\mathsf{sk} \in \{0,1\}^\nu$ such that $\tilde{H}_\infty(\mathsf{sk} \mid Y) \geq \log q + 2\lambda$. Then it holds that*

$$\left\{\mathbf{g}_1, \ldots, \mathbf{g}_\eta, \mathbf{g}_1^{\mathsf{sk}}, \ldots, \mathbf{g}_\eta^{\mathsf{sk}}, Y\right\} \overset{c}{\approx} \left\{\mathbf{g}_1, \ldots, \mathbf{g}_\eta, g_1', \ldots, g_\eta', Y\right\},$$

where $g_1', \ldots, g_\eta' \leftarrow\!\!\$\ \mathbb{G}$.

Using these lemmas we can now proceed with proving key privacy under leakage of Construction 15.

Proof of Theorem 27. Similarly to the proof of IND-CPA security we will only consider the case $\beta = 1$, so the key $\sigma(\mathsf{sk}^0)$ remains secret and the adversary $\mathcal{A}$ is given $\sigma(\mathsf{sk}^1)$. The case $\beta = 0$ follows analogously. Furthermore, we assume that $\alpha_1 = \cdots = \alpha_Q = 0$ for all the α_i specified by the adversary, in order to simplify our notation. Since the gate KMHE scheme treats the keys assigned to a "left" or "right" input wire identically, this is without loss of generality.

We describe a series of hybrid distributions from case $b = 0$ to case $b = 1$ of the game $\mathsf{GKMHE\text{-}KPUL}^0_{\mathcal{A},\mathsf{GKMHE}}$. We will then prove these hybrids indistinguishable.

Hybrid 1: Case $b = 0$ of the game $\mathsf{GKMHE\text{-}KPUL}^0_{\mathcal{A},\mathsf{GKMHE}}$. We write $\mathsf{ct}_1, \ldots, \mathsf{ct}_Q$, where

$$\mathsf{ct}_i = \left(\mathsf{ct}_i^{00}, \mathsf{ct}_i^{01}, \mathsf{ct}_i^{10}, \mathsf{ct}_i^{11}, g_{i,0,1}, \ldots, g_{i,0,\kappa}, g_{i,1,1}, \ldots, g_{i,1,\kappa}\right)$$

for $i \in [Q]$, to denote the challenge ciphertexts given to the adversary. Furthermore, we write $\mathsf{ct}_i^{\delta_0\delta_1} = (c_{i,\delta_0\delta_1,1}, \ldots, c_{i,\delta_0\delta_1,\kappa}, y_{i,\delta_0\delta_1})$ for all $i \in [Q], \delta_0, \delta_1 \in \{0,1\}$ and define

$$\mathbf{g}_{i,0} = (g_{i,0,1}, \ldots, g_{i,0,\kappa}) \qquad \text{and} \qquad \mathbf{g}_{i,1} = (g_{i,1,1}, \ldots, g_{i,1,\kappa})$$

for all $i \in [Q]$.

Hybrid 2: For Hybrid 2, we alter how each ct_i, $i \in [Q]$ is created by Enc, specifically how the values $y_{i,00}$ and $y_{i,01}$ are computed. To this end, parse the permuted keys as $\sigma(\mathsf{sk}^{0,0}) = (s_1, \ldots, s_\kappa)$. Then in the computation of

$$\mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{pp}, \sigma(\mathsf{sk}^0), \widehat{\mathsf{sk}}_i^{0,1}, \widehat{\mathsf{sk}}_i^{1,0}, \widehat{\mathsf{sk}}_i^{1,1}, m_i^{00}, m_i^{01}, m_i^{10}, m_i^{11}),$$

during computation of ct_i, samples $g_i' \leftarrow\!\!\$\ \mathbb{G}$ and then computes

$$y_{i,00} = \mathrm{e}\left(g_i' \cdot \mathbf{g}_{i,1}^{\widehat{\mathsf{sk}}_i^{1,0}}, h\right),$$
$$y_{i,01} = \mathrm{e}\left(g_i' \cdot \mathbf{g}_{i,1}^{\widehat{\mathsf{sk}}_i^{1,1}}, h\right).$$

It then proceeds with the rest of the computation of ct_i.

Hybrid 3: Case $b = 1$ of the game $\mathsf{GKMHE\text{-}KPUL}^0_{\mathcal{A},\mathsf{GKMHE}}$.

We now show that the hybrids are indistinguishable.

Hybrid 1 $\overset{\mathbf{c}}{\approx}$ Hybrid 2: The proof of this step follows largely the proof of indistinguishability of Game 1 and Game 2 in the proof of IND-CPA-security (Theorem 21), with a slight difference in how we lower-bound the entropy of the secret key $\sigma(\mathsf{sk}^0)$ due to the additional leakage.

In Game 1, each $y_{i,00}$ and $y_{i,01}$ in ct_i is computed as

$$y_{i,00} = \mathrm{e}\left(\mathbf{g}_{i,0}^{\sigma(\mathsf{sk}^0)} \cdot \mathbf{g}_{i,1}^{\widehat{\mathsf{sk}}_i^{1,0}}, h\right),$$
$$y_{i,01} = \mathrm{e}\left(\mathbf{g}_{i,0}^{\sigma(\mathsf{sk}^0)} \cdot \mathbf{g}_{i,1}^{\widehat{\mathsf{sk}}_i^{1,1}}, h\right).$$

for all $i \in [Q]$. Our choice of κ and Lemma 29 ensures that

$$\tilde{H}_\infty\left(\sigma(\mathsf{sk}^0) \mid \mathsf{sk}^0, \mathsf{sk}^1, \sigma(\mathsf{sk}^1)\right) \geq \kappa - \frac{3}{2}\log\kappa \geq \log q + 2\lambda.$$

Since DDH holds over $\mathbb{G}$, we can then apply Lemma 30, resulting in

$$\left\{\mathbf{g}_{1,0}, \ldots, \mathbf{g}_{Q,0}, \mathbf{g}_{1,0}^{\sigma(\mathsf{sk}^0)}, \ldots, \mathbf{g}_{Q,0}^{\sigma(\mathsf{sk}^0)}\right\} \overset{\mathrm{c}}{\approx} \left\{\mathbf{g}_{1,0}, \ldots, \mathbf{g}_{Q,0}, g'_1, \ldots, g'_Q\right\}. \quad (2)$$

where $g'_1, \ldots, g'_Q \leftarrow\!\!\$\ \mathbb{G}$. The right side of Equation (2) thus corresponds to Hybrid 2.

Hybrid 2 $\overset{\mathrm{c}}{\approx}$ Hybrid 3: To insert a fresh key sk^*, we now apply Lemma 13. By our choice of κ and Lemma 12 we then have

$$H_\infty(\mathsf{sk}^*) \geq \kappa - \frac{3}{2}\log\kappa \geq \kappa - \frac{1}{2}\log\kappa - 1/2 \geq \log q + 2\lambda.$$

We can then apply Lemma 13, resulting in

$$\left\{\mathbf{g}_{1,0}, \ldots, \mathbf{g}_{Q,0}, g'_1, \ldots, g'_Q\right\} \overset{\mathrm{c}}{\approx} \left\{\mathbf{g}_{1,0}, \ldots, \mathbf{g}_{Q,0}, \mathbf{g}_{1,0}^{\mathsf{sk}^*}, \ldots, \mathbf{g}_{Q,0}^{\mathsf{sk}^*}\right\}. \quad (3)$$

The right side of Equation (3) then corresponds to Hybrid 3.

This completes the proof. □

5 Rerandomizable Garbling Schemes

We define RGS, mostly following [GHV10, AHKP22]. The main difference is a new definition of rerand privacy plus a Setup algorithm to generate public parameters.

Let $(\mathcal{F}^\lambda_{\mathsf{RGS}})_{\lambda\in\mathbb{N}}$ be a family of function sets $\mathcal{F}^\lambda_{\mathsf{RGS}}$ where $\mathcal{F}^\lambda_{\mathsf{RGS}} = \{f : \mathcal{X} \to \mathcal{Y}\}$ is a set of functions f. Additionally, we define leakage function family $(\phi_\lambda)_{\lambda\in\mathbb{N}}$ to be a family of functions $\phi_\lambda : \mathcal{F}^\lambda_{\mathsf{RGS}} \to \{0,1\}^*$ which map a function $f \in \mathcal{F}^\lambda_{\mathsf{RGS}}$ to the allowed leakage of the garbling scheme $\phi_\lambda(f)$.

Definition 31 (Rerandomizable Garbling Scheme). *A* rerandomizable garbling scheme *(RGS) for $(\mathcal{F}^\lambda_{\mathsf{RGS}})_{\lambda\in\mathbb{N}}$ and leakage function family $(\phi_\lambda)_{\lambda\in\mathbb{N}}$ is a tuple of PPT algorithms $\mathsf{RGS} = (\mathsf{Setup}, \mathsf{Gb}, \mathsf{Rerand}, \mathsf{En}, \mathsf{Ev})$ with the following syntax.*

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$**:** *This algorithm takes as input the security parameter λ and returns public parameters* pp*. The public parameters define a label space $\mathcal{L}$, function family $\mathcal{F}^\lambda_{\mathsf{RGS}}$ and leakage function ϕ_λ.*

$(\mathcal{C}, L) \leftarrow \mathsf{Gb}(\mathsf{pp}, f)$**:** *Given a function $f \in \mathcal{F}^\lambda_{\mathsf{RGS}}$, this algorithm returns a garbled circuit $\mathcal{C}$ and a set of input labels $L \in \mathcal{L}$.*

$(\mathcal{C}', \pi_{\mathsf{En}}) \leftarrow \mathsf{Rerand}(\mathsf{pp}, \mathcal{C})$**:** *Given a garbled circuit $\mathcal{C}$, this algorithm returns a garbled circuit $\mathcal{C}'$ and a label encoding function $\pi_{\mathsf{En}} : \mathcal{L} \to \mathcal{L}$.*

$I \leftarrow \mathsf{En}(\mathsf{pp}, L, x)$**:** *This algorithm takes a set of input labels L and an input $x \in \mathcal{X}$, and outputs an input encoding I.*

$y \leftarrow \mathsf{Ev}(\mathsf{pp}, \mathcal{C}, I)$**:** *Given a garbled circuit $\mathcal{C}$ and an input encoding I, this algorithm outputs a $y \in \mathcal{Y}$.*

An RGS must fulfill correctness, garbling privacy, and rerand privacy as defined below. As pointed out in [AHKP22], correctness and rerand privacy together imply correctness of rerandomization.

Definition 32 (Correctness). *We say that* RGS *is correct, if for all $\lambda \in \mathbb{N}$, $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $f \in \mathcal{F}^\lambda_{\mathsf{RGS}}$, and $x \in \mathcal{X}$ there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\Pr\left[y = f(x) \;\middle|\; \begin{array}{c} (\mathcal{C}, L) \leftarrow \mathsf{Gb}(\mathsf{pp}, f) \\ I \leftarrow \mathsf{En}(\mathsf{pp}, L, x) \\ y = \mathsf{Ev}(\mathsf{pp}, \mathcal{C}, I) \end{array} \right] \geq 1 - \mathsf{negl}(\lambda).$$

Just like for standard garbled circuits, garbling privacy ensures that the garbled function f and circuit input x remain hidden, except for the function output $f(x)$ and the allowed leakage $\phi_\lambda(f)$.

Definition 33 (Garbling Privacy). $\mathsf{RGS} = (\mathsf{Setup}, \mathsf{Gb}, \mathsf{Rerand}, \mathsf{En}, \mathsf{Ev})$ *has garbling privacy, if there exists an efficient simulator* $\mathsf{Sim}_{\mathsf{Gb}}$ *such that for all $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, and for every $f \in \mathcal{F}^\lambda_{\mathsf{RGS}}$, $x \in \mathcal{X}$ it holds that*

$$\{\mathcal{C}, I\}_{\substack{(\mathcal{C}, L) \leftarrow \mathsf{Gb}(\mathsf{pp}, f) \\ I \leftarrow \mathsf{En}(\mathsf{pp}, L, x)}} \overset{c}{\approx} \{\mathsf{Sim}_{\mathsf{Gb}}(\mathsf{pp}, f(x), \phi_\lambda(f))\}.$$

Similar to KMH rerandomizability of gate KMHE, rerand privacy of RGS guarantees that rerandomized garbled circuits are indistinguishable from fresh garbled circuits.

Definition 34 (Rerand Privacy). *We say that* $\mathsf{RGS} = (\mathsf{Setup}, \mathsf{Gb}, \mathsf{Rerand}, \mathsf{En}, \mathsf{Ev})$ *has rerand privacy, if for any PPT adversary $\mathcal{A}$ there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\Pr[\mathsf{RER\text{-}PRIV}_{\mathcal{A}, \mathsf{RGS}}(1^\lambda) = 1] \leq 1/2 + \mathsf{negl}(\lambda),$$

where the $\mathsf{RER\text{-}PRIV}$ *experiment is defined in Figure 5.*

$\mathsf{RER\text{-}PRIV}_{\mathcal{A},\mathsf{RGS}}(1^\lambda)$

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda);\ (f, x, 1^t) \leftarrow \mathcal{A}(\mathsf{pp}, 1^\lambda)$
$r_1 \leftarrow\!\$\ R_{\mathsf{Gb}};\ (r_2, \ldots, r_{t-1}) \leftarrow\!\$\ R_{\mathsf{Rerand}};\ b \leftarrow\!\$\ \{0,1\}$
if $b = 0$ **then**
  $r_t \leftarrow\!\$\ R_{\mathsf{Rerand}};\ (\mathcal{C}_1, L_1) \leftarrow \mathsf{Gb}(\mathsf{pp}, f; r_1)$
  $(\mathcal{C}_{i+1}, \pi_{\mathsf{En}}^{i+1}) \leftarrow \mathsf{Rerand}(\mathsf{pp}, \mathcal{C}_i; r_{i+1})$ for $i \in [t-1]$
  $L \leftarrow \pi_{\mathsf{En}}^t(\cdots \pi_{\mathsf{En}}^2(L_1)\cdots);\ I \leftarrow \mathsf{En}(L, x); \mathcal{C} \leftarrow \mathcal{C}_t$
else
  $(\mathcal{C}, L) \leftarrow \mathsf{Gb}(\mathsf{pp}, f);\ I \leftarrow \mathsf{En}(L, x)$
endif
$b' \leftarrow \mathcal{A}((r_1, \ldots, r_{t-1}), \mathcal{C}, I)$
return $(b' = b)$

Fig. 5. Definition of Game RER-PRIV.

5.1 RGS from Gate KMHE

We proceed by constructing the RGS scheme. It largely follows Construction 1 in [AHKP22], which in turn follows standard garbled circuits [Yao86, LP09], but using gate KMHE instead of the sharable KMHE from [AHKP22].

Let $\mathsf{GKMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ be a gate KMHE scheme with associated key space $\mathcal{K}$. We also define

- the function family $\mathcal{F}_{\mathsf{RGS}}^\lambda = \{f : \{0,1\}^{\ell_{\mathsf{in}}(f)} \to \{0,1\}^{\ell_{\mathsf{out}}(f)}\}$ as the set of Boolean circuits of size polynomial in 1^λ, and
- label space $\mathcal{L} = \mathcal{K}^{2\ell_{\mathsf{in}}}$.

We generally consider Boolean circuits consisting of binary gates with two inputs and one output. The fan-out is arbitrary, meaning that the output wire can be connected to arbitrarily many gates. Given a circuit $f \in \mathcal{F}_{\mathsf{RGS}}^\lambda$, we use $\mathcal{W}$ to denote the list of wires, $\mathcal{W}_\mathcal{O}$ the list of output wires, and $\mathcal{W}_I$ the list of input wires. M denotes the number of wires and N the number of gates. A gate g_i is described by a tuple $g_i = (w_\ell, w_r, w_o, op)$, where w_ℓ and w_r are the left and right input wires, respectively, w_o the output wire and op is the gate function, i.e. any function $op : \{0,1\} \times \{0,1\} \mapsto \{0,1\}$. The leakage function ϕ_λ, given a Boolean circuit, outputs its topology, meaning everything but the gate operation op of each gate.

Construction 35 (RGS from Gate KMHE).

$\mathsf{Setup}(1^\lambda)$**:** *Generate* GKMHE *public parameters* $\mathsf{pp}_{\mathsf{GKMHE}} \leftarrow \mathsf{Setup}(1^\lambda)$ *and sample output labels* $\mathsf{sk}_0, \mathsf{sk}_1 \leftarrow \mathsf{KGen}(\mathsf{pp}_{\mathsf{GKMHE}})$. *Output*

$$\mathsf{pp} \leftarrow (\mathsf{pp}_{\mathsf{GKMHE}}, \mathsf{sk}_0, \mathsf{sk}_1)\,.$$

$\mathsf{Gb}(\mathsf{pp}, f)$**:** *Parse f as a series of boolean gates $g_1, \ldots, g_N$, ordered topologically. For every wire $w_i \in \mathcal{W} \setminus \mathcal{W}_\mathcal{O}$ sample labels $\mathsf{sk}^0_{w_i}, \mathsf{sk}^1_{w_i} \leftarrow \mathsf{KGen}(\mathsf{pp}_{\mathsf{GKMHE}})$. The output wire labels have already been fixed at setup. Then, for every gate $g_i = (w_\ell, w_r, w_o, op)$, compute the garbling of g_i as*

$$G_i \leftarrow \mathsf{Enc}(\mathsf{pp}_{\mathsf{GKMHE}}, \mathsf{sk}^0_{w_\ell}, \mathsf{sk}^1_{w_\ell}, \mathsf{sk}^0_{w_r}, \mathsf{sk}^1_{w_r}, \mathsf{sk}^{op(0,0)}_{w_o}, \mathsf{sk}^{op(0,1)}_{w_o}, \mathsf{sk}^{op(1,0)}_{w_o}, \mathsf{sk}^{op(1,1)}_{w_o})$$

Finally, output the garbling $\mathcal{C} = (\phi_\lambda(f), G_1, \ldots, G_N)$ and the input labels $L = \{\mathsf{sk}^0_{w_i}, \mathsf{sk}^1_{w_i}\}_{w_i \in \mathcal{W}_I}$.

$\mathsf{Rerand}(\mathsf{pp}, \mathcal{C})$**:** *This algorithm relies on the circuit topology described in $\phi_\lambda(f)$ to perform a consistent rerandomization. For every wire $w_i \in \mathcal{W} \setminus \mathcal{W}_\mathcal{O}$, sample a key transformation function $\sigma_{w_i} \xleftarrow{\$} \mathcal{F}$. For all output wires $w_i \in \mathcal{W}_\mathcal{O}$, let $\sigma_{w_i} = \mathsf{id} \in \mathcal{F}$ be the identity function. Now parse $\mathcal{C} = (\phi_\lambda(f), G_1, \ldots, G_N)$. To rerandomize the garbling G_i of g_i for all $i \in [N]$, compute*

$$G'_i \leftarrow \mathsf{Eval}(\mathsf{pp}_{\mathsf{GKMHE}}, \sigma_{w_\ell}, \sigma_{w_r}, \sigma_{w_o}, G_i).$$

Finally, output $\mathcal{C}' = (\phi_\lambda(f), G'_1, \ldots, G'_N)$ and π_{En}, where π_{En} is defined by

$$\pi_{\mathsf{En}} : \mathcal{L} \to \mathcal{L}$$
$$\{\mathsf{sk}^0_{w_i}, \mathsf{sk}^1_{w_i}\}_{w_i \in \mathcal{W}_I} \mapsto \{\sigma_{w_i}(\mathsf{sk}^0_{w_i}), \sigma_{w_i}(\mathsf{sk}^1_{w_i})\}_{w_i \in \mathcal{W}_I}.$$

$\mathsf{En}(\mathsf{pp}, L, x)$**:** *Given input labels $L = \{\mathsf{sk}^0_{w_i}, \mathsf{sk}^1_{w_i}\}_{w_i \in \mathcal{W}_I}$ and an input $x \in \{0,1\}^{\ell_{\mathsf{in}}}$, output input encoding $I = \{\mathsf{sk}^{x_i}_{w_i}\}_{w_i \in \mathcal{W}_I}$.*

$\mathsf{Ev}(\mathsf{pp}, \mathcal{C}, I)$**:** *Parse $I = \{\mathsf{sk}_{w_i}\}_{w_i \in \mathcal{W}_I}$ and $\mathsf{pp} = (\mathsf{pp}_{\mathsf{GKMHE}}, \mathsf{sk}_0, \mathsf{sk}_1)$. The circuit is evaluated by decrypting gates using Dec in topological order.*

1. *The input labels in I are used to evaluate the first gates, i.e. the ones that only use input wires as inputs.*
2. *Decryption of a garbled gate then provides the evaluator with the output label for that gate, which correspond to the input labels of subsequent gates, allowing them to proceed topologically.*
3. *Eventually, labels $\mathsf{sk}_{w_{o_i}}$ for all the output wires $\mathcal{W}_\mathcal{O} = (w_{o_1}, \ldots, w_{o_{\ell_{\mathsf{out}}}})$ are obtained. The output can then be decoded by checking whether each output label is equal to sk_0 or sk_1, resulting in output $y \in \{0,1\}^{\ell_{\mathsf{out}}}$ with $y_i = 0$ if $\mathsf{sk}_{w_{o_i}} = \mathsf{sk}_0$ and 1 otherwise.*

We will now argue correctness, garbling privacy, and rerand privacy of Construction 35. The proofs largely follow the standard security proofs for (rerandomizable) garbled circuits, but adopted to gate KMHE.

Theorem 36. *Construction 35 is correct, assuming that the gate KMHE schme* GKMHE *is correct.*

Proof sketch. Correctness of Construction 35 follows trivially by correctness of the garbled circuit construction (see [LP09, Claim 6] for details) as well as correctness of GKMHE.

Theorem 37. *Construction 35 has garbling privacy, assuming that the gate KMHE scheme* GKMHE *is CPA-secure and position-hiding.*

Proof sketch. Proof of garbling privacy follows similarly to the proof of Theorem 7 in [LP09], therefore we will only sketch it here. From $\phi_\lambda(f)$ the simulator $\mathsf{Sim}_{\mathsf{Gb}}$ obtains all information about the circuit f except for the gate operations. $\mathsf{Sim}_{\mathsf{Gb}}$ can then sample labels for each wire as done by Gb. It picks a random set of wire labels as the active set of labels and then programs the garbled circuit to always return $f(x)$ by encrypting only those active labels in all garbled gate ciphertexts. The subset of active labels corresponding to input wires is then returned as the input label set I. Encrypting only the active labels is ensured by the CPA security of Construction 15, while selecting a random set of wire labels as the active set, rather than using the actual active set for input x, is enabled by the position-hiding property of Construction 15.

Theorem 38. *Construction 35 has rerand privacy (Definition 34), assuming that the gate KMHE scheme* GKMHE *satisfies CPA security, KMH rerandomizability, key privacy, and key privacy under leakage.*

Proof sketch. The proof resembles the proof of rerand privacy of the RGS in [AHKP22]. Given a rerandomized garbling, as in case $b = 0$ of the game RER-PRIV, we first use KMH rerandomizability to replace all the rerandomized gate KMHE ciphertexts with fresh encryptions. We then replace the transformed wire labels with fresh labels in topological order, using key privacy under leakage, key privacy, and CPA security. At the end we obtain a simulated garbling, as output by $\mathsf{Sim}_{\mathsf{Gb}}$ in the garbling privacy proof. By definition of garbling privacy, this simulation is indistinguishable from a fresh garbling as in case $b = 1$ of the game RER-PRIV. See Appendix K for the full proof.

References

AGM+. D. F. Aranha, C. P. L. Gouvêa, T. Markmann, R. S. Wahby, and K. Liao. RELIC is an Efficient LIbrary for Cryptography. `https://github.com/relic-toolkit/relic`.

AHKP22. Anasuya Acharya, Carmit Hazay, Vladimir Kolesnikov, and Manoj Prabhakaran. SCALES - MPC with small clients and larger ephemeral servers. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part II*, volume 13748 of *LNCS*, pages 502–531. Springer, Cham, November 2022.

AHKP24. Anasuya Acharya, Carmit Hazay, Vladimir Kolesnikov, and Manoj Prabhakaran. Malicious security for SCALES - outsourced computation with ephemeral servers. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part IX*, volume 14928 of *LNCS*, pages 3–38. Springer, Cham, August 2024.

AJN+16. Prabhanjan Ananth, Aayush Jain, Moni Naor, Amit Sahai, and Eylon Yogev. Universal constructions and robust combiners for indistinguishability obfuscation and witness encryption. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 491–520. Springer, Berlin, Heidelberg, August 2016.

Ber06. Daniel J. Bernstein. Curve25519: New Diffie-Hellman speed records. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006*, volume 3958 of *LNCS*, pages 207–228. Springer, Berlin, Heidelberg, April 2006.

BHHO08. Dan Boneh, Shai Halevi, Michael Hamburg, and Rafail Ostrovsky. Circular-secure encryption from decision Diffie-Hellman. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 108–125. Springer, Berlin, Heidelberg, August 2008.

BMR90. Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). In *22nd ACM STOC*, pages 503–513. ACM Press, May 1990.

CDGK22. Ignacio Cascudo, Bernardo David, Lydia Garms, and Anders Konring. YOLO YOSO: Fast and simple encryption and secret sharing in the YOSO model. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 651–680. Springer, Cham, December 2022.

DFMV17. Ivan Damgård, Sebastian Faust, Pratyay Mukherjee, and Daniele Venturi. Bounded tamper resilience: How to go beyond the algebraic barrier. *Journal of Cryptology*, 30(1):152–190, January 2017.

DRS04. Yevgeniy Dodis, Leonid Reyzin, and Adam Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 523–540. Springer, Berlin, Heidelberg, May 2004.

FKMV12. Sebastian Faust, Markulf Kohlweiss, Giorgia Azzurra Marson, and Daniele Venturi. On the non-malleability of the Fiat-Shamir transform. In Steven D. Galbraith and Mridul Nandi, editors, *INDOCRYPT 2012*, volume 7668 of *LNCS*, pages 60–79. Springer, Berlin, Heidelberg, December 2012.

GHK+21. Craig Gentry, Shai Halevi, Hugo Krawczyk, Bernardo Magri, Jesper Buus Nielsen, Tal Rabin, and Sophia Yakoubov. YOSO: You only speak once - secure MPC with stateless ephemeral roles. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 64–93, Virtual Event, August 2021. Springer, Cham.

GHV10. Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. i-Hop homomorphic encryption and rerandomizable Yao circuits. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 155–172. Springer, Berlin, Heidelberg, August 2010.

HILL99. Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4):1364–1396, 1999.

HLP11. Shai Halevi, Yehuda Lindell, and Benny Pinkas. Secure computation on the web: Computing without simultaneous interaction. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 132–150. Springer, Berlin, Heidelberg, August 2011.

HSS20. Carmit Hazay, Peter Scholl, and Eduardo Soria-Vazquez. Low cost constant round MPC combining BMR and oblivious transfer. *Journal of Cryptology*, 33(4):1732–1786, October 2020.

KRY22. Sebastian Kolby, Divya Ravi, and Sophia Yakoubov. Towards efficient YOSO MPC without setup. Cryptology ePrint Archive, Report 2022/187, 2022.

LP09. Yehuda Lindell and Benny Pinkas. A proof of security of Yao's protocol for two-party computation. *Journal of Cryptology*, 22(2):161–188, April 2009.

LPSY15. Yehuda Lindell, Benny Pinkas, Nigel P. Smart, and Avishay Yanai. Efficient constant round multi-party computation combining BMR and SPDZ. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 319–338. Springer, Berlin, Heidelberg, August 2015.

NR97. Moni Naor and Omer Reingold. Number-theoretic constructions of efficient pseudo-random functions. In *38th FOCS*, pages 458–467. IEEE Computer Society Press, October 1997.

NS09. Moni Naor and Gil Segev. Public-key cryptosystems resilient to key leakage. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 18–35. Springer, Berlin, Heidelberg, August 2009.

Sma23. Nigel P. Smart. Practical and efficient FHE-based MPC. Cryptology ePrint Archive, Report 2023/981, 2023.

Sta01. Pantelimon Stanica. Good lower and upper bounds on binomial coefficients. *Journal of Inequalities in Pure and Applied Mathematics*, 2(3):30, 2001.

Vil12. Jorge Luis Villar. Optimal reductions of some decisional problems to the rank problem. In Xiaoyun Wang and Kazue Sako, editors, *ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 80–97. Springer, Berlin, Heidelberg, December 2012.

Yao86. Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.

ZLXL23. Qingsong Zhao, Ximeng Liu, Huanliang Xu, and Yanbin Li. Practical reusable garbled circuits with parallel updates. *Comput. Stand. Interfaces*, 86:103721, 2023.

Supplementary Material

A Performance Analysis and Comparison

Table 2. Detailed comparison of the RGS from SCALES [AHKP22] and our work in bytes and clock cycles (cc). We set security parameter $\lambda = 128$ and $\kappa \geq \log q + 2\lambda + 3/2 \log \kappa$, where q refers either to the group order of $\mathbb{H}$ or $\mathbb{G}_{25519}$. ct, Encryption (Enc.) and Evaluation (Eval.) refer to sharable KMHE in [AHKP22, Construction 3] and our basic KMHE (Construction 7), respectively.

			[AHKP22]	Our work	Improvement
Curve			Curve25519	BLS12-381	
κ			522	526	
$\lvert\mathsf{ct}\rvert$	# Elems.	in $\mathbb{G}_{25519}$	$2\kappa^2 + 3\kappa + 1$	0	
		in $\mathbb{G}$	0	κ	
		in $\mathbb{H}$	0	κ	
		in $\mathbb{G}_T$	0	$\kappa + 1$	
	Size		16.68 MB	0.36 MB	97.83 %
Size of one garbled gate			133.43 MB	1.35 MB	98.99 %
Size of one garbled Max circuit			125.87 GB	1.27 GB	98.99 %
Size of one garbled Mult circuit			1.74 TB	18.04 GB	98.99 %
Size of one garbled AES circuit			4.4 TB	45.60 GB	98.99 %
Enc.	# Expon.	in $\mathbb{G}_{25519}$	$\kappa^3 + \kappa^2$	0	
		in $\mathbb{G}$	0	1	
		in $\mathbb{H}$	0	κ	
		in $\mathbb{G}_T$	0	κ	
	Clock cycles		1.6×10^{13}(4 min)	6.10×10^{8}(0.01 s)	99.996 %
Garbling one gate (cc)			1.28×10^{14}(33 min)	2.44×10^{9}(0.04 s)	99.998 %
Garbling a Max circuit (cc)			1.24×10^{17}(22 d)	2.36×10^{12}(37 s)	99.998 %
Garbling a Mult circuit (cc)			1.75×10^{18}(317 d)	3.33×10^{13}(9 min)	99.998 %
Garbling an AES circuit (cc)			4.43×10^{18}(2 yr)	8.43×10^{13}(20 min)	99.998 %
Eval.	# Expon.	in $\mathbb{G}_{25519}$	$4\kappa^3 + 7\kappa^2 + 4\kappa + 1$	0	
		in $\mathbb{G}$	0	κ	
		in $\mathbb{H}$	0	2κ	
		in $\mathbb{G}_T$	0	$\kappa + 1$	
	Clock cycles		6.41×10^{13}(17 min)	9.64×10^{8}(0.02 s)	99.998 %
Rerand. a garbled gate (cc)			5.13×10^{14} (2.23 h)	3.35×10^{9}(0.05 s)	99.9993 %
Rerand. a garbl. Max circuit (cc)			4.95×10^{17}(90 d)	3.24×10^{12}(51 s)	99.9993 %
Rerand. a garbl. Mult circuit (cc)			7.01×10^{18}(3.5 yr)	4.58×10^{13}(11 min)	99.9993 %
Rerand. a garbl. AES circuit (cc)			1.77×10^{19}(9 yr)	1.16×10^{14}(30 min)	99.9993 %

Methodology. Table 2 shows complexity estimates comparing the RGS from SCALES and our work in terms of size and performance. We instantiate [AHKP22]

with the high-performance Curve25519 [Ber06], denoting the group with $\mathbb{G}_{25519}$. Our protocols are instantiated with BLS12-381, denoting the source groups of the pairing with $\mathbb{G}$ and $\mathbb{H}$, where $\mathbb{H}$ is the group with a smaller representation of elements, and the target group with $\mathbb{G}_T$. Hence, group elements in $\mathbb{G}_{25519}, \mathbb{G}, \mathbb{H}$ and $\mathbb{G}_T$ can be represented by 256, 768, 384 and 4608 bits, respectively. Using the benchmarking toolkit RELIC [AGM+] on a 2019 AMD Ryzen 9 3950X with 16 times 4.7 GHz, the number of clock cycles (cc) we obtained for exponentiations in $\mathbb{G}_{25519}, \mathbb{G}, \mathbb{H}$ and $\mathbb{G}_T$ are 112 269, 481 170, 196 432 and 957 284, respectively. Additionally, we measured 2 313 901 clock cycles for a BLS12-381 pairing. We then estimate concrete runtimes in seconds (s), minutes (min) and hours (h) by extrapolating the clock cycles, assuming a processor with 16 cores each running at 4 GHz. However, garbling and rerandomization are actually parallelizable to up to κ cores. Hence, runtime could be even lower than shown in Table 2. The Max, Mult and AES circuits refers to Boolean circuits comprising 966 gates computing the max of eight 8 bit numbers, 13 675 gates computing a 64 bit multiplication, and 34 576 gates computing an AES-128 encryption, respectively.

B Correctness of the Basic Scheme

Definition 39 (Correctness). *We say that* $\mathsf{KMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *is* correct *if for all* $\lambda \in \mathbb{N}$, $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $\mathsf{sk} \leftarrow \mathsf{KGen}(\mathsf{pp})$, *and* $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}, m)$ *with* $m \in \mathcal{M}$ *it holds that*

$$\Pr\left[\mathsf{Dec}(\mathsf{pp}, \mathsf{sk}, \mathsf{ct}) = m\right] = 1.$$

Theorem 40. *Construction 7 satisfies* correctness.

The proof is a straightforward calculation.

Proof. Let $\mathsf{sk} = (s_1, \ldots, s_\kappa)$, $m = (m_1, \ldots, m_\kappa)$ and $\mathsf{ct} = (c_1, \ldots, c_\kappa, g_1, \ldots, g_\kappa, y) \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}, m)$. For every $i \in [\kappa]$, decryption $\mathsf{Dec}(\mathsf{pp}, \mathsf{sk}, \mathsf{ct})$ computes

$$\begin{aligned} g_T^{m_i'} &= c_{i,2} \cdot \mathrm{e}\left(\prod_{j \in [\kappa]} g_j^{s_j}, c_{i,1}\right)^{-1} \\ &= g_T^{m_i} \cdot y^{a_i} \cdot \mathrm{e}\left(\prod_{j \in [\kappa]} g_j^{s_j}, h^{a_i}\right)^{-1} \\ &= g_T^{m_i} \cdot \mathrm{e}\left(\prod_{j \in [\kappa]} g_j^{s_j}, h^{a_i}\right) \cdot \mathrm{e}\left(\prod_{j \in [\kappa]} g_j^{s_j}, h^{a_i}\right)^{-1} \\ &= g_T^{m_i} \end{aligned}$$

□

C KMH Correctness of the Basic Scheme

The following lemma will be useful to prove KMH correctness.

Lemma 41. *Let* $\mathsf{ct} = (c_1, \ldots, c_\kappa, g_1, \ldots, g_\kappa, y)$ *be such that*

$$\begin{aligned} y &= \mathrm{e}\left(\prod_{j\in[\kappa]} g_j^{s_j}, h\right), \\ c_i &= (h^{a_i}, g_T^{m_i} \cdot y^{a_i}) \quad \forall i \in [\kappa], \end{aligned}$$

where $a_1, \ldots, a_\kappa \in \mathbb{Z}_q$, $g_1, \ldots, g_\kappa \in \mathbb{G}$, $m_1, \ldots, m_\kappa \in \{0,1\}$, *and* $s_1, \ldots, s_\kappa \in \{0,1\}$. *Then for all* $\lambda \in \mathbb{N}$, $\sigma, \tau \in \mathcal{F}$, $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $r = (a'_1, \ldots, a'_\kappa, d) \in R_{\mathsf{Eval}}$, *and* $\mathsf{ct}' = (c'_1, \ldots, c'_\kappa, g'_1, \ldots, g'_\kappa, y') \leftarrow \mathsf{Eval}(\mathsf{pp}, \sigma, \tau, \mathsf{ct}; r)$, *it holds that*

$$\begin{aligned} y' &= y^d, \\ g'_{\sigma(i)} &= g_i^d \quad \forall i \in [\kappa], \\ c'_{\tau(i)} &= (h^{u_i}, g_T^{m_i} \cdot y'^{u_i}) \quad \forall i \in [\kappa], \end{aligned} \tag{4}$$

where $u_1, \ldots, u_\kappa \in \mathbb{Z}_q$ *and* $d \in \mathbb{Z}_q^*$.

Proof. By definition of our KMHE scheme, ct' is given by

$$\begin{aligned} y' &= y^d, \\ g'_{\sigma(i)} &= g_i^d \quad \forall i \in [\kappa], \\ c'_{\tau(i)} &= (h^{(a_i+a'_i)/d}, g_T^{m_i} \cdot y^{a_i+a'_i}) \quad \forall i \in [\kappa]. \end{aligned}$$

Now rewrite

$$y^{a_i+a'_i} = y^{b(a_i+a'_i)/d} = y'^{(a_i+a'_i)/d},$$

and set $u_i = (a_i + a'_i)/d$ for all $i \in [\kappa]$, fulfilling Equation (4). □

Theorem 42. *Construction 7 satisfies* KMH correctness.

Proof. Let $\mathsf{ct}_1 = (\hat{c}_1, \ldots, \hat{c}_\kappa, \hat{g}_1, \ldots, \hat{g}_\kappa, \hat{y}) \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}, m; r_1)$. Consider now $\mathsf{ct}_2 = (c'_1, \ldots, c'_\kappa, g'_1, \ldots, g'_\kappa, y') \leftarrow \mathsf{Eval}(\mathsf{pp}, \sigma, \tau, \mathsf{ct}_1; r_2)$. By definition of the KMHE scheme, ct_1 satisfies the conditions of Lemma 41.

Therefore, by Lemma 41, ct_2 fulfills

$$\begin{aligned} y' &= \hat{y}^b = \mathrm{e}\left(\prod_{j\in[\kappa]} \hat{g}_j^{ds_j}, h\right), \\ g'_{\sigma(i)} &= \hat{g}_i^d \quad \forall i \in [\kappa], \\ c'_{\tau(i)} &= (h^{u_i}, g_T^{m_i} \cdot y'^{u_i}) \quad \forall i \in [\kappa], \end{aligned}$$

where $u_1, \ldots, u_\kappa \in \mathbb{Z}_q$ and $d \in \mathbb{Z}_q^*$.

We now prove that there exists $r' = (g_1, \ldots, g_\kappa, a_1, \ldots, a_\kappa) \in R_{\mathsf{Enc}}$ such that $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \sigma(\mathsf{sk}), \tau(m); r')$ satisfies $\mathsf{ct} = \mathsf{ct}_2$. Specifically, we choose $r' = (g'_1, \ldots, g'_\kappa, u_1, \ldots, u_\kappa)$. Then ct is given by

$$\begin{aligned} y &= \mathrm{e}\left(\prod_{j\in[\kappa]} g'^{s_j}_{\sigma(j)}, h\right) = \mathrm{e}\left(\prod_{j\in[\kappa]} \hat{g}_j^{bs_j}, h\right) = \hat{y}^b = y', \\ g'_{\sigma(j)} &= g_i \quad \forall i \in [\kappa], \\ c_{\tau(i)} &= (h^{a_i} = h^{u_i}, g_T^{m_i} \cdot y^{a_i} = g_T^{m_i} \cdot y'^{u_i}) \quad \forall i \in [\kappa], \end{aligned}$$

which is exactly ct_2. □

D Proof of Lemma 12

Proof. We have $|HW_{\kappa,\kappa/2}| = \binom{\kappa}{\kappa/2}$. Stanica [Sta01] gives the following bound for the binomial coefficient:

$$\binom{\kappa}{\kappa/2} \geq 2^{\kappa-1} \cdot \frac{1}{\sqrt{\kappa/2}}$$

Combining this bound and the fact that sk is sampled uniformly from $HW_{\kappa,\kappa/2}$ we get that

$$H_\infty(\mathsf{sk}) = \log \binom{\kappa}{\kappa/2} \geq \kappa - \frac{1}{2}\log\kappa - 1/2.$$

□

E Proof of Lemma 13

Proof. Define $\mathbf{G} \in \mathbb{G}^{\eta\times\nu}$ to be the matrix whose rows are given by $\mathbf{g}_1, \ldots, \mathbf{g}_\eta$. Since we are assuming hardness of the DDH problem in $\mathbb{G}$, by Lemma 3 it holds that $\mathbf{G} \overset{\mathrm{c}}{\approx} \hat{\mathbf{G}}$, where $\hat{\mathbf{G}}$ is a uniformly random rank-1 matrix of the same dimension. Let now $\hat{\mathbf{g}}_1, \ldots, \hat{\mathbf{g}}_\eta \in \mathbb{G}^\nu$ be the rows of $\hat{\mathbf{G}}$. Since $\hat{\mathbf{G}}$ is a uniformly random rank-1 matrix, with $\alpha_1 = 1$ there exist uniformly distributed $\alpha_2, \ldots, \alpha_\eta$ such that

$$\hat{\mathbf{g}}_i = \hat{\mathbf{g}}_1^{\alpha_i} \quad \forall i \in [\eta],$$

which yields that

$$\left\{\mathbf{g}_1, \ldots, \mathbf{g}_\eta, \mathbf{g}_1^{\mathsf{sk}}, \ldots, \mathbf{g}_\eta^{\mathsf{sk}}\right\} \overset{\mathrm{c}}{\approx} \left\{\hat{\mathbf{g}}_1, \ldots, \hat{\mathbf{g}}_\eta, \left(\hat{\mathbf{g}}_1^{\mathsf{sk}}\right)^{\alpha_1}, \ldots, \left(\hat{\mathbf{g}}_1^{\mathsf{sk}}\right)^{\alpha_\eta}\right\}, \tag{5}$$

Parsing $\hat{\mathbf{g}}_1 = (\hat{g}_{1,1}, \ldots, \hat{g}_{1,\nu})$, since $\hat{\mathbf{g}}_1$ is uniformly distributed, there exist uniformly distributed $x_1, \ldots x_\nu \in \mathbb{Z}_q$ such that $\hat{g}_{1,j} = g^{x_j}$ for all $j \in [\nu]$ and some generator $g \in \mathbb{G}$. Then we can write

$$\hat{\mathbf{g}}_1^{\mathsf{sk}} = \prod_{j\in[\nu]} \hat{g}_{1,j}^{s_j} = \prod_{j\in[\nu]} g^{x_j s_j} = g^{\sum_{j\in[\nu]} x_j s_j}.$$

Since $H_\infty(\mathsf{sk}) \geq \log q + 2\lambda$ and $x_1, \ldots, x_\nu \leftarrow\!\!\$\ \mathbb{Z}_q$, we can apply the left-over hash lemma (Lemma 11), resulting in

$$\Delta\left(\hat{\mathbf{g}}_1^{\mathsf{sk}}, \hat{g}\right) = \Delta\left(\prod_{j\in[\nu]} g^{x_j s_j}, \hat{g}\right) \leq 2^{-\lambda} = \mathsf{negl}(\lambda), \tag{6}$$

where $\hat{g} \leftarrow\!\!\$\ \mathbb{G}$. Since the $\alpha_2, \ldots, \alpha_\eta$ are uniformly distributed, it holds that

$$\hat{g}^{\alpha_i} \equiv g'_i \quad \forall i \in [\eta], \tag{7}$$

where $g'_1, \ldots, g'_\eta \leftarrow\!\!\$\ \mathbb{G}$. Combining Equations (5) to (7) plus another invocation of DDH assumption to replace $\hat{\mathbf{G}}$ with $\mathbf{G}$ again we then obtain the desired statement. □

F Gaps in the Proofs of [AHKP22]

We have discovered some small gaps in the proofs in [AHKP22]. These gaps motivate the changes we have made to the RGS and KMHE definitions. Note that we do believe that the KMHE and RGS constructions in [AHKP22] also fulfill our new definitions. Therefore, this is mainly an issue of formal correctness.

The gaps occur in their construction of incremental decomposable randomized encodings (iDRE) [AHKP22, Figure 2]. An iDRE is a multi-party protocol for ℓ parties designed to evaluate a function f on some input $x \in \{0,1\}^m$.The construction in [AHKP22] relies on a (weak) KMHE scheme and an RGS scheme. Each party starts off by sampling KMHE keys $\{e^0_{i,j}, e^1_{i,j}\}_{i\in[m]}$ for each input bit x_i in the case of the first party, or KMHE key transformation functions $\{e^0_{i,j}, e^1_{i,j}\}_{i\in[m]}$ for the remaining parties. This is done offline and ahead of the protocol. Once the protocol starts, the first party starts by garbling f, leading to a garbling $\mathcal{C}$ and input labels $L = (L^0_1, L^1_1, \ldots, L^0_m, L^1_m)$ for all possible inputs $x \in \{0,1\}^m$. Each input label is then separately encrypted using the KMHE scheme. The garbling $\mathcal{C}$ and the ciphertexts are then passed on to the next party. Each following party now rerandomizes the garbling via the Rerand algorithm of the RGS scheme. Rerand also outputs the permutations applied to the input labels via π_{En}. These are also applied to the encrypted labels via the KMHE message homomorphism. Lastly, the keys in each label ciphertext are updated by applying the functions given by $\{e^0_{i,j}, e^1_{i,j}\}_{i\in[m]}$ via the KMHE key homomorphism. This last procedure ensures that the label ciphertexts can only be decrypted if all parties collaborate to recompute the keys. Once all parties have finished this process the garbling can be evaluated on some given input x. To do this, the parties together reveal the keys for the input labels $L^{x_1}_1, \ldots, L^{x_m}_m$. These can then be used to evaluate the garbling and obtain the output $f(x)$.

The relevant security notion is called privacy. It ensures that a semi-honest adversary that corrupts all but one party j^*, learns nothing but the function f, the output $f(x)$, and the corrupted parties' inputs. Importantly, it should not be able to learn the honest parties input bit even given the input labels for the

whole input x. We argue that the security properties of the weak KMHE scheme and the RGS scheme do not suffice to prove privacy of the iDRE construction.

The proof is given in Claim 3 in [AHKP22]. It works roughly as follows: First CPA security of the KMHE scheme is used to argue that the KMHE ciphertexts only reveal the labels for the input x, the inactive labels are hidden. Then, rerand privacy of the RGS is used to argue that the garbling output by the $(j^* + 1)^{\text{th}}$ party looks like a fresh garbling. By garbling privacy of the RGS, this fresh garbling then ensures that given the input labels for x, only the output $f(x)$ is leaked, as desired by privacy of the iDRE scheme.

The first gap in this proof occurs in the use of rerand privacy. The rerand privacy in [AHKP22] is the same as our definition but with $t = 1$ fixed. This means it guarantees that a honest rerandomization of a semi-honest garbling looks like a fresh honest garbling. However, the $(j^* + 1)^{\text{th}}$ party does not obtain a semi-honest garbling, it obtains a garbling that has been semi-honestly rerandomized several times before by the previous parties. This case is addressed for our definition with arbitrary t. It is also not clear whether the $t = 1$ definition implies the definition with arbitrary t. Using a standard hybrid argument seems to not work since the honest rerandomization is only guaranteed to rerandomize a fresh garbling, not a semi-honestly rerandomized one. However, we were not able to formally separate the two definitions, e.g. by showing a construction that fulfills the $t = 1$ variant but not the arbitrary t variant. This would require an RGS construction where somehow a semi-honest rerandomization breaks the following honest rerandomization. This is difficult as the only difference between a semi-honest and an honest rerandomization is that the adversary knows the randomness for the former. Therefore it is hard to design the rerandomization such that in one case it works but in the other it breaks. We resolve this issue by changing the definition of rerand privacy (Definition 34) to consider an arbitrary number t of intermediate semi-honest rerandomizations.

The second gap is with the KMHE scheme itself. Acharya et al. [AHKP22] only require this to be a *weak* KMHE scheme, meaning it does not fulfill their notion of KMH privacy. KMH privacy, however, is the only notion in [AHKP22] that guarantees that an output of Eval looks like a fresh encryption, i.e. that Eval rerandomizes the ciphertext. KMH correctness does not suffice since it does not say anything about the distribution of the output of Eval, just that it is a valid ciphertext. Without the ability to replace an output of Eval with a fresh encryption, it is not possible to apply CPA security. To address this we have split KMH privacy up into KMH rerandomizability (Definition 24) and key privacy under leakage (Definition 26) where now KMH rerandomizability is part of the (weak) KMHE definition and key privacy under leakage is required for a strong KMHE scheme.

We now discuss where exactly these issues occur in the proof's reductions and how they can be fixed using our definitions. Mainly of interest is the proof of indistinguishability of hybrids H_2 and H_3 in the proof of their Claim 3, which they prove via a reduction to CPA security of the KMHE scheme. The hybrid H_3 there is defined as the adversary's view in a real-world execution of the

iDRE protocol. Acharya et al. [AHKP22] then assume a distinguisher D that can distinguish hybrid H_2 and H_3 and then construct a distinguisher D' that uses D to break CPA security of the KMHE scheme. However, we argue that D' does not correctly simulate the hybrid H_3.

The first issue is that D' creates the value F_{j^*} as a fresh semi-honest garbling, even though in a real-world execution of the iDRE protocol this would be a semi-honestly rerandomized garbling since j^* is not necessarily the first party in the chain of parties. This then allows Acharya et al. to later use rerand privacy with $t = 1$. If we fix the definition of F_{j^*} to be a rerandomization as in the real-world, then it is possible to use our new rerand privacy definition with arbitrary t to argue indistinguishability of the hybrids H_2 and H_1.

The second issue with the distinguisher D' is that the encrypted labels are created as fresh encryptions instead of being outputs of Eval as they would be in the real world. This can be fixed by defining H_3 to use fresh encryptions and adding another hybrid H_4 which then actually corresponds to the real-world execution. Indistinguishability of H_3 and H_4 then follows from KMH rerandomizability of the KMHE scheme while H_3 and H_2 uses CPA security as before.

Note that Acharya et al. have since fixed these gaps by switching from the standard BHHO scheme to the expanded version, which enables perfect rerandomization and by proxy also perfect rerand privacy.

G Position-Hiding

In this section we formally define what we mean by hiding the key and message positions and sketch the proof of this property for Construction 15.

For the game we will assume that the adversary, like an honest garbled gate evaluator, has access to a set of secret keys, one key for each input wire. The first two secret keys correspond to the left input wire while the third and fourth secret keys correspond to the right input wire. A ciphertext should then hide in which order the four encrypted messages were supplied to the encryption oracle, even if the adversary knows all of the encrypted messages. We formally model this by requiring an encryption of m^1, m^2, m^3, m^4 and an encryption of $\hat{\pi}(m^1, m^2, m^3, m^4)$ to be indistinguishable, where $\hat{\pi} \leftarrow\!\!\$\ S_4$ is a random permutation.

The key position hiding is slightly more complex. We cannot and do not want to hide to which input wire the keys belong[2], only whether each key is the first or second key of that input wire. We model this by individually permuting the two pairs of input keys using permutations $\hat{\pi}^0, \hat{\pi}^1 \leftarrow\!\!\$\ S_2$. This ensures that the keys stay assigned to the same wires for both case $b = 0$ and case $b = 1$ of the security game.

We proceed by formalizing these requirements. Similar to CPA security, the resulting game is parameterized by two bits β_0 and β_1 which determine the keys the adversary is given as part of the challenge.

[2] The adversary can always use the decryption algorithm to test which of his keys is a left or right input wire key.

Definition 43 (Position-hiding). $\mathsf{GKMHE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ *is position-hiding, if for all PPT adversaries* $\mathcal{A}$ *there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all* $\beta_0, \beta_1 \in \{0,1\}$ *it holds that*

$$\left|2 \cdot \Pr\left[\mathsf{GKMHE\text{-}PH}^{\beta_0,\beta_1}_{\mathcal{A},\mathsf{GKMHE}}(1^\lambda) = 1\right] - 1\right| \leq \mathsf{negl}(\lambda),$$

where the GKMHE-PH *experiment is defined in Figure 6.*

$\mathsf{GKMHE\text{-}PH}^{\beta_0,\beta_1}_{\mathcal{A},\mathsf{GKMHE}}(1^\lambda)$

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda); b \leftarrow\!\!\$\ \{0,1\}$

$(m^1, m^2, m^3, m^4) \leftarrow \mathcal{A}(1^\lambda, \mathsf{pp})$

$\mathsf{sk}^{0,0}, \mathsf{sk}^{0,1}, \mathsf{sk}^{1,0}, \mathsf{sk}^{1,1} \leftarrow \mathsf{KGen}(\mathsf{pp})$

if $b = 0$ **then**

  $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}^{0,0}, \mathsf{sk}^{0,1}, \mathsf{sk}^{1,0}, \mathsf{sk}^{1,1}, m^1, m^2, m^3, m^4)$

else

  $\hat{\pi}^0, \hat{\pi}^1 \leftarrow\!\!\$\ S_2, \hat{\pi} \leftarrow\!\!\$\ S_4$

  $\left(\widehat{\mathsf{sk}}^{0,0}, \widehat{\mathsf{sk}}^{0,1}, \widehat{\mathsf{sk}}^{1,0}, \widehat{\mathsf{sk}}^{1,1}\right) \leftarrow \left(\hat{\pi}^0(\mathsf{sk}^{0,0}, \mathsf{sk}^{0,1}), \hat{\pi}^1(\mathsf{sk}^{1,0}, \mathsf{sk}^{1,1})\right)$

  $\left(\widehat{m}^1, \widehat{m}^2, \widehat{m}^3, \widehat{m}^4\right) \leftarrow \hat{\pi}\left(m^1, m^2, m^3, m^4\right)$

  $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \widehat{\mathsf{sk}}^{0,0}, \widehat{\mathsf{sk}}^{0,1}, \widehat{\mathsf{sk}}^{1,0}, \widehat{\mathsf{sk}}^{1,1}, \widehat{m}^1, \widehat{m}^2, \widehat{m}^3, \widehat{m}^4)$

endif

$b' \leftarrow \mathcal{A}^{O(\cdot,\mathsf{sk}^{0,1-\beta_0},\cdot,\mathsf{sk}^{1,1-\beta_1},\cdot,\cdot,\cdot,\cdot),O(\cdot,\mathsf{sk}^{0,1-\beta_0},\cdot,\cdot,\cdot,\cdot,\cdot,\cdot),O(\cdot,\cdot,\cdot,\mathsf{sk}^{1,1-\beta_1},\cdot,\cdot,\cdot,\cdot)}(\mathsf{ct}, \mathsf{sk}^{0,\beta_0}, \mathsf{sk}^{1,\beta_1})$

return $(b' = b)$

Fig. 6. Definition of Game GKMHE-PH. The oracle works exactly as in the definition of IND-CPA security: $O(\mathsf{sk}^{0,\beta_0}, \mathsf{sk}^{0,1-\beta_0}, \mathsf{sk}^{1,\beta_1}, \mathsf{sk}^{1,1-\beta_1}, m^1, m^2, m^3, m^4)$, given secret keys $\mathsf{sk}^{0,\beta_0}, \mathsf{sk}^{0,1-\beta_0}, \mathsf{sk}^{1,\beta_1}, \mathsf{sk}^{1,1-\beta_1}$, and messages m^1, m^2, m^3, m^4, returns the output of $\mathsf{Enc}(\mathsf{pp}, \mathsf{sk}^{0,0}, \mathsf{sk}^{0,1}, \mathsf{sk}^{1,0}, \mathsf{sk}^{1,1}, m^1, m^2, m^3, m^4)$.

Theorem 44. *Construction 15 is position-hiding (Definition 43), assuming hardness of the SXDH problem (Definition 4) in the bilinear groups defined by* pp.

Proof Sketch. The position-hiding of the messages is trivial, since each message m^i is contained in ciphertext part ct^i, and Enc in Construction 15 permutes the ct^i randomly.

The position-hiding of the keys is a bit more complex. We examine what effect the application of $\hat{\pi}^0$ and $\hat{\pi}^1$ has to the ciphertext. A permutation in S_2 can either be the identity function or swap its to arguments. We only need to consider the case that a swap is done as otherwise the key positions stay the

same in both cases $b = 0$ and $b = 1$. If $\hat{\pi}^1$ is a swap, then this can be modelled as first encrypting using the unpermuted $\mathsf{sk}^{0,0}$ and $\mathsf{sk}^{0,1}$, and then swapping y_3 and y_4 during execution of Enc. This has the same effect on the resulting ciphertext as permuting those input keys. If $\hat{\pi}^0$ is a swap, then the same holds for y_1 and y_2. Since the y_i already get permuted by the $\pi \in S_4$ at the end of Enc, the order of the y_i is random for both $b = 0$ and $b = 1$, however. Therefore cases $b = 0$ and $b = 1$ are indistinguishable.

H Proof of Theorem 19

Proof. Parse $(s_1^{\beta_0\beta_1}, \ldots, s_\kappa^{\beta_0\beta_1}) = \mathsf{sk}^{\beta_0\beta_1}$ for all $\beta_0, \beta_1 \in \{0,1\}$. Let r_1 be given as in Definition 18 and set

$$\begin{aligned}&(g_{0,1}, \ldots, g_{0,\kappa}, g_{1,1}, \ldots, g_{1,\kappa},\\ &a_1^{(1)}, \ldots, a_\kappa^{(1)}, a_1^{(2)}, \ldots, a_\kappa^{(2)},\\ &a_1^{(3)}, \ldots, a_\kappa^{(3)}, a_1^{(4)}, \ldots, a_\kappa^{(4)}, \pi_1) = r_1 \in \mathbb{G}^{2\kappa} \times \mathbb{Z}_q^{4\kappa} \times S_4,\end{aligned}$$

and

$$\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}^{00}, \mathsf{sk}^{01}, \mathsf{sk}^{10}, \mathsf{sk}^{11}, m^1, m^2, m^3, m^4; r_1),$$

where

$$\begin{aligned}\left(\mathsf{ct}^1, \mathsf{ct}^2, \mathsf{ct}^3, \mathsf{ct}^4, g_{0,1}, \ldots, g_{0,\kappa}, g_{1,1}, \ldots, g_{1,\kappa}\right) &= \mathsf{ct},\\ (c_{i,1}, \ldots, c_{i,\kappa}, y_i) &= \mathsf{ct}^i \quad \forall i \in [4].\end{aligned}$$

By definition of Enc, ct is computed as

$$\begin{pmatrix} y_{\pi_1(1)} \\ y_{\pi_1(2)} \\ y_{\pi_1(3)} \\ y_{\pi_1(4)} \end{pmatrix} = \begin{pmatrix} \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{00}} \cdot g_{1,j}^{s_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{00}} \cdot g_{1,j}^{s_j^{11}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{10}} \cdot g_{1,j}^{s_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{10}} \cdot g_{1,j}^{s_j^{11}}\right), h\right) \end{pmatrix}, \tag{8}$$

$$c_{\pi_1(i),j} = \left(h^{a_j^{(i)}}, g_T^{m_j^i} \cdot y_{\pi_1(i)}^{a_j^{(i)}}\right) \quad \forall i \in [4], j \in [\kappa].$$

Consider now

$$\left(\widehat{\mathsf{ct}}^1, \widehat{\mathsf{ct}}^2, \widehat{\mathsf{ct}}^3, \widehat{\mathsf{ct}}^4, \hat{g}_{0,1}, \ldots, \hat{g}_{0,\kappa}, \hat{g}_{1,1}, \ldots, \hat{g}_{1,\kappa}\right) = \widehat{\mathsf{ct}} \leftarrow \mathsf{Eval}(\mathsf{pp}, \sigma_0, \sigma_1, \tau, \mathsf{ct}; r_2)$$

where

$$\begin{aligned}&(\hat{a}_1^{(1)}, \ldots, \hat{a}_\kappa^{(1)}, \hat{a}_1^{(2)}, \ldots, \hat{a}_\kappa^{(2)},\\ &\hat{a}_1^{(3)}, \ldots, \hat{a}_\kappa^{(3)}, \hat{a}_1^{(4)}, \ldots, \hat{a}_\kappa^{(4)}, d, \pi_2) = r_2 \in R_{\mathsf{Eval}} = \mathbb{Z}_q^{4\kappa} \times \mathbb{Z}_q^* \times S_4,\\ &(\hat{c}_{i,1}, \ldots, \hat{c}_{i,\kappa}, \hat{y}_i) = \widehat{\mathsf{ct}}^i \quad \forall i \in [4].\end{aligned}$$

By definition of Eval in Construction 15, $\widehat{\mathsf{ct}}$ is then computed as

$$\begin{pmatrix} \hat{y}_{\pi_2(\pi_1(1))} \\ \hat{y}_{\pi_2(\pi_1(2))} \\ \hat{y}_{\pi_2(\pi_1(3))} \\ \hat{y}_{\pi_2(\pi_1(4))} \end{pmatrix} = \begin{pmatrix} y^d_{\pi_1(1)} \\ y^d_{\pi_1(2)} \\ y^d_{\pi_1(3)} \\ y^d_{\pi_1(4)} \end{pmatrix} = \begin{pmatrix} \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{00}} \cdot g_{1,j}^{s_j^{10}} \right)^d, h \right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{00}} \cdot g_{1,j}^{s_j^{11}} \right)^d, h \right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{01}} \cdot g_{1,j}^{s_j^{10}} \right)^d, h \right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{01}} \cdot g_{1,j}^{s_j^{11}} \right)^d, h \right) \end{pmatrix}, \tag{9}$$

$$\begin{aligned} \hat{g}_{0,\sigma_0(j)} &= g_{0,j}^d \quad \forall j \in [\kappa], \\ \hat{g}_{1,\sigma_1(j)} &= g_{1,j}^d \quad \forall j \in [\kappa], \\ \hat{c}_{\pi_2(\pi_1(i)),\tau(j)} &= \left(h^{\left(a_j^{(i)} + \hat{a}_{\tau(j)}^{(\pi_1(i))}\right)/d}, g_T^{m_j^i} \cdot y_{\pi_1(i)}^{a_j^{(i)} + \hat{a}_{\tau(j)}^{(\pi_1(i))}} \right) \quad \forall j \in [\kappa], i \in [4]. \end{aligned}$$

We now prove that there exists $r' \in R_{\mathsf{Enc}}$ such that

$$\mathsf{ct}' \leftarrow \mathsf{Enc}(\mathsf{pp}, \sigma_0(\mathsf{sk}^{00}), \sigma_0(\mathsf{sk}^{01}), \sigma_1(\mathsf{sk}^{10}), \sigma_1(\mathsf{sk}^{11}), \tau(m^1), \tau(m^2), \tau(m^3), \tau(m^4); r')$$

satisfies $\mathsf{ct}' = \widehat{\mathsf{ct}}$. Specifically, we choose

$$\begin{aligned} r' = (&\hat{g}_{0,1}, \ldots, \hat{g}_{0,\kappa}, \hat{g}_{1,1}, \ldots, \hat{g}_{1,\kappa}, \\ &u_1^{(1)}, \ldots, u_\kappa^{(1)}, u_1^{(2)}, \ldots, u_\kappa^{(2)}, \\ &u_1^{(3)}, \ldots, u_\kappa^{(3)}, u_1^{(4)}, \ldots, u_\kappa^{(4)}, \pi_2 \circ \pi_1) \in \mathbb{G}^{2\kappa} \times \mathbb{Z}_q^{4\kappa} \times S_4, \end{aligned}$$

where we define

$$u_{\tau(j)}^{(i)} = \left(a_j^{(i)} + \hat{a}_{\tau(j)}^{(\pi_1(i))} \right)/d \quad \forall j \in [\kappa], i \in [4].$$

Then, parsing

$$\begin{aligned} \left(\mathsf{ct}'^1, \mathsf{ct}'^2, \mathsf{ct}'^3, \mathsf{ct}'^4, g'_{0,1}, \ldots, g'_{0,\kappa}, g'_{1,1}, \ldots, g'_{1,\kappa}\right) &= \mathsf{ct}, \\ (c'_{i,1}, \ldots, c'_{i,\kappa}, y'_{i_1}) &= \mathsf{ct}'^i \quad \forall i \in [4], \end{aligned}$$

it holds by the definition of Enc in Construction 15 that

$$\begin{aligned}
\begin{pmatrix} y'_{\pi_2(\pi_1(1))} \\ y'_{\pi_2(\pi_1(2))} \\ y'_{\pi_2(\pi_1(3))} \\ y'_{\pi_2(\pi_1(4))} \end{pmatrix} &= \begin{pmatrix} \mathrm{e}\left(\prod_{j\in[\kappa]} \left(\hat{g}_{0,j}^{\sigma_0(\mathsf{sk}^{00})_j} \cdot \hat{g}_{1,j}^{\sigma_1(\mathsf{sk}^{10})_j}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(\hat{g}_{0,j}^{\sigma_0(\mathsf{sk}^{00})_j} \cdot \hat{g}_{1,j}^{\sigma_1(\mathsf{sk}^{10})_j}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(\hat{g}_{0,j}^{\sigma_0(\mathsf{sk}^{01})_j} \cdot \hat{g}_{1,j}^{\sigma_1(\mathsf{sk}^{11})_j}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(\hat{g}_{0,j}^{\sigma_0(\mathsf{sk}^{01})_j} \cdot \hat{g}_{1,j}^{\sigma_1(\mathsf{sk}^{11})_j}\right), h\right) \end{pmatrix} \\
&= \begin{pmatrix} \mathrm{e}\left(\prod_{j\in[\kappa]} \left(\hat{g}_{0,\sigma_0(j)}^{s_j^{00}} \cdot \hat{g}_{1,\sigma_1(j)}^{s_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(\hat{g}_{0,\sigma_0(j)}^{s_j^{00}} \cdot \hat{g}_{1,\sigma_1(j)}^{s_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(\hat{g}_{0,\sigma_0(j)}^{s_j^{01}} \cdot \hat{g}_{1,\sigma_1(j)}^{s_j^{11}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(\hat{g}_{0,\sigma_0(j)}^{s_j^{01}} \cdot \hat{g}_{1,\sigma_1(j)}^{s_j^{11}}\right), h\right) \end{pmatrix} \\
&= \begin{pmatrix} \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{00}} \cdot g_{1,j}^{s_j^{10}}\right)^d, h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{00}} \cdot g_{1,j}^{s_j^{10}}\right)^d, h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{01}} \cdot g_{1,j}^{s_j^{11}}\right)^d, h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{01}} \cdot g_{1,j}^{s_j^{11}}\right)^d, h\right) \end{pmatrix} = \begin{pmatrix} \hat{y}_{\pi_2(\pi_1(1))} \\ \hat{y}_{\pi_2(\pi_1(2))} \\ \hat{y}_{\pi_2(\pi_1(3))} \\ \hat{y}_{\pi_2(\pi_1(4))} \end{pmatrix},
\end{aligned}$$

and

$$\begin{aligned}
c'_{\pi_2(\pi_1(i)),\tau(j)} &= \left(h^{u'^{(i)}_{\tau(j)}}, g_T^{m_j^i} \cdot y'^{u'^{(i)}_{\tau(j)}}_{\pi_2(\pi_1(i))}\right) \\
&= \left(h^{\left(a_j^{(i)} + \hat{a}^{(\pi_1(i))}_{\tau(j)}\right)/d}, g_T^{m_j^i} \cdot \hat{y}^{\left(a_j^{(i)} + \hat{a}^{(\pi_1(i))}_{\tau(j)}\right)/d}_{\pi_2(\pi_1(i))}\right) \\
&= \left(h^{\left(a_j^{(i)} + \hat{a}^{(\pi_1(i))}_{\tau(j)}\right)/d}, g_T^{m_j^i} \cdot y^{d\left(a_j^{(i)} + \hat{a}^{(\pi_1(i))}_{\tau(j)}\right)/d}_{\pi_1(i)}\right) \\
&= \left(h^{\left(a_j^{(i)} + \hat{a}^{(\pi_1(i))}_{\tau(j)}\right)/d}, g_T^{m_j^i} \cdot y^{a_j^{(i)} + \hat{a}^{(\pi_1(i))}_{\tau(j)}}_{\pi_1(i)}\right) \\
&= \hat{c}_{\pi_2(\pi_1(i)),\tau(j)} \quad \forall j \in [\kappa], i \in [4].
\end{aligned}$$

Since we also chose $g'_{\beta,j} = \hat{g}_{\beta,j}$ for all $\beta \in \{0,1\}, j \in [\kappa]$, we have proven that $\mathsf{ct}' = \widehat{\mathsf{ct}}$, completing the proof. □

I Proof of Theorem 21

Proof of Theorem 21. The game GKMHE-IND-CPA is parameterized by two bits $\beta_0, \beta_1 \in \{0,1\}$ which determine the keys the adversary obtains. To simplify our notation, without loss of generality we will assume that $\beta_0 = \beta_1 = 0$. The other three cases follow analogously.

We define a sequence of games, starting from $\mathsf{GKMHE\text{-}IND\text{-}CPA}^{0,0}_{\mathcal{A},\mathsf{GKMHE}}$ and ending with a game where the challenge messages are perfectly hidden. Since the adversary $\mathcal{A}$ cannot win in the later game, computational indistinguishability of the sequence of games then ensures that $\mathcal{A}$ also cannot win in game $\mathsf{GKMHE\text{-}IND\text{-}CPA}^{0,0}_{\mathcal{A},\mathsf{GKMHE}}$, except with negligible probability.

Game 1. This is the $\mathsf{GKMHE\text{-}IND\text{-}CPA}^{0,0}_{\mathcal{A},\mathsf{GKMHE}}$ game. Let

$$\begin{aligned}
\mathsf{ct}_{1,i_1} &= \left(\mathsf{ct}^{00}_{1,i_1}, \mathsf{ct}^{01}_{1,i_1}, \mathsf{ct}^{10}_{1,i_1}, \mathsf{ct}^{11}_{1,i_1}, g_{1,i_1,0,1}, \ldots, g_{1,i_1,0,\kappa}, g_{1,i_1,1,1}, \ldots, g_{1,i_1,1,\kappa}\right)\\
\mathsf{ct}_{2,i_2} &= \left(\mathsf{ct}^{00}_{2,i_2}, \mathsf{ct}^{01}_{2,i_2}, \mathsf{ct}^{10}_{2,i_2}, \mathsf{ct}^{11}_{2,i_2}, g_{2,i_2,0,1}, \ldots, g_{2,i_2,0,\kappa}, g_{2,i_2,1,1}, \ldots, g_{2,i_2,1,\kappa}\right)\\
\mathsf{ct}_{3,i_3} &= \left(\mathsf{ct}^{00}_{3,i_3}, \mathsf{ct}^{01}_{3,i_3}, \mathsf{ct}^{10}_{3,i_3}, \mathsf{ct}^{11}_{3,i_3}, g_{3,i_3,0,1}, \ldots, g_{3,i_3,0,\kappa}, g_{3,i_3,1,1}, \ldots, g_{3,i_3,1,\kappa}\right)
\end{aligned}$$

for $i_1 \in [Q_1], i_2 \in [Q_2], i_3 \in [Q_3]$, be the ciphertexts given to the adversary via the oracles

$$\begin{aligned}
&O(\cdot, \mathsf{sk}^{0,1-\beta_0}, \cdot, \mathsf{sk}^{1,1-\beta_1}, \cdot, \cdot, \cdot, \cdot),\\
&O(\cdot, \mathsf{sk}^{0,1-\beta_0}, \cdot, \cdot, \cdot, \cdot, \cdot, \cdot), \text{ and}\\
&O(\cdot, \cdot, \cdot, \mathsf{sk}^{1,1-\beta_1}, \cdot, \cdot, \cdot, \cdot),
\end{aligned}$$

respectively. Let also

$$\mathsf{ct} = (\mathsf{ct}^{00}, \mathsf{ct}^{01}, \mathsf{ct}^{10}, \mathsf{ct}^{11}, g_{0,1}, \ldots, g_{0,\kappa}, g_{1,1}, \ldots, g_{1,\kappa})$$

be the challenge ciphertext. Furthermore, parse

$$\begin{aligned}
\mathsf{ct}^{\delta_0\delta_1}_{\gamma,i_\gamma} &= (c_{\gamma,i_\gamma,\delta_0\delta_1,1}, \ldots, c_{\gamma,i_\gamma,\delta_0\delta_1,\kappa}, y_{\gamma,i_\gamma,\delta_0\delta_1}),\\
\mathsf{ct}^{\delta_0\delta_1} &= (c_{\delta_0\delta_1,1}, \ldots, c_{\delta_0\delta_1,\kappa}, y_{\delta_0\delta_1})
\end{aligned}$$

for all $\gamma \in [3]$, $i_\gamma \in [Q_\gamma]$, $\delta_0, \delta_1 \in \{0,1\}$.

Game 2. We alter how each query response ciphertext, including the challenge ciphertext, is created by Enc. Specifically, we replace the terms involving the keys sk^{01} and sk^{11} unknown to the adversary with uniform exponents. More precisely, for each $\mathsf{ct}_{1,i}$, $i \in [Q_1]$, Enc, given secret keys $\mathsf{sk}^{00}_{1,i}$, sk^{01}, $\mathsf{sk}^{10}_{1,i}$, and

sk^{11}, samples additional $x_{1,i}^{(01)}, x_{1,i}^{(11)} \leftarrow\!\!\$\ \mathbb{Z}_q^*$ and then computes

$$y_{1,i,01} = \mathrm{e}\left(\prod_{j\in[\kappa]}\left(g_{1,i,0,j}^{\mathsf{sk}_{1,i,j}^{00}}\right)\cdot g^{x_{1,i}^{(11)}}, h\right),$$
$$y_{1,i,10} = \mathrm{e}\left(g^{x_{1,i}^{(01)}}\cdot\prod_{j\in[\kappa]} g_{1,i,1,j}^{\mathsf{sk}_{1,i,j}^{10}}, h\right),$$
$$y_{1,i,11} = \mathrm{e}\left(g^{x_{1,i}^{(01)}}\cdot g^{x_{1,i}^{(11)}}, h\right),$$

The rest of the computation of $\mathsf{ct}_{1,i}$ is exactly as in Game 1. The challenge ciphertext ct is adjusted in the exact same manner using elements $x^{(01)}, x^{(11)} \leftarrow\!\!\$\ \mathbb{Z}_q^*$ since it also is computed using the keys sk^{01} and sk^{11}.

The remaining ciphertexts now follow a similar process albeit with some small differences due to the different keys used. For each $\mathsf{ct}_{2,i}$, $i \in [Q_2]$, Enc, given secret keys $\mathsf{sk}_{2,i}^{00}$, sk^{01}, $\mathsf{sk}_{2,i}^{10}$, and $\mathsf{sk}_{2,i}^{11}$, samples an additional $x_{2,i}^{(01)} \leftarrow\!\!\$\ \mathbb{Z}_q^*$ and then computes

$$y_{2,i,10} = \mathrm{e}\left(g^{x_{2,i}^{(01)}}\cdot\prod_{j\in[\kappa]} g_{2,i,1,j}^{\mathsf{sk}_{2,i,j}^{10}}, h\right),$$
$$y_{2,i,11} = \mathrm{e}\left(g^{x_{2,i}^{(01)}}\cdot\prod_{j\in[\kappa]} g_{2,i,1,j}^{\mathsf{sk}_{2,i,j}^{11}}, h\right).$$

For each $\mathsf{ct}_{3,i}$, $i \in [Q_3]$, Enc, given secret keys $\mathsf{sk}_{3,i}^{00}$, $\mathsf{sk}_{3,i}^{01}$, $\mathsf{sk}_{3,i}^{10}$, and sk^{11}, samples an additional $x_{3,i}^{(11)} \leftarrow\!\!\$\ \mathbb{Z}_q^*$ and then computes

$$y_{3,i,01} = \mathrm{e}\left(\prod_{j\in[\kappa]}\left(g_{3,i,0,j}^{\mathsf{sk}_{3,i,j}^{00}}\right)\cdot g^{x_{3,i}^{(11)}}, h\right),$$
$$y_{3,i,11} = \mathrm{e}\left(\prod_{j\in[\kappa]}\left(g_{3,i,0,j}^{\mathsf{sk}_{3,i,j}^{01}}\right)\cdot g^{x_{3,i}^{(11)}}, h\right).$$

Game 3. In this game we alter the challenge ciphertext to perfectly hide the challenge messages m^{01}, m^{10} and m^{11}. In Game 2, by definition of Enc, each $c_{\delta_0\delta_1,i}$, $i \in [\kappa], \delta_0, \delta_1 \in \{0,1\}$, is computed as

$$c_{\delta_0\delta_1,i} = \left(h^{a_i^{(\delta_0\delta_1)}}, g_T^{m_i^{\delta_0\delta_1}}\cdot y_{\delta_0\delta_1}^{a_i^{(\delta_0\delta_1)}}\right)$$

with

$$
\begin{aligned}
y_{01}^{a_i^{(01)}} &= \mathrm{e}\left(\prod_{j\in[\kappa]}\left(g_{0,j}^{\mathsf{sk}_j^{00}}\right), h^{a_i^{(01)}}\right)\cdot \mathrm{e}\left(g^{x^{(11)}}, h^{a_i^{(01)}}\right),\\
y_{10}^{a_i^{(10)}} &= \mathrm{e}\left(g^{x^{(01)}}, h^{a_i^{(10)}}\right)\cdot \mathrm{e}\left(\prod_{j\in[\kappa]} g_{1,j}^{\mathsf{sk}_j^{10}}, h^{a_i^{(10)}}\right),\\
y_{11}^{a_i^{(11)}} &= \mathrm{e}\left(g^{x^{(01)}}, h^{a_i^{(11)}}\right)\cdot \mathrm{e}\left(g^{x^{(11)}}, h^{a_i^{(11)}}\right),
\end{aligned}
$$

for all $i \in [\kappa]$. For Game 3, we change each $c_{\delta_0\delta_1,i}$, $i \in [\kappa], \delta_0, \delta_1 \in \{0,1\}$ except for $\delta_0 = \delta_1 = 0$ to be computed as

$$
c_{\delta_0\delta_1,i} = \left(h^{a_i^{(\delta_0\delta_1)}}, g_T^{m_i^{\delta_0\delta_1}}\cdot y_{\delta_0\delta_1,i}\right),
$$

where $y_{\delta_0\delta_1,1}, \ldots, y_{\delta_0\delta_1,\kappa} \leftarrow\!\!\$\ \mathbb{G}$. Now each $m_i^{\delta_0\delta_1}$ is perfectly hidden by the uniform $y_{\delta_0\delta_1,i}$.

We now prove that the hybrid games are computationally indistinguishable.

Game 1 $\overset{\mathrm{c}}{\approx}$ Game 2: Define vectors

$$
\begin{aligned}
\mathbf{g}_{\gamma,i_\gamma,\delta} &= (g_{\gamma,i_\gamma,\delta,1}, \ldots, g_{\gamma,i_\gamma,\delta,\kappa}) \quad \forall \gamma \in [3], i \in [Q_\gamma], \delta \in \{0,1\},\\
\mathbf{g}_\delta &= (g_{\delta,1}, \ldots, g_{\delta,1}) \quad \forall \delta \in \{0,1\}.
\end{aligned}
$$

The elements inside the ciphertexts that depend on sk^{01} and sk^{11} can then be written as

$$
\begin{aligned}
\prod_{j\in[\kappa]} g_{\gamma,i_\gamma,\delta,j}^{\mathsf{sk}_j^{\delta 1}} &= \mathbf{g}_{\gamma,i_\gamma,\delta}^{\mathsf{sk}^{\delta 1}} \quad \forall \gamma \in [3], i \in [Q_\gamma], \delta \in \{0,1\},\\
\prod_{j\in[\kappa]} g_{\delta,j}^{\mathsf{sk}_j^{\delta 1}} &= \mathbf{g}_\delta^{\mathsf{sk}^{\delta 1}} \quad \forall \delta \in \{0,1\}.
\end{aligned}
$$

Our choice of κ and Lemma 12 ensures that

$$
H_\infty(\mathsf{sk}^{01}) = H_\infty(\mathsf{sk}^{11}) \geq \kappa - \frac{3}{2}\log\kappa \geq \kappa - \frac{1}{2}\log\kappa - 1/2 \geq \log q + 2\lambda,
$$

where the second inequality holds due to our choice of κ as a positive, even integer. Hence, we have $\kappa \geq 2$ (and actually $\kappa \gg 2$ much larger for reasonable choices of λ).
We can now apply Lemma 13, resulting in

$$
\begin{aligned}
\left\{\mathbf{g}_{\gamma,i_\gamma,0}, \mathbf{g}_0, \mathbf{g}_{\gamma,i_\gamma,0}^{\mathsf{sk}^{01}}, \mathbf{g}_0^{\mathsf{sk}^{01}}\right\}_{\gamma\in[3],i_\gamma\in[Q_\gamma]} &\overset{\mathrm{c}}{\approx} \left\{\mathbf{g}_{\gamma,i_\gamma,0}, \mathbf{g}_0, g'_{\gamma,i_\gamma,0}, g'_0\right\}_{\gamma\in[3],i_\gamma\in[Q_\gamma]},\\
\left\{\mathbf{g}_{\gamma,i_\gamma,1}, \mathbf{g}_1, \mathbf{g}_{\gamma,i_\gamma,1}^{\mathsf{sk}^{11}}, \mathbf{g}_1^{\mathsf{sk}^{11}}\right\}_{\gamma\in[3],i_\gamma\in[Q_\gamma]} &\overset{\mathrm{c}}{\approx} \left\{\mathbf{g}_{\gamma,i_\gamma,1}, \mathbf{g}_1, g'_{\gamma,i_\gamma,1}, g'_1\right\}_{\gamma\in[3],i_\gamma\in[Q_\gamma]},
\end{aligned}
\tag{10}
$$

where $g'_{\gamma,i_\gamma,0}, g'_{\gamma,i_\gamma,1}, g'_0, g'_1 \leftarrow\!\!\$\ \mathbb{G}$ look like fresh uniform elements. Since $\mathbb{G}$ is cyclic, these elements can also be written with respect to basis g. Hence, the right side of Equation (10) corresponds to Game 2.

Game 2 $\overset{\mathrm{c}}{\approx}$ Game 3: We apply Corollary 5 twice, resulting in

$$\begin{aligned}
&\{h, h^{a_1^{(01)}}, \ldots, h^{a_\kappa^{(01)}}, h^{a_1^{(11)}}, \ldots, h^{a_\kappa^{(11)}},\\
&\quad h^{x^{(11)}}, h^{x^{(11)}a_1^{(01)}}, \ldots, h^{x^{(11)}a_\kappa^{(01)}}, h^{x^{(11)}a_1^{(11)}}, \ldots, h^{x^{(11)}a_\kappa^{(11)}}\}\\
\overset{\mathrm{c}}{\approx}\ &\{h, h^{a_1^{(01)}}, \ldots, h^{a_\kappa^{(01)}}, h^{a_1^{(11)}}, \ldots, h^{a_\kappa^{(11)}},\\
&\quad h^{(11)}, h_1^{(01)}, \ldots, h_\kappa^{(01)}, h_1^{(1111)}, \ldots, h_\kappa^{(1111)}\},
\end{aligned}$$

where $h^{(11)}, h_1^{(01)}, \ldots, h_\kappa^{(01)}, h_1^{(1111)}, \ldots, h_\kappa^{(1111)} \leftarrow\!\!\$\ \mathbb{H}$, and

$$\begin{aligned}
&\{h, h^{a_1^{(10)}}, \ldots, h^{a_\kappa^{(10)}}, h^{a_1^{(11)}}, \ldots, h^{a_\kappa^{(11)}},\\
&\quad h^{x^{(01)}}, h^{x^{(01)}a_1^{(10)}}, \ldots, h^{x^{(01)}a_\kappa^{(10)}}, h^{x^{(01)}a_1^{(11)}}, \ldots, h^{x^{(01)}a_\kappa^{(11)}}\}\\
\overset{\mathrm{c}}{\approx}\ &\{h, h^{a_1^{(10)}}, \ldots, h^{a_\kappa^{(10)}}, h^{a_1^{(11)}}, \ldots, h^{a_\kappa^{(11)}},\\
&\quad h^{(01)}, h_1^{(10)}, \ldots, h_\kappa^{(10)}, h_1^{(0111)}, \ldots, h_\kappa^{(0111)}\},
\end{aligned}$$

where $h^{(01)}, h_1^{(10)}, \ldots, h_\kappa^{(10)}, h_1^{(0111)}, \ldots, h_\kappa^{(0111)} \leftarrow\!\!\$\ \mathbb{H}$. Applying this to the $c_{\delta_0\delta_1,i}$, $i \in [\kappa]$, $\delta_0, \delta_1 \in \{0,1\}$ except $\delta_0 = \delta_1 = 0$, in Game 2, we obtain

$$\begin{aligned}
c_{01,i} &= \left(h^{a_i^{(01)}}, g_T^{m_i^{01}} \cdot y_{01}^{a_i^{(01)}}\right)\\
&= \left(h^{a_i^{(01)}}, g_T^{m_i^{01}} \cdot \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{\mathsf{sk}_j^{00}}\right), h^{a_i^{(01)}}\right) \cdot \mathrm{e}\left(g^{x^{(11)}}, h^{a_i^{(01)}}\right)\right)\\
&\overset{\mathrm{c}}{\approx} \left(h^{a_i^{(01)}}, g_T^{m_i^{01}} \cdot \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{\mathsf{sk}_j^{00}}\right), h^{a_i^{(01)}}\right) \cdot \mathrm{e}\left(g, h_i^{(01)}\right)\right),\\
c_{10,i} &= \left(h^{a_i^{(10)}}, g_T^{m_i^{10}} \cdot y_{10}^{a_i^{(10)}}\right)\\
&= \left(h^{a_i^{(10)}}, g_T^{m_i^{10}} \cdot \mathrm{e}\left(g^{x^{(01)}}, h^{a_i^{(10)}}\right) \cdot \mathrm{e}\left(\prod_{j\in[\kappa]} g_{1,j}^{\mathsf{sk}_j^{10}}, h^{a_i^{(10)}}\right)\right) \qquad (11)\\
&\overset{\mathrm{c}}{\approx} \left(h^{a_i^{(10)}}, g_T^{m_i^{10}} \cdot \mathrm{e}\left(g, h_i^{(10)}\right) \cdot \mathrm{e}\left(\prod_{j\in[\kappa]} g_{1,j}^{\mathsf{sk}_j^{10}}, h^{a_i^{(10)}}\right)\right),\\
c_{11,i} &= \left(h^{a_i^{(11)}}, g_T^{m_i^{11}} \cdot y_{11}^{a_i^{(11)}}\right)\\
&= \left(h^{a_i^{(11)}}, g_T^{m_i^{11}} \cdot \mathrm{e}\left(g^{x^{(01)}}, h^{a_i^{(11)}}\right) \cdot \mathrm{e}\left(g^{x^{(11)}}, h^{a_i^{(11)}}\right)\right)\\
&\overset{\mathrm{c}}{\approx} \left(h^{a_i^{(11)}}, g_T^{m_i^{11}} \cdot \mathrm{e}\left(g, h_i^{(0111)}\right) \cdot \mathrm{e}\left(g, h_i^{(1111)}\right)\right).
\end{aligned}$$

Since the $h_i^{(01)}, h_i^{(10)}, h_i^{(0111)}$, and $h_i^{(1111)}$ are uniformly distributed, the message terms are also uniformly distributed. Therefore, the right side of Equation (11) corresponds to Game 3.

□

J Proof of Theorem 25

For the proof of Theorem 25 we will use the following helper lemma. It roughly states that Eval in Construction 15

- applies the given key and message homomorphisms correctly,
- permutes the ciphertexts correctly,
- and satisfies some requirements on the randomness of the resulting ciphertexts.

Essentially, it covers all requirements of the KMH rerandomizability definition except for the rerandomization of the ciphertext. For that we will use a separate argument.

Lemma 45. *Let*

$$\mathsf{ct} = \left(\mathsf{ct}^1, \mathsf{ct}^2, \mathsf{ct}^3, \mathsf{ct}^4, g_{0,1}, \ldots, g_{0,\kappa}, g_{1,1}, \ldots, g_{1,\kappa}\right)$$

with

$$\mathsf{ct}^i = (c_{i,1}, \ldots, c_{i,\kappa}, y_i) \quad \forall i \in [4]$$

be given by

$$\begin{pmatrix} y_{\pi_1(1)} \\ y_{\pi_1(2)} \\ y_{\pi_1(3)} \\ y_{\pi_1(4)} \end{pmatrix} = \begin{pmatrix} \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{00}} \cdot g_{1,j}^{s_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{00}} \cdot g_{1,j}^{s_j^{11}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{10}} \cdot g_{1,j}^{s_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{10}} \cdot g_{1,j}^{s_j^{11}}\right), h\right) \end{pmatrix},$$

$$c_{\pi_1(i),j} = \left(h^{a_{1,j}^{(i)}}, g_T^{m_j^i} \cdot y_{\pi_1(i)}^{a_{1,j}^{(i)}}\right) \quad \forall j \in [\kappa], i \in [4].$$

where

$$\begin{aligned} &(a_{1,1}^{(i)}, \ldots, a_{1,\kappa}^{(i)}) \in \mathbb{Z}_q^\kappa \quad \forall i \in [4], \\ &(g_{0,1}, \ldots, g_{0,\kappa}), (g_{1,1}, \ldots, g_{1,\kappa}) \in \mathbb{G}^\kappa, \\ &(s_1^{\beta_0\beta_1}, \ldots, s_\kappa^{\beta_0\beta_1}) \in \{0,1\}^\kappa \quad \forall \beta_0, \beta_1 \in \{0,1\}, \\ &\pi_1 \in S_4. \end{aligned}$$

Then for all $\lambda, \nu \in \mathbb{N}$; $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $(\sigma_{0,n}, \sigma_{1,n}, \tau_n)_{n\in[\nu-1]} \in \mathcal{F}^{3\nu-3}$, $r_2, \ldots, r_\nu \in R_{\mathsf{Eval}}$ *with*

$$\begin{aligned} r_n = (&a^{(1)}_{n,1}, \ldots, a^{(1)}_{n,\kappa}, a^{(2)}_{n,1}, \ldots, a^{(2)}_{n,\kappa}, \\ &a^{(3)}_{n,1}, \ldots, a^{(3)}_{n,\kappa}, a^{(4)}_{n,1}, \ldots, a^{(4)}_{n,\kappa}, d_n, \pi_n) \in \mathbb{Z}_q^{4\kappa} \times \mathbb{Z}_q^* \times S_4 \quad \forall n = 2, \ldots, \nu, \end{aligned}$$

and

$$\widehat{\mathsf{ct}} \leftarrow \mathsf{Eval}(\mathsf{pp}, \sigma_{0,\nu-1}, \sigma_{1,\nu-1}, \tau_{\nu-1}, \ldots \mathsf{Eval}(\mathsf{pp}, \sigma_{0,1}, \sigma_{1,1}, \tau_1, \mathsf{ct}; r_2) \ldots; r_\nu),$$

where

$$\begin{aligned} \left(\widehat{\mathsf{ct}}^1, \widehat{\mathsf{ct}}^2, \widehat{\mathsf{ct}}^3, \widehat{\mathsf{ct}}^4, \hat{g}_{0,1}, \ldots, \hat{g}_{0,\kappa}, \hat{g}_{1,1}, \ldots, \hat{g}_{1,\kappa}\right) &= \widehat{\mathsf{ct}}, \\ (\hat{c}_{i,1}, \ldots, \hat{c}_{i,\kappa}, \hat{y}_i) &= \widehat{\mathsf{ct}}^i \quad \forall i \in [4] \end{aligned}$$

it holds that

$$\begin{aligned} \hat{y}_{\pi(\nu,i)} &= y^d_{\pi_1(i)} \quad \forall i \in [4], \\ \hat{g}_{\beta,\sigma_\beta(\nu-1,j)} &= g^d_{\beta,j} \quad \forall \beta \in \{0,1\}, j \in [\kappa], \\ \hat{c}_{\pi(\nu,i),\tau(\nu-1,j)} &= \left(h^{u^{(i)}_j}, g_T^{m^i_j} \cdot \hat{y}^{u^{(i)}_j}_{\pi(\nu,i)}\right) \forall i \in [4], j \in [\kappa] \end{aligned}$$

where $u^{(i)}_1, \ldots, u^{(i)}_\kappa \in \mathbb{Z}_q$ *for all* $i \in [4]$, $d \in \mathbb{Z}_q^*$ *and we denote* $\pi(n, \cdot) = \pi_n(\ldots \pi_1(\cdot) \ldots)$ *for all* $n \in [\nu]$, *and* $\sigma_{\beta_0}(n, \cdot) = \sigma_{\beta,n}(\ldots \sigma_{\beta,1}(\cdot) \ldots)$ *and* $\tau(n, \cdot) = \tau_n(\ldots \tau_1(\cdot) \ldots)$ *for all* $\beta \in \{0,1\}, n \in [\nu - 1]$.

Proof. We prove this via induction We first prove the base case $\nu = 1$. To that end, let

$$\widehat{\mathsf{ct}} \leftarrow \mathsf{Eval}(\mathsf{pp}, \sigma_{0,1}, \sigma_{1,1}, \tau_1, \mathsf{ct}; r_2),$$

where

$$\begin{aligned} \left(\widehat{\mathsf{ct}}^1, \widehat{\mathsf{ct}}^2, \widehat{\mathsf{ct}}^3, \widehat{\mathsf{ct}}^4, \hat{g}_{0,1}, \ldots, \hat{g}_{0,\kappa}, \hat{g}_{1,1}, \ldots, \hat{g}_{1,\kappa}\right) &= \widehat{\mathsf{ct}}, \\ (\hat{c}_{i,1}, \ldots, \hat{c}_{i,\kappa}, \hat{y}_i) &= \widehat{\mathsf{ct}}^i \quad \forall i \in [4]. \end{aligned}$$

By definition of Eval in Construction 15, $\widehat{\mathsf{ct}}$ is computed as

$$\begin{aligned} \hat{y}_{\pi_2(\pi_1(i))} &= y^{d_2}_{\pi_1(i)} \quad \forall i \in [4], \\ \hat{g}_{\beta,\sigma_{\beta,1}(j)} &= g^{d_2}_{\beta,j} \quad \forall \beta \in \{0,1\}, j \in [\kappa], \\ \hat{c}_{\pi_2(\pi_1(i)),\tau(j)} &= \left(h^{\left(a^{(i)}_{1,j} + a^{(\pi_1(i))}_{2,\tau(j)}\right)/d_2}, g_T^{m^i_j} \cdot y^{a^{(i)}_{1,j} + a^{(\pi_1(i))}_{2,\tau(j)}}_{\pi_1(i)}\right) \quad \forall i \in [4], j \in [\kappa]. \end{aligned}$$

By writing

$$y^{a^{(i)}_{1,j} + a^{(\pi_1(i))}_{2,\tau(j)}}_{\pi_1(i)} = \hat{y}^{\left(a^{(i)}_{1,j} + a^{(\pi_1(i))}_{2,\tau(j)}\right)/d_2}_{\pi_2(\pi_1(i))},$$

and setting

$$u_j^{(i)} = (a_{1,j}^{(i)} + a_{2,\tau(j)}^{(\pi_1(i))})/d_2 \quad \forall j \in [\kappa], i \in [4]$$

we obtain the desired result for the base case $\nu = 1$.

We now prove that if Lemma 45 holds for some $\nu = z - 1 \in \mathbb{N}$, then it holds also for $\nu = z$. For all $n \in [z]$, define

$$\mathsf{ct}_n \leftarrow \mathsf{Eval}(\mathsf{pp}, \sigma_{0,n-1}, \sigma_{1,n-1}, \tau_{n-1}, \dots \mathsf{Eval}(\mathsf{pp}, \sigma_{0,1}, \sigma_{1,1}, \tau_1, \mathsf{ct}; r_2) \dots; r_n),$$

where

$$\begin{aligned}\left(\mathsf{ct}_n^1, \mathsf{ct}_n^2, \mathsf{ct}_n^3, \mathsf{ct}_n^4, g_{n,0,1}, \dots, g_{n,0,\kappa}, g_{n,1,1}, \dots, g_{n,1,\kappa}\right) &= \mathsf{ct}_n, \\ (c_{n,i,1}, \dots, c_{n,i,\kappa}, y_{n,i}) &= \mathsf{ct}_n^i \quad \forall i \in [4].\end{aligned}$$

Since we assumed that Lemma 45 holds for $\nu = z - 1$, we can write ct_{z-1} as

$$\begin{aligned} y_{z-1,\pi(z-1,i)} &= y_{\pi_1(i)}^d \quad \forall i \in [4], \\ g_{z-1,\beta,\sigma_\beta(z-2,j)} &= g_{\beta,j}^d \quad \forall \beta \in \{0,1\}, j \in [\kappa], \\ c_{z-1,\pi(z-1,i),\tau(z-2,j)} &= \left(h^{u_j^{(i)}}, g_T^{m_j^i} \cdot y_{z-1,\pi(z-1,i)}^{u_j^{(i)}}\right) \forall i \in [4], j \in [\kappa] \end{aligned}$$

where $u_1^{(i)}, \dots, u_\kappa^{(i)} \in \mathbb{Z}_q$ for all $i \in [4]$, $d \in \mathbb{Z}_q^*$. By definition of Eval in Construction 15, ct_z is then given by

$$\begin{aligned} y_{z,\pi(z,i)} &= y_{z-1,\pi(z-1,i)}^{d_z} = y_{\pi_1(i)}^{dd_z} \quad \forall i \in [4], \\ g_{z,\beta,\sigma_\beta(z-1,j)} &= g_{\beta,j}^{dd_z} \quad \forall \beta \in \{0,1\}, j \in [\kappa], \\ c_{z,\pi(z,i),\tau(z-1,j)} &= \left(h^{\left(u_j^{(i)} + a_{z,\tau(z-1,j)}^{(\pi(z-1,i))}\right)/d_z}, g_T^{m_j^i} \cdot y_{z-1,\pi(z-1,i)}^{u_j^{(i)} + a_{z,\tau(z-1,j)}^{(\pi(z-1,i))}}\right) \forall i \in [4], j \in [\kappa]. \end{aligned}$$

We rewrite

$$\begin{aligned} c_{z,\pi(z,i),\tau(z-1,j)} &= \left(h^{\left(u_j^{(i)} + a_{z,\tau(z-1,j)}^{(\pi(z-1,i))}\right)/d_z}, g_T^{m_j^i} \cdot y_{z-1,\pi(z-1,i)}^{u_j^{(i)} + a_{z,\tau(z-1,j)}^{(\pi(z-1,i))}}\right) \\ &= \left(h^{\left(u_j^{(i)} + a_{z,\tau(z-1,j)}^{(\pi(z-1,i))}\right)/d_z}, g_T^{m_j^i} \cdot y_{z-1,\pi(z-1,i)}^{d_z\left(u_j^{(i)} + a_{z,\tau(z-1,j)}^{(\pi(z-1,i))}\right)/d_z}\right) \\ &= \left(h^{\left(u_j^{(i)} + a_{z,\tau(z-1,j)}^{(\pi(z-1,i))}\right)/d_z}, g_T^{m_j^i} \cdot y_{z,\pi(z,i)}^{\left(u_j^{(i)} + a_{z,\tau(z-1,j)}^{(\pi(z-1,i))}\right)/d_z}\right) \end{aligned}$$

and set

$$w_j^{(i)} = \left(u_j^{(i)} + a_{z,\tau(z-1,j)}^{(\pi(z-1,i))}\right)/d \quad \forall j \in [\kappa], i \in [4].$$

Overall ct_z then satisfies

$$\begin{aligned} y_{z,\pi(z,i)} &= y_{\pi_1(i)}^{dd_z} \quad \forall i \in [4], \\ g_{z,\beta,\sigma_\beta(z-1,j)} &= g_{\beta,j}^{dd_z} \quad \forall \beta \in \{0,1\}, j \in [\kappa], \\ c_{z,\pi(z,i),\tau(z-1,j)} &= \left(h^{w_j^{(i)}}, g_T^{m_j^i} \cdot y_{z,\pi(z,i)}^{w_j^{(i)}}\right) \forall i \in [4], j \in [\kappa] \end{aligned}$$

which completes the proof, since $dd_z \in \mathbb{Z}_q^*$ is exactly the statement of Lemma 45 for $\nu = z$. □

We can now proceed with the proof of Theorem 25.

Proof of Theorem 25. We start with case $b = 0$ of the $\mathsf{GKMHE\text{-}RERAND}_{\mathcal{A},\mathsf{GKMHE}}$ game and show that it is computationally indistinguishable from case $b = 1$.

Let ct be as in case $b = 0$ of the game $\mathsf{GKMHE\text{-}RERAND}_{\mathcal{A},\mathsf{GKMHE}}$, so

$$\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}^{00}, \mathsf{sk}^{01}, \mathsf{sk}^{10}, \mathsf{sk}^{11}, m^1, m^2, m^3, m^4; r_1)$$

where

$$\begin{aligned}
&(g_{0,1}, \ldots, g_{0,\kappa}, g_{1,1}, \ldots, g_{1,\kappa},\\
&\ a_{1,1}^{(1)}, \ldots, a_{1,\kappa}^{(1)}, a_{1,1}^{(2)}, \ldots, a_{1,\kappa}^{(2)},\\
&\ a_{1,1}^{(3)}, \ldots, a_{1,\kappa}^{(3)}, a_{1,1}^{(4)}, \ldots, a_{1,\kappa}^{(4)}, \pi_1) = r_1 \in \mathbb{G}^{2\kappa} \times \mathbb{Z}_q^{4\kappa} \times S_4,\\
&(s_1^{\beta_0\beta_1}, \ldots, s_\kappa^{\beta_0\beta_1}) = \mathsf{sk}^{\beta_0\beta_1} \in \{0,1\}^\kappa \quad \forall \beta_0, \beta_1 \in \{0,1\},\\
&(m_1^i, \ldots, m_\kappa^i) = m^i \in \{0,1\}^\kappa \quad \forall i \in [4],
\end{aligned}$$

and

$$\begin{aligned}
\left(\mathsf{ct}^1, \mathsf{ct}^2, \mathsf{ct}^3, \mathsf{ct}^4, g_{0,1}, \ldots, g_{0,\kappa}, g_{1,1}, \ldots, g_{1,\kappa}\right) &= \mathsf{ct},\\
(c_{i,1}, \ldots, c_{i,\kappa}, y_i) &= \mathsf{ct}^i \quad \forall i \in [4].
\end{aligned}$$

Then by definition of Enc in Construction 15, ct is computed as

$$\begin{aligned}
\begin{pmatrix} y_{\pi_1(1)} \\ y_{\pi_1(2)} \\ y_{\pi_1(3)} \\ y_{\pi_1(4)} \end{pmatrix} &= \begin{pmatrix} \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{00}} \cdot g_{1,j}^{s_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{00}} \cdot g_{1,j}^{s_j^{11}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{10}} \cdot g_{1,j}^{s_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{s_j^{10}} \cdot g_{1,j}^{s_j^{11}}\right), h\right) \end{pmatrix},\\
c_{\pi_1(i),j} &= \left(h^{a_{1,j}^{(i)}}, g_T^{m_j^i} \cdot y_{\pi_1(i)}^{a_{1,j}^{(i)}}\right) \quad \forall i \in [4], j \in [\kappa].
\end{aligned}$$

Now consider the $(t-2)^{\text{nd}}$ rerandomization of ct, given by

$$\widehat{\mathsf{ct}} \leftarrow \mathsf{Eval}(\mathsf{pp}, \sigma_{0,t-2}, \sigma_{1,t-2}, \tau_{t-2}, \ldots \mathsf{Eval}(\sigma_{0,1}, \sigma_{1,1}, \tau_1, \mathsf{ct}; r_2) \ldots; r_{t-1}).$$

where we parse

$$\begin{aligned}
&(a_{t-1,1}^{(1)}, \ldots, a_{t-1,\kappa}^{(1)}, a_{t-1,1}^{(2)}, \ldots, a_{t-1,\kappa}^{(2)},\\
&\ a_{t-1,1}^{(3)}, \ldots, a_{t-1,\kappa}^{(3)}, a_{t-1,1}^{(4)}, \ldots, a_{t-1,\kappa}^{(4)}, d_{t-1}, \pi_{t-1}) = r_{t-1} \in \mathbb{Z}_q^{4\kappa} \times \mathbb{Z}_q^* \times S_4,
\end{aligned}$$

and

$$\left(\widehat{\mathsf{ct}}^1, \widehat{\mathsf{ct}}^2, \widehat{\mathsf{ct}}^3, \widehat{\mathsf{ct}}^4, \hat{g}_{0,1}, \ldots, \hat{g}_{0,\kappa}, \hat{g}_{1,1}, \ldots, \hat{g}_{1,\kappa}\right) = \widehat{\mathsf{ct}},$$
$$(\hat{c}_{i,1}, \ldots, \hat{c}_{i,\kappa}, \hat{y}_i) = \widehat{\mathsf{ct}}^i \quad \forall i \in [4].$$

Furthermore, define notation $\pi(n,\cdot) = \pi_n(\ldots\pi_1(\cdot)\ldots)$ for all $n \in [\nu]$, and $\sigma_{\beta_0}(n,\cdot) = \sigma_{\beta,n}(\ldots\sigma_{\beta,1}(\cdot)\ldots)$ and $\tau(n,\cdot) = \tau_n(\ldots\tau_1(\cdot)\ldots)$ for all $\beta \in \{0,1\}, n \in [\nu-1]$ as in Lemma 45.

By applying Lemma 45 with $\nu = t-1$, we can write $\widehat{\mathsf{ct}}$ as

$$\begin{aligned}
\hat{y}_{\pi(t-1,i)} &= y^d_{\pi_1(i)} \quad \forall i \in [4],\\
\hat{g}_{\beta,\sigma_\beta(t-2,j)} &= g^d_{\beta,j} \quad \forall \beta \in \{0,1\}, j \in [\kappa],\\
\hat{c}_{\pi(t-1,i),\tau(t-2,j)} &= \left(h^{u^{(i)}_j}, g_T^{m^i_j} \cdot \hat{y}^{u^{(i)}_j}_{\pi(t-1,i)}\right) \forall i \in [4], j \in [\kappa],
\end{aligned}$$

where $u^{(i)}_1, \ldots, u^{(i)}_\kappa \in \mathbb{Z}_q$ for all $i \in [4]$, and $d \in \mathbb{Z}^*_q$.

We now show that the last call to Eval where the adversary $\mathcal{A}$ does *not* obtain the randomness r_t suffices to rerandomize $\widehat{\mathsf{ct}}$. The full challenge ciphertext in case $b = 0$ is given by

$$\mathsf{ct}' \leftarrow \mathsf{Eval}(\mathsf{pp}, \sigma_{0,t-1}, \sigma_{1,t-1}, \tau_{t-1}, \widehat{\mathsf{ct}}; r_t).$$

where we parse

$$\begin{aligned}
&(a^{(1)}_{t,1}, \ldots, a^{(1)}_{t,\kappa}, a^{(2)}_{t,1}, \ldots, a^{(2)}_{t,\kappa},\\
&\ a^{(3)}_{t,1}, \ldots, a^{(3)}_{t,\kappa}, a^{(4)}_{t,1}, \ldots, a^{(4)}_{t,\kappa}, d_t, \pi_t) = r_t \in \mathbb{Z}_q^{4\kappa} \times Z_q^* \times S_4,\\
&(\mathsf{ct}'^1, \mathsf{ct}'^2, \mathsf{ct}'^3, \mathsf{ct}'^4, g'_{0,1}, \ldots, g'_{0,\kappa}, g'_{1,1}, \ldots, g'_{1,\kappa}) = \mathsf{ct}',\\
&(c'_{i,1}, \ldots, c'_{i,\kappa}, y'_i) = \mathsf{ct}'^i \quad \forall i \in [4].
\end{aligned}$$

Again, by definition of Eval in Construction 15, ct' is computed as

$$\begin{aligned}
y'_{\pi(t,i)} &= \hat{y}^{d_t}_{\pi(t-1,i)} = y^{dd_t}_{\pi_1(i)}\\
g'_{\beta,\sigma_\beta(t-1,j)} &= \hat{g}^{d_t}_{\beta,\sigma_\beta(t-2,j)} = g^{dd_t}_{\beta,j} \quad \forall \beta \in \{0,1\},\\
c'_{\pi(t,i),\tau(t-1,j)} &= \left(h^{\left(u^{(i)}_j + a^{(\pi(t-1,i))}_{t,\tau(t-1,j)}\right)/d_t}, g_T^{m^i_j} \cdot \hat{y}^{u^{(i)}_j + a^{(\pi(t-1,i))}_{t,\tau(z-1,j)}}_{\pi(t-1,i)}\right)\\
&= \left(h^{\left(u^{(i)}_j + a^{(\pi(t-1,i))}_{t,\tau(t-1,j)}\right)/d_t}, g_T^{m^i_j} \cdot y'^{\left(u^{(i)}_j + a^{(\pi(t-1,i))}_{t,\tau(z-1,j)}\right)/d_t}_{\pi(t,i)}\right)
\end{aligned}$$

for all $i \in [4], j \in [\kappa]$. Since the r_t value is sampled uniformly at random from R_{Eval} and never given to the adversary $\mathcal{A}$, each $a^{(\pi(t-1,i))}_{t,\tau(t-1,j)}$, for $j \in [\kappa], i \in [4]$, looks uniformly random to $\mathcal{A}$. Then also the value $(u^{(i)}_j + a^{(\pi(t-1,i))}_{t,\tau(z-1,j)})/d_t$ looks uniformly random to $\mathcal{A}$. Therefore we get that

$$c'_{\pi(t,i),\tau(t-1,j)} \equiv \left(h^{w^{(i)}_j}, g_T^{m^i_j} \cdot y'^{w^{(i)}_j}_{\pi(t,i)}\right) \quad \forall i \in [4], j \in [\kappa],$$

where $w_j^{(i)} \leftarrow\!\!\$\ \mathbb{Z}_q$ for all $j \in [\kappa], i \in [4]$.

It remains to prove that the remaining elements $y'_{\pi(t,i)}$, $i \in [4]$, and $g'_{\beta,\sigma_\beta(t-1,j)}$, $\beta \in \{0,1\}$, $j \in [\kappa]$, look fresh as well. We rewrite

$$\begin{pmatrix} y'_{\pi(t,1)} \\ y'_{\pi(t,2)} \\ y'_{\pi(t,3)} \\ y'_{\pi(t,4)} \end{pmatrix} = \begin{pmatrix} \hat{y}^{d_t}_{\pi(t-1,1)} \\ \hat{y}^{d_t}_{\pi(t-1,2)} \\ \hat{y}^{d_t}_{\pi(t-1,3)} \\ \hat{y}^{d_t}_{\pi(t-1,4)} \end{pmatrix} = \begin{pmatrix} y^{dd_t}_{\pi_1(1)} \\ y^{dd_t}_{\pi_1(2)} \\ y^{dd_t}_{\pi_1(3)} \\ y^{dd_t}_{\pi_1(4)} \end{pmatrix}$$
$$= \begin{pmatrix} \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{dd_t s_j^{00}} \cdot g_{1,j}^{dd_t s_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{dd_t s_j^{00}} \cdot g_{1,j}^{dd_t s_j^{11}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{dd_t s_j^{10}} \cdot g_{1,j}^{dd_t s_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(g_{0,j}^{dd_t s_j^{10}} \cdot g_{1,j}^{dd_t s_j^{11}}\right), h\right) \end{pmatrix}$$

Since

- d_t stays hidden from $\mathcal{A}$ and is uniformly distributed over $\mathbb{Z}_q^*$ due to $r_t \leftarrow\!\!\$\ R_{\mathsf{Eval}}$,
- and each $g_{\beta,j}$, $\beta \in \{0,1\}, j \in [\kappa]$, is uniformly distributed over $\mathbb{G}$ due to $r_1 \leftarrow\!\!\$\ R_{\mathsf{Enc}}$

we can now apply generalized DDH (Corollary 5) over the group $\mathbb{G}$ to obtain

$$\begin{aligned} &\left\{g_{0,1}, \ldots, g_{0,\kappa}, g_{1,1}, \ldots g_{1,\kappa}, g_{0,1}^{d_t}, \ldots, g_{0,\kappa}^{d_t}, g_{1,1}^{d_t}, \ldots g_{1,\kappa}^{d_t}\right\} \\ &\overset{\mathrm{c}}{\approx} \left\{g_{0,1}, \ldots, g_{0,\kappa}, g_{1,1}, \ldots g_{1,\kappa}, v_{0,1}, \ldots, v_{0,\kappa}, v_{1,1}, \ldots v_{1,\kappa}\right\}, \end{aligned} \tag{12}$$

where $v_{\beta,j} \leftarrow\!\!\$\ \mathbb{G}$ for all $\beta \in \{0,1\}, j \in [\kappa]$. Using Equation (12) we obtain

$$g'_{\beta,\sigma_\beta(t-1,j)} = g_{\beta,j}^{dd_t} \overset{\mathrm{c}}{\approx} v_{\beta,j}^{d} \quad \forall \beta \in \{0,1\}, j \in [\kappa],$$

and

$$\begin{pmatrix} y'_{\pi(t,1)} \\ y'_{\pi(t,2)} \\ y'_{\pi(t,3)} \\ y'_{\pi(t,4)} \end{pmatrix} \overset{\mathrm{c}}{\approx} \begin{pmatrix} \mathrm{e}\left(\prod_{j\in[\kappa]} \left(v_{0,j}^{ds_j^{00}} \cdot v_{1,j}^{ds_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(v_{0,j}^{ds_j^{00}} \cdot v_{1,j}^{ds_j^{11}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(v_{0,j}^{ds_j^{10}} \cdot v_{1,j}^{ds_j^{10}}\right), h\right) \\ \mathrm{e}\left(\prod_{j\in[\kappa]} \left(v_{0,j}^{ds_j^{10}} \cdot v_{1,j}^{ds_j^{11}}\right), h\right) \end{pmatrix}.$$

Now let

$$\begin{aligned} \underline{r} = (&v_{0,1}^d, \ldots, v_{0,\kappa}^d, v_{1,1}^d, \ldots, v_{1,\kappa}^d, \\ &w_1^{(1)}, \ldots, w_\kappa^{(1)}, w_1^{(2)}, \ldots, w_\kappa^{(2)}, \\ &w_1^{(3)}, \ldots, w_\kappa^{(3)}, w_1^{(4)}, \ldots, w_1^{(4)}, \pi_t \circ \cdots \circ \pi_1) \in \mathbb{G}^{2\kappa} \times \mathbb{Z}_q^{4\kappa} \times S_4. \end{aligned}$$

By the previous arguments we obtain that

$$\mathsf{ct}' \overset{c}{\approx} \mathsf{Enc}(\mathsf{pp}, \mathsf{sk}'^{00}, \mathsf{sk}'^{01}, \mathsf{sk}'^{10}, \mathsf{sk}'^{11}, m'^1, m'^2, m'^3, m'^4; \underline{r}),$$

where

$$\begin{aligned} \mathsf{sk}'^{\beta_0\beta_1} &= \sigma_{\beta_0,t-1}(\sigma_{\beta_0,t-2}(\dots(\mathsf{sk}^{\beta_0\beta_1})\dots)) \quad \forall \beta_0, \beta_1 \in \{0,1\} \\ m'^i &= \tau_{t-1}(\tau_{t-2}(\dots(m^i)\dots)) \quad \forall i \in [4] \end{aligned}$$

as in case $b = 1$ of the game $\mathsf{GKMHE\text{-}RERAND}_{\mathcal{A},\mathsf{GKMHE}}$.

It remains to show that $\underline{r} \equiv r'$, as required for the case $b = 1$ of the game $\mathsf{GKMHE\text{-}RERAND}_{\mathcal{A},\mathsf{GKMHE}}$. Note that the $v_{\beta,j}^d$ are uniformly distributed over $\mathbb{G}$ since exponentiation with d is a bijection. Furthermore, the composition of permutations $\pi_t \circ \dots \circ \pi_1$ is uniformly distributed over S_4 since $\pi_t \leftarrow\!\!\$\ S_4$ is uniform. Overall we get that $\underline{r}$ is uniformly distributed over R_{Eval}, so $\underline{r} \equiv r'$ as desired. □

K Proof of Theorem 38

Proof. For the proof we will need some concepts related to evaluation of Boolean circuits and their garblings. First is the concept of topological ordering. A topological ordering of a directed acyclic graph, such as a Boolean circuit, is an ordering of the nodes, such that a node G comes before another node G' if there is a path from G to G'. Intuitively, for a Boolean circuit, a topological ordering of the gates is an ordering in which the gates can be evaluated. The topological ordering ensures that, when evaluating some gate, all necessary input values for the gate have been computed already. In the proof we will instead need a topological ordering of the wires. In such an ordering, a wire must be placed before another wire if the former is needed to compute the value on the latter wire when evaluating the circuit.

Specifically for garbled circuits, we will also need the concept of *active* and *inactive* labels [LP09]. Each wire in a garbled circuit has two associated labels. Given some input x as well as corresponding input labels for the circuit, the active labels are those that are obtained by the evaluator when evaluating the circuit on x. This includes the given input labels. The inactive labels are the ones that are not obtained, they remain hidden.

We now prove Theorem 38 using a sequence of hybrids to prove indistinguishability of case $b = 0$ and case $b = 1$ of the game $\mathsf{RER\text{-}PRIV}_{\mathcal{A},\mathsf{RGS}}$.

H_1**:** This hybrid corresponds to case $b = 0$ of the $\mathsf{RER\text{-}PRIV}_{\mathcal{A},\mathsf{RGS}}$ game.

H_2**:** In H_1, each garbled gate G'_i of gate $g_i = (w_\ell, w_r, w_o, op)$, $i \in [N]$, in $\mathcal{C}$ is computed as

$$G_i \leftarrow \mathsf{Enc}(\mathsf{pp}_{\mathsf{GKMHE}}, \mathsf{sk}^0_{w_\ell}, \mathsf{sk}^1_{w_\ell}, \mathsf{sk}^0_{w_r}, \mathsf{sk}^1_{w_r}, \mathsf{sk}^{op(0,0)}_{w_o}, \mathsf{sk}^{op(0,1)}_{w_o}, \mathsf{sk}^{op(1,0)}_{w_o}, \mathsf{sk}^{op(1,1)}_{w_o})$$

and

$$G'_i \leftarrow \mathsf{Eval}(\mathsf{pp}_{\mathsf{GKMHE}}, \sigma^{(t-1)}_{w_\ell}, \sigma^{(t-1)}_{w_r}, \sigma^{(t-1)}_{w_o}, \\ \dots \mathsf{Eval}(\mathsf{pp}_{\mathsf{GKMHE}}, \sigma^{(1)}_{w_\ell}, \sigma^{(1)}_{w_r}, \sigma^{(1)}_{w_o}, G_i) \dots).$$

In H_2 we instead now directly compute

$$G'_i \leftarrow \mathsf{Enc}(\mathsf{pp}_{\mathsf{GKMHE}}, \sigma_{w_\ell}(t-1, \mathsf{sk}^0_{w_\ell}), \sigma_{w_\ell}(t-1, \mathsf{sk}^1_{w_\ell}), \sigma_{w_r}(t-1, \mathsf{sk}^0_{w_r}), \\ \sigma_{w_r}(t-1, \mathsf{sk}^1_{w_r}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(0,0)}_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(0,1)}_{w_o}), \\ \sigma_{w_o}(t-1, \mathsf{sk}^{op(1,0)}_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(1,1)}_{w_o})),$$

where from now on we define

$$\sigma_{w_\ell}(i, \cdot) = \sigma^{(i)}_{w_\ell}(\dots \sigma^{(1)}_{w_\ell}(\cdot) \dots), \\ \sigma_{w_r}(i, \cdot) = \sigma^{(i)}_{w_r}(\dots \sigma^{(1)}_{w_r}(\cdot) \dots), \\ \sigma_{w_o}(i, \cdot) = \sigma^{(i)}_{w_o}(\dots \sigma^{(1)}_{w_o}(\cdot) \dots).$$

Each garbled gate is therefore a fresh encryption which contains the correctly transformed keys and messages. Indistinguishability of Hybrid 1 and Hybrid 2 follows straightforwardly by KMH rerandomizability (Definition 24) of the gate KMHE scheme GKMHE.

$H''_{3,0}$**:** To simplify notation we define this hybrid to be the same as hybrid H_2.

$H_{3,i} \forall i \in [M]$**:** By evaluating $\mathcal{C}$ using the challenge input labels I, corresponding to input x, we can designate *active* and *inactive* labels as defined previously. Without loss of generality let now $\mathsf{sk}^1_{w_j}$ be the active label and $\mathsf{sk}^0_{w_j}$ be the inactive label of each wire w_j. The other cases follow analogously.
Consider now the i-th wire w_i (by topological ordering) of the circuit $\mathcal{C}$ and all the gates $g_1, \dots, g_n$ that have this wire as one of their input wires. For notational simplicity assume without loss of generality that w_i is the left input wire and that w_i comes before w_r in the topological ordering, the other cases follow similarly. More details on these assumptions are given towards the end of the definition of $H_{3,i}$. Keeping these assumptions in mind, the garbling G'_j of gate $g_j = (w_i, w_r, w_o, op)$ from the previous hybrid can be written as

$$G'_j \leftarrow \mathsf{Enc}(\mathsf{pp}_{\mathsf{GKMHE}}, \sigma_{w_i}(t-1, \mathsf{sk}^0_{w_i}), \sigma_{w_i}(t-1, \mathsf{sk}^1_{w_i}), \sigma_{w_r}(t-1, \mathsf{sk}^0_{w_r}), \\ \sigma_{w_r}(t-1, \mathsf{sk}^1_{w_r}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(0,0)}_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(0,1)}_{w_o}), \\ \sigma_{w_o}(t-1, \mathsf{sk}^{op(1,0)}_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(1,1)}_{w_o})),$$

For hybrid $H_{3,i}$ we now replace the transformed *inactive* label $\sigma_{w_i}(t-1, \mathsf{sk}^0_{w_i})$ with a fresh key $\underline{\mathsf{sk}}^0_{w_i} \leftarrow \mathsf{KGen}(\mathsf{pp}_{\mathsf{GKMHE}})$. The new garbling G'_j for all $j \in [n]$ is then given by

$$G'_j \leftarrow \mathsf{Enc}(\mathsf{pp}_{\mathsf{GKMHE}}, \underline{\mathsf{sk}}^0_{w_i}, \sigma_{w_i}(t-1, \mathsf{sk}^1_{w_i}), \sigma_{w_r}(t-1, \mathsf{sk}^0_{w_r}), \\ \sigma_{w_r}(t-1, \mathsf{sk}^1_{w_r}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(0,0)}_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(0,1)}_{w_o}), \\ \sigma_{w_o}(t-1, \mathsf{sk}^{op(1,0)}_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(1,1)}_{w_o})),$$

Note that if w_i comes after w_r in the topological ordering, then the key corresponding to w_r would have been replaced in a previous hybrid already. The case that w_i is the right input wire only affects the ordering of arguments. We will explain below that indistinguishable of this hybrid from the previous one follows from key privacy under leakage (Definition 26) of the gate KMHE scheme.

$H'_{3,i} \forall i \in [M]$**:** Consider again the i-th wire w_i (topological ordering) of the circuit $\mathcal{C}$ and and all the gates $g_1, \ldots, g_n$, along with corresponding garblings $G'_1, \ldots, G'_n$, that have this wire as one of their input wires. In hybrid $H'_{3,i}$ we now replace the transformed *active* label $\sigma_{w_i}(t-1, \mathsf{sk}^1_{w_i})$ with a fresh key $\underline{\mathsf{sk}}^1_{w_i} \leftarrow \mathsf{KGen}(\mathsf{pp}_{\mathsf{GKMHE}})$.Each G'_j, $j \in [n]$, is then given by

$$\begin{aligned} G'_j \leftarrow \mathsf{Enc}(\mathsf{pp}_{\mathsf{GKMHE}}, \underline{\mathsf{sk}}^0_{w_i}, \underline{\mathsf{sk}}^1_{w_i}, \sigma_{w_r}(t-1, \mathsf{sk}^0_{w_r}), \\ \sigma_{w_r}(t-1, \mathsf{sk}^1_{w_r}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(0,0)}_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(0,1)}_{w_o}), \\ \sigma_{w_o}(t-1, \mathsf{sk}^{op(1,0)}_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(1,1)}_{w_o})). \end{aligned}$$

Indistinguishability from the previous hybrid follows by key privacy of the gate KMHE scheme (Definition 22).

$H''_{3,i} \forall i \in [M]$**:** Consider again the i-th wire w_i (topological ordering) of the circuit $\mathcal{C}$ and and all the gates $g_1, \ldots, g_n$, along with corresponding garblings $G'_1, \ldots, G'_n$, that have this wire as one of their input wires. For all $g_j = (w_\ell, w_r, w_o, \cdot)$, $j \in [n]$, check if w_i is the second input wire by topological ordering. If not, then define $H''_{3,i} = H'_{3,i}$. Otherwise, this means that the key replacement in $H'_{3,i}$ lead to both input wires of g_j now having fresh inactive and active labels. In $H''_{3,i}$, we now change all encrypted messages inside each G'_j to the active output label $\sigma_{w_o}(t-1, \mathsf{sk}^1_{w_o})$, which means that G'_j is computed as

$$\begin{aligned} G'_j \leftarrow \mathsf{Enc}(\mathsf{pp}_{\mathsf{GKMHE}}, \underline{\mathsf{sk}}^0_{w_\ell}, \underline{\mathsf{sk}}^1_{w_\ell}, \underline{\mathsf{sk}}^0_{w_r}, \underline{\mathsf{sk}}^0_{w_r}, \\ \sigma_{w_o}(t-1, \mathsf{sk}^1_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^1_{w_o}), \\ \sigma_{w_o}(t-1, \mathsf{sk}^1_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^1_{w_o})). \end{aligned}$$

Since $\underline{\mathsf{sk}}^0_{w_\ell}$ and $\underline{\mathsf{sk}}^0_{w_r}$ are kept secret from the adversary distinguishing between $H'_{3,i}$ and $H_{3,i}$, this step follows from CPA security (Definition 20) of the gate KMHE scheme.

Note that in $H'_{3,M}$, all ciphertexts inside the garbled circuit $\mathcal{C}$ encrypt only the active output labels. Therefore, $\mathcal{C}$ looks exactly like a simulated garbling as output by $\mathsf{Sim}_{\mathsf{Gb}}$.

H_4**:** This hybrid corresponds to case $b = 1$ of the $\mathsf{RER\text{-}PRIV}_{\mathcal{A},\mathsf{RGS}}$ game, i.e. $\mathcal{A}$ is given a fresh garbling.

We now sketch how to prove indistinguishability of these hybrids.

$H_1 \overset{\mathrm{c}}{\approx} H_2 = H''_{3,0}$**:** As mentioned before, this is just a straightforward application of KMH rerandomizability (Definition 24) of the gate KMHE scheme.

$H''_{3,i-1} \overset{\mathrm{c}}{\approx} H_{3,i} \forall i \in [M]$**:** This indistinguishability follows from key privacy under leakage (Definition 26) of the gate KMHE scheme. We will now sketch why this is the case.

Let $g_1, \ldots, g_n$ be all the gates that have wire w_i as one of their input wires (assume it is the left one again for simplicity), and let $G_1, \ldots, G_n$ be the corresponding garbled gates in the form of gate KMHE ciphertexts as in hybrid $H'_{3,i-1}$. Each G'_j is then, by definition of $H'_{3,i-1}$ given by

$$\begin{aligned} G'_j \leftarrow \mathsf{Enc}(\mathsf{pp}_{\mathsf{GKMHE}}, &\sigma_{w_i}(t-1, \mathsf{sk}^0_{w_i}), \sigma_{w_i}(t-1, \mathsf{sk}^1_{w_i}), \sigma_{w_r}(t-1, \mathsf{sk}^0_{w_r}), \\ &\sigma_{w_r}(t-1, \mathsf{sk}^1_{w_r}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(0,0)}_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(0,1)}_{w_o}), \\ &\sigma_{w_o}(t-1, \mathsf{sk}^{op(1,0)}_{w_o}), \sigma_{w_o}(t-1, \mathsf{sk}^{op(1,1)}_{w_o})). \end{aligned}$$

Also recall that

$$\begin{aligned} \sigma_{w_i}(t-1, \cdot) &= \sigma^{(t-1)}_{w_i}(\ldots \sigma^{(1)}_{w_i}(\cdot)\ldots), \\ \sigma_{w_r}(t-1, \cdot) &= \sigma^{(t-1)}_{w_r}(\ldots \sigma^{(1)}_{w_r}(\cdot)\ldots), \\ \sigma_{w_o}(t-1, \cdot) &= \sigma^{(t-1)}_{w_o}(\ldots \sigma^{(1)}_{w_o}(\cdot)\ldots). \end{aligned}$$

The function $\sigma^{(t-1)}_{w_i}$ is implied by the randomness r_t which an adversary distinguishing between $H'_{3,i-1}$ and $H_{3,i}$ does not obtain. The only leakage of $\sigma_{w_i}(t-1, \mathsf{sk}^0_{w_i})$ in each G'_j is via the active label $\sigma_{w_i}(t-1, \mathsf{sk}^1_{w_i})$ which the adversary can obtain via the active input labels I, and the randomness values $r_1, \ldots, r_{t-1}$ which leak the values $\sigma_{w_i}(t-2, \mathsf{sk}^0_{w_i})$ and $\sigma_{w_i}(t-2, \mathsf{sk}^1_{w_i})$. The only other leakage could be via the gates that have the wire w_i as their output wire, since those garbled gates could contain the inactive label $\sigma_{w_i}(t-1, \mathsf{sk}^0_{w_i})$ as one of the messages. However, definition of $H'_{3,i-1}$ ensures that all the garbled gates that are topologically before w_i, only encrypt the active labels.

Therefore, there is no further leakage of $\sigma_{w_i}(t-1, \mathsf{sk}^0_{w_i})$ except for $\sigma_{w_i}(t-1, \mathsf{sk}^1_{w_i})$, $\sigma_{w_i}(t-2, \mathsf{sk}^0_{w_i})$ and $\sigma_{w_i}(t-2, \mathsf{sk}^1_{w_i})$, exactly as required by the definition of key privacy under leakage (Definition 26). Therefore, we can use key privacy under leakage to replace the key $\sigma_{w_i}(t-1, \mathsf{sk}^0_{w_i})$ in all G'_j with a fresh key $\underline{\mathsf{sk}}^0_{w_i}$.

$H_{3,i} \overset{\mathrm{c}}{\approx} H'_{3,i} \forall i \in [M]$**:** From hybrid $H_{3,i}$ to $H'_{3,i}$ we replace the active label $\sigma_{w_i}(t-1, \mathsf{sk}_{w_i^1})$ with the fresh key $\underline{\mathsf{sk}}^1_{w_i}$. Since in hybrid $H_{3,i}$ we already replaced the inactive label $\sigma_{w_i}(t-1, \mathsf{sk}_{w_i^0})$ with a fresh label, there is no longer any leakage of the key transformation function $\sigma^{(t-1)}_{w_i}$. Therefore, since $r_t \leftarrow\!\!\$\ R_{\mathsf{Rerand}}$ it holds that $\sigma^{(t-1)}_{w_i}$ is uniformly distributed over $\mathcal{F}$ and

$$\left\{\sigma_{w_i}(t-2, \mathsf{sk}^1_{w_i}), \sigma_{w_i}(t-1, \mathsf{sk}_{w_i^1})\right\} \overset{\mathrm{c}}{\approx} \left\{\sigma_{w_i}(t-2, \mathsf{sk}^1_{w_i}), \underline{\mathsf{sk}}^1_{w_i}\right\}$$

by key privacy of the gate KMHE scheme (Definition 22).

$H'_{3,i} \overset{\mathbf{c}}{\approx} H''_{3,i} \forall i \in [M]$**:** In hybrid $H'_{3,i}$ all secret keys used to create the garbled gate ciphertexts G'_j are freshly sampled keys. The adversary distinguishing between $H'_{3,i}$ and $H''_{3,i}$ obtains only the active keys. Therefore, by CPA security of the gate KMHE scheme (Definition 20), the messages not decryptable by those active keys remaining computationally hidden. Those are exactly the messages we replace in hybrid $H''_{3,i}$. Therefore, this step follows from CPA security of the gate KMHE scheme.

$H''_{3,M} \overset{\mathbf{c}}{\approx} H_4$**:** Indistinguishability of these hybrids is an immediate consequence of garbling privacy (Definition 33).

□